OpenText™ Service Manager

Upgrade

Version : 9.81

PDF Generated on : May 23, 2026

© Copyright 2026 Open Text

Table of Contents

1. Upgrade 1
1.1. Backup and backout instructions 5
1.2. Upgrade the platform components 8
1.2.1. Upgrade the Service Manager server 9
1.2.2. Upgrade the Windows client 13
1.2.3. Upgrade the web tier 14
1.2.4. Upgrade the Mobility client 16
1.2.5. Upgrade Smart Analytics 17
1.2.5.1. Upgrade Smart Analytics on Windows 19
1.2.5.2. Upgrade Smart Analytics on Linux 37
1.2.6. Upgrade Service Manager Collaboration 58
1.2.7. Install platform unloads 61
1.3. Upgrade the applications 64
1.3.1. Applications upgrade overview 65
1.3.2. Applications upgrade paths 77
1.3.3. Applications upgrade process 81
1.3.4. Planning an upgrade 85
1.3.5. Upgrade tasks on the development environment 89
1.3.5.1. Step 1: Duplicate the production environment 90
1.3.5.2. Step 2: Upgrade server and client 92
1.3.5.3. Step 3: Perform preparatory tasks 93
1.3.5.4. Step 4: Load the applications upgrade files 99
1.3.5.5. Step 5: Run the Upgrade Utility 101
1.3.5.6. Step 6: View the upgrade results 108
1.3.5.7. Step 7: Manage the upgrade result data 128
1.3.5.8. Step 8: Resolve exceptions 129
1.3.5.9. Step 9: Resolve conflicts 140

1.3.5.10. Step 10: Perform additional manual tasks 156
1.3.5.11. Step 11: Migrate Process Designer data 161
1.3.5.12. Step 12: Return the system to normal operation 168
1.3.5.13. Step 13: Test the development environment 169
1.3.5.14. Step 14: Back up the system 170
1.3.5.15. Step 15: Build a custom upgrade 171
1.3.6. Upgrade tasks on the test environment 175
1.3.7. Upgrade tasks on the production environment 180
1.3.8. Appendix A: Additional manual migration tasks on Service Manager Hybrid 182
1.3.8.1. Mandatory manual migration tasks 183
1.3.8.2. Optional manual migration tasks 196
1.3.9. Appendix B: Data migration 234
1.3.9.1. Field mapping and value mapping 235
1.3.9.2. Batch Update mode 240
1.3.9.3. Out-of-box data migration settings 242
1.3.10. Appendix C: Run the SQL compare utility 263
1.3.11. Appendix D: Mapping between legacy security profiles and current PD security roles 272
1.3.11.1. Change Management security profiles mappings 273
1.3.11.2. Service Desk security profiles mappings 278
1.3.11.3. Problem Management security profiles mappings 281
1.3.11.4. Incident Management security profiles mappings 284
1.3.11.5. Service Level Management security profiles mappings 289
1.3.11.6. Configuration Management security profiles mappings 292

1. Upgrade

This section provides instructions on how to upgrade from an existing implementation of Service Manager to the latest version. These instructions cover both platform components (such as the Service Manager server, Windows client, web tier, and Smart Analytics) and the Service Manager applications.

For instructions on performing a new installation of Service Manager, see Install.

Suggested upgrade order

For existing implementations, follow the upgrade order below:

1. Upgrade your RDBMS if needed.
2. If you are on a version earlier than 9.7x, install the Service Manager 9.80 server without loading the applications and demonstration data, and restore your old server configurations. Then, upgrade Service Manager 9.80 Server to 9.81. For details, see Upgrade the Service Manager server.
3. Install the Service Manager 9.81 Windows client and restore your old configurations. For details, see Upgrade the Windows client.
4. Install the Service Manager 9.81 web tier and restore your old configurations. For details, see Upgrade the web tier.
5. Upgrade other components (such as Mobility, Smart Analytics, and Service Manager Collaboration) as needed, and restore their old configurations if any.
6. (Recommended) Upgrade your applications to version 9.81. While it is possible to upgrade certain core platform components (Server, Windows Client, and Web Tier) and run an earlier version of the applications, many of the new features are not supported by the earlier applications. To get the benefit of all the new features, you must upgrade to the latest Service Manager applications as well. For more information, see Upgrade the applications.

   For information about the compatibility between the Service Manager platform components and applications, refer to the Service Manager support matrix.



Note

The Upgrade Utility upgrades not only your applications but also your language packs to version 9.80.

Upgrade matrix

Follow the upgrade matrix to determine which versions of Service Manager can be upgraded to version 9.81.

Server

Base version	Supported upgrade path	Rollback
9.64 to 9.72	Install the Service Manager 9.80 server, and then apply the Service Manager 9.81 server patch.	Supported

Windows client

Base version	Supported upgrade path	Rollback
9.6x to 9.80	Install the Service Manager 9.81 Windows client and restore your old configurations.	Supported

Web tier

Base version	Supported upgrade path	Rollback
9.6x to 9.80	Install the Service Manager 9.81 web tier and restore your old configurations.	Supported

Mobility client

Base version	Supported upgrade path	Rollback
9.6x to 9.80	Install the Service Manager 9.81 Mobility client and restore your old configurations.	Supported

Smart Analytics

Base version	Supported upgrade path	Rollback
9.40 and later	Upgrade to Service Manager 9.81 Smart Analytics.	Supported

Collaboration

Base version	Supported upgrade path	Rollback
9.6x to 9.80	Install the Service Manager 9.81 chat server, chat service, and chat UI. Next, restore your old configurations.	Supported

Applications

Base version	Supported upgrade path	Rollback
9.3x (No PDCP installed) 9.31 or later (PDCP3 installed) 9.4x (Classic) 9.41 (Hybrid) 9.5x (Hybrid) 9.6x (Hybrid) 9.7x (Hybrid) 9.8x (Hybrid)	Upgrade to Service Manager 9.81 Hybrid.	Supported
9.31 or later (PDCP4 installed) 9.4x (Codeless) 9.5x (Codeless) 9.6x (Codeless) 9.7x (Codeless) 9.8x (Codeless)	Upgrade to Service Manager 9.81 Codeless.	Supported

1.1. Backup and backout instructions

In case you need to restore your Service Manager system to its original state after installing the component patches in this release, make necessary backups before each patch installation. If you need a rollback, follow the restore instructions.

Server

Follow these steps to back out the server upgrade:

1. Stop the Service Manager server.
2. Execute the PatchUninstall.bat on Windows or PatchUninstall.sh on Linux (the file is located in the *<SM Installation>*_uninstall directory).

   Note

   You can only use the server patch uninstallation tool if you have applied the current patch by using the server patch installation tool. Furthermore, you can only use the tool to restore a version that was backed up by the server patch installation tool.

3. When prompted, enter the full path of the Service Manager server backup directory. A list of the available backups is displayed.
4. Select the desired backup version and confirm your selection.

   Note

   If you have a horizontally-scaled system, make sure that every server instance is replaced with its backup.

5. If you have also loaded platform unload files required for your server changes, you must also roll back the application changes made by the unload files. See **Applications**.
6. Restart the Service Manager server.

If the tool fails to back out the server patch, the details are recorded in the PatchUninstall.log file.

Applications



Tip

You can use Unload Manager (recommended) or Database Manager (if Unload Manager is not available in your application version) to roll back to your old data, as described in the following instructions.

To roll back to your old data using Unload Manager, follow these steps:

1. Go to **System Administration** > **Ongoing Maintenance** > **Unload Manager**.
2. Double-click **Apply Unload** to open the wizard.
3. Select the unload file generated in the backup process, specify a backup file, and then click **Next**. Details of the unload file are displayed.
4. Double-click a conflicting object in the table to open the merge tool:
    1. Merge the object, and then select the **Reconciled** check box.
    2. Click **Save** to return to the wizard.
5. Click **Next** after all the conflicting objects are reconciled.
6. Click **Yes** on the confirmation window to apply the backup unload.
7. Click **Finish**.

To roll back to your old data using Database Manager, follow these steps:

1. Go to Database Manager, click **More** > **Import/Load**.
2. Browse to the backup unload file you created.
3. Click **Load FG**.

Web tier

To back out the web tier upgrade, follow these steps:

1. Delete or uninstall the existing web tier.
2. Clear the cache of your web application server (for example, Tomcat).
3. Redeploy the old web tier.
4. Restore your old customizations.

Windows client

To back out the Windows client upgrade, follow these steps:

1. Uninstall the new Windows client.
2. Reinstall the previous Windows client.
3. Restore your old Windows connections and configurations.

SM Collaboration

To back out the SM Collaboration upgrade, follow these steps:

1. Remove the new Openfire installation folder.
2. Restore the renamed Openfire folder.

1. Stop the Service Manager server.
2. Run `PatchUninstall.bat` on Windows or `PatchUninstall.sh` on Linux (the file locates in the `<SM Installation>\_uninstall` directory).

   Note

   You can only use the server patch uninstallation tool if you have applied the current patch by using the server patch installation tool. Furthermore, you can only use the tool to restore a version backed up by the server patch installation tool.

3. When prompted, enter the full path of the Service Manager server backup directory. The system displays a list of the available backups.
4. Select the desired backup version and confirm your selection.

   Note

   If you have a horizontally scaled system, make sure you have replaced every server instance with its backup.

5. If you have also loaded platform unload files required for your server changes, you must also roll back the application changes made by the unload files. See the *Applications* section below.
6. Restart the Service Manager server.

If the tool fails to restore from the server patch, the system records the details in the `PatchUninstall.log` file.

1.2. Upgrade the platform components

Before you can upgrade the applications, you must upgrade the Service Manager Server and Windows Client. Additionally, you may also want to or need to upgrade other components, such as the Web Tier, or Smart Analytics. In this section, "platform components" refers to the Service Manager components other than the applications.



Note

For an RDBMS upgrade, see your specific database documentation. For applications upgrade, see Upgrade the applications.

A platform upgrade may include the following components.

Upgrade the Service Manager Server

Upgrade the Windows Client

Upgrade the Web Tier

Upgrade the Mobility Client

Upgrade Smart Analytics

Upgrade Service Manager Collaboration

Install platform unloads

1.2.1. Upgrade the Service Manager server

Upgrade from a version earlier than 9.80

To upgrade the Service Manager Server from a version earlier than 9.80 to version 9.81, you must first install Service Manager 9.80, and then upgrade to Service Manager 9.81. To do this, follow these steps:

1. Before you install the Service Manager Server, make a backup of your existing Server installation folder.

   > **Note**
   >
   > If you have a horizontally scaled system, be sure to back up the server installation folder for each server instance.

2. Install the Service Manager 9.80 Server. For detailed steps, see the "Install the Service Manager 9.80 Server" section in Install the Service Manager Server. At this point, perform the server installation only and skip the database configuration task, which you will need to perform after applying the Service Manager 9.81 Server patch.
3. Merge your old Service Manager Server configuration files in the Server's RUN folder, including the **sm.ini** and **sm.cfg** files.

   > **Note**
   >
   > If you have changed the smmasterkey storage location in previous version (in this case, the sm.ini file doesn't include the smmasterkey parameter, and the keystorage parameter is flagged as 1), you need to firstly use the sm -keyexport command to export the master key, then use the sm -keyimport command to migrate the key to the upgraded system. See Import the master key.

4. If you have configured SSL or single sign-on, you must copy the following files from the previous installation folder to the new installation folder (your own keystore file names may differ):

   - <SM_install>\Server\RUN\cacerts
   - <SM_install>\Server\RUN\server.keystore

- <SM_install>\Server\RUN\trustedclients.keystore

5. Restart the Service Manager server.
6. Follow the steps in the **Upgrade from version 9.80** section to upgrade the Server to version 9.81.

Check and update your custom JavaScript code

Important

The embedded Mozilla SpiderMonkey JavaScript engine was upgraded to a higher version in the 9.80 server. In order to avoid any potential problems, you need to check your existing scripts and pay attentions when you add new scripts. See JavaScript tailoring guideline.

Upgrade from version 9.80

To upgrade the Service Manager Server from version 9.80, you can use a patch installation tool or perform the upgrade manually. In addition, you need to upgrade IR Expert if you have not already done so in a previous upgrade.

Install the rng-tools package

Before you start, you need to ensure that the the rngd service is enabled in the system if your Service Manager Server is installed on a Linux system, .

The **rng-tools** package can increase the entropy collected into the `/dev/random` file and improve the performance of Service Manager.

If you haven't installed the **rng-tools** package, run the following command to install:

- For RedHat or Oracle Linux:
  `sudo yum install rng-tools`
- For SUSE:
  `sudo zypper install rng-tools`

After you install the **rng-tools** package, run the `systemctl start rngd` command to start the rngd service.

Upgrade the server by using the server patch installation tool

To upgrade the Service Manager server from version 9.80 to version 9.81, follow these steps:

1. Stop all Service Manager clients.
2. Stop the Service Manager Server.

3. Extract the Server patch (`sm9.81.XXXX_Windows_Server.zip` or `sm9.81.XXXX_Linux.tar.xz`) to a directory on the Server host, and then do the following:
   Note: You may use the `tar -xvf` command to extract the .tar.xz file for Linux.
   - **Windows**: Run the PatchSetup.bat file.
   - **Linux**: Run the PatchSetup.sh file. The file is located in the Service Manager 9.81 Server patch directory. If you do not have an OpenJDK JRE or Oracle JRE installed, you cannot run the PatchSetup.sh file. Instead, you need to upgrade the server manually.

   > **Note**
   > Running these tools will overwrite the configuration files (such as the LWSSO configuration file and XML files for Jgroups) in the RUN directory in the Service Manager server installation directory. Remember to reapply any changes that you have made to these files after you run the patch installation or uninstallation tool.

4. When prompted, enter the full path of the current Service Manager server installation directory and the full path of the Service Manager server backup directory.
5. Wait for the tool to run. When it has run successfully, the message "Finished applying the SM Server patch" is displayed. If the tool cannot run successfully, an error message is displayed and the details are recorded in the PatchSetup.log file.
6. If you have made any customizations or changes to the original RUN/tomcat folder, restore them in the new RUN/tomcat folder.
7. Your old schemastub.xml file (in the *<SM_Server_Home>*\RUN\km\styles\ directory) has been updated to a newer version. Either keep your old file by copying it back or keep the updated version (a full reindex for the knowledgebases is then required).
8. Run the **sm -unlockdatabase** command.

   > **Note**
   > This step is required only the first time you upgrade to 9.30p4 or later; it is also required whenever you change the server's IP address after your upgrade to 9.30p4 or later. The purpose of this step is to prevent stale license information from being kept in the system. In a scaling implementation, you can run this command from any one of your servers.

9. Restart the Service Manager server.
10. Restart the Service Manager clients.
11. Verify the server version by using either of the following methods:
    - From the Windows client, click **Help** > **About Service Manager Server**.
    - From the server's RUN folder, run the **sm -version** command.

Upgrade the server manually

Instead of running the server patch installation tool, you may still follow the manual steps used to upgrade earlier versions of Service Manager. However, if you perform the upgrade manually, you will not be able to use the server patch uninstallation tool to back out the patch.

Follow these steps to upgrade the server manually:

1. Stop the Service Manager server.
2. Backup the Server folder.
3. Delete the RUN/tomcat directory.
4. Delete the RUN/lib directory.
5. Delete the RUN/jre directory. This is to avoid conflicts between the old JRE and the new JRE.
6. Extract the compressed files for your operating system to the main Service Manager directory on the server.
7. (For Linux system only) Execute removeLinks.sh and setupLinks.sh in the RUN folder to remove old soft links and setup the new soft links.
8. If you have made any customizations or changes to the original RUN/tomcat folder, restore them in the new RUN/tomcat folder.

(Optional) Upgrade IR Expert

- If you are using the legacy IR search (version before 9.62), you may run the IR Upgrade utility to enable the new IR solution. See Upgrade the legacy IR to the new IR solution.
- If you are using the new IR solution, after upgrading applications or after adding IR keys, you need to regenerate the IR indexes. See Regenerate the IR indexes.

1.2.2. Upgrade the Windows client

To upgrade the Windows client, you need to install the new Windows client. To do this, follow these steps:

1. Make a backup copy of the following items of your existing Windows client.
    - Your Windows client home folder. For example, `C:\Users\<username>\ServiceManager`. This folder stores your connections and personalized settings.

      Note

      This is the out-of-the-box home directory, and could differ from yours if you made changes to `<Client>\configuration\config.ini` file. If so, back up the files from the location specified in that file.

    - Your certificate configuration files if any (**Window** > **Preferences** > **Service Manager** > **Security**). For example, your CA certificates file and client keystore file.
    - The following folder: `<Client>\plugins\com.hp.ov.sm.client.eclipse.user_x.xx.xxxx\src\resources\icons\obj16\`.
2. Uninstall the existing Windows client. The system retains your connection and personalized settings.
3. Install the new Windows client. For detailed steps, see Install the Service Manager Windows Client.
4. Copy the backup items of your old Windows client to the new Windows client, and restore the configurations.
5. Check the version in **Help** > **About Service Manager**.

1.2.3. Upgrade the web tier

Tip:

You can define all context parameters of the web application and initialization parameters of Servlets from both the web.xml file and the webtier.properties file, except that you can define the <customize-folder> parameter only in the web.xml file. The parameters configured in these files take precedence in the following order:

- Custom webtier.properties located in the <customize-folder>/config folder
- Default webtier.properties located in the <web tier>/WEB-INF folder
- The web.xml file located in the <web tier>/WEB-INF folder

You can define web tier parameters in your custom webtier.propteries file to easily restore your web tier configurations during an upgrade.

To upgrade the Service Manager web tier, follow these steps:

1. Before deploying the new web tier, make a backup of the following items:
    - web.xml file
    - application-context.xml
    - log4j2.properties
    - splash screen
    - style sheets
    - The folder that's defined in the *customize-folder* parameter in the web.xml file or the <web tier>\WEB-INF\webtier.properties file if the *customize-folder* parameter isn't specified
    - any other customizations you made, including your webtier-<*version*>.war file.
2. Delete or uninstall the existing web tier .war file.
3. Clear the cache of your web application server.
4. Deploy the new webtier-x.xx.war file. For details, see Install the Service Manager web tier.



Note

It's best practice to deploy with a unique context root. For example, /webtier-9.80.

5. Use a diff utility to compare the new web tier's web.xml and webtier.properties files with your backed-up version to make sure that any new parameters are properly merged into the files used in your final deployment. Do this for **application-context.xml** as well as any other files you may have customized (such as style sheets and splash screens).
6. Make any new customizations that are necessary for your deployment.
7. Restart the web application server.
8. Check the version by clicking the company logo (**About Service Manager**) icon.



Caution

After you upgrade the web tier to version 9.80, if you have deployed the web tier behind a reverse proxy or a load balancer, you must set the *X-Forwarded-Proto* and the *X-Forwarded-Host* request headers appropriately in the reverse proxy or in the load balancer. For detailed instructions, see Configure a web server to redirect requests to the web tier.

1.2.4. Upgrade the Mobility client

To upgrade the Mobility client (also referred to as the "Mobile Applications Client"), follow these steps:

1. Make a backup copy of your old Mobility client deployment folder.
2. Deploy the new Mobility client. For detailed steps, see Install the Mobile Applications client.
3. Restore your old configurations in the new deployment.

1.2.5. Upgrade Smart Analytics

Upgrade path for Smart Analytics installer 9.80

Smart Analytics Installer Source Version	How to upgrade to version 9.80
9.34	Uninstall and reinstall
9.35	Uninstall and reinstall
9.40	Upgrade using new installer
9.41	Upgrade using new installer
9.50	Upgrade using new installer
9.51	Upgrade using new installer
9.52	Upgrade using new installer
9.60	Upgrade using new installer
9.64.p1	The Smart Analytics 9.71.p1 installer is basically the same as the 964.p1 installer, except that patch 1 includes a Confluence connector. Hence, run the 9.71.p1 installer only if you need a Confluence connector.
9.70	The Smart Analytics 9.71.p1 installer is basically the same as the 9.70 installer, except that patch 1 includes a Confluence connector. Hence, run the 9.71.p1 installer only if you need a Confluence connector.
9.71.p1	Upgrade using new installer

Related topics

Upgrade Smart Analytics on Windows
Upgrade Smart Analytics on Linux

1.2.5.1. Upgrade Smart Analytics on Windows

If you have installed an earlier version of Smart Analytics, follow the instructions in this section to upgrade to Smart Analytics that is shipped with Service Manager 9.81.

To upgrade your Smart Analytics, follow these steps:

1. Before you upgrade your Smart Analytics, complete these tasks:
    1. Make sure that all the index tasks are finished.
    2. Stop and then delete all the Smart Analytics services.

       Note To delete a service, run `sc delete "SERVICE_NAME"` in the command window.
    3. Back up all the Smart Analytics files.
2. Obtain the Smart Analytics installer for Windows.
3. Unpack the .zip file and then double-click the setup application (`setupSmartAnalyticsWindowsX64.exe`).
4. Read the introduction, and then click **Next**.
5. Read the License Agreement. To continue the installation, select **I accept the terms of the License Agreement**, and then click **Next**.
6. Select **Upgrade**, and then click **Next**.
7. Review the message on the Preparation for Upgrade dialog box, and then click **Next**.
8. Specify the version of Smart Analytics installed by selecting **Smart Analytics 9.40**, **Smart Analytics 9.41**, or **Smart Analytics 9.50 or later**, and then click **Next**.
9. Continue with prompted upgrade steps, which vary based on the Smart Analytics version you selected in the previous step. See the following sections below.
    - Smart Analytics 9.40
    - Smart Analytics 9.41
    - Smart Analytics 9.50 or later
10. Wait for the upgrade to complete, and then click **Done**.

Smart Analytics 9.40

If you are upgrading from **Smart Analytics 9.40**, continue with these steps:

1. Choose the current architecture of the Smart Analytics installed:
    - Standalone
    - Distributed
2. Specify the IP address of the machine on which you have installed Service Manager server.

    You need to specify the IP addresses (or host names) of the Service Manager servers that are permitted to send administrative and query actions to the Smart Analytics servers. Use commas to separate multiple addresses (do not use a space before or after a comma).

    Note use a valid FQDN or IP address for the server address. Do not use the localhost or 127.0.0.1.
3. Select the components that you want to upgrade. The servers that are available for upgrade vary depending on the Smart Analytics architecture you specified:

    Note

    - For Smart Analytics components that were installed in the same root folder, upgrade them at the same time in a single upgrade process. Otherwise, components may not be uninstalled completely when you uninstall Smart Analytics.
    - Upgrade the components separately by performing the upgrade process multiple times under the following conditions:
        - Your current version of Smart Analytics is 9.34
        - The Smart Analytics components to be upgraded are installed in different parent folders or physical machines

Architecture	Components Available for Upgrade
Standalone	IDOL Server Media Server
Distributed	Smart Analytics Proxy Content Server Media Server Media Proxy Server

4. Specify the parent folder where the selected Smart Analytics components are installed, and then click **Next**.

   Note

   - If you are upgrading the Smart Analytics proxy server, the parent folder should include the IDOL folder.
   - If you are upgrading a distributed content server, the folder should be the parent of the content folder.

5. Follow the on-screen steps to configure the components you selected. Click **Next** after each step.

   Note The number of copies (that is, the value of Replicas plus one) must be no more than the number of child servers. If you create more copies than there are child servers, DIH does not start.

6. Review the pre-installation summary, and then click **Install**. If you want to change your configuration, click **Previous**.

Post-upgrade actions

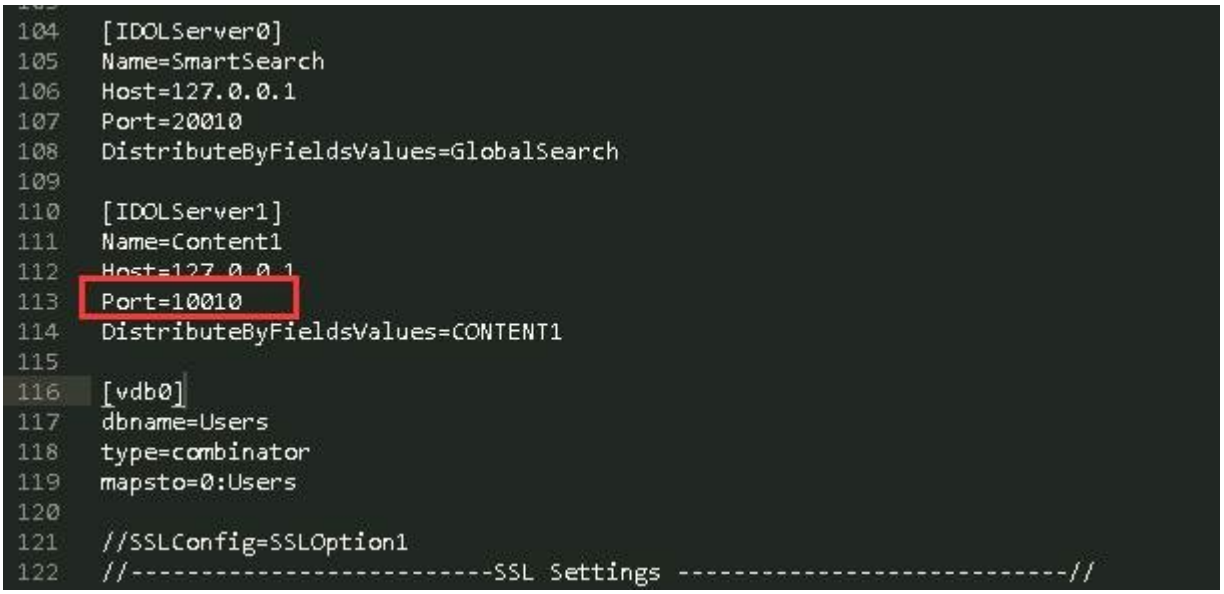
Perform the following actions after you upgrade your Smart Analytics:

- When the installation is complete, check the Start.bat and Stop.bat files under the <Smart Analytics Installation>/Scripts folder for any commands that start or stop services not on your local machine. Delete or comment out such commands and then add `sc start "[Service Name of your previous content]"` or `sc stop "[Service Name of your previous content]"` as needed.

- If you upgraded from Smart Analytics 9.40 to Smart Analytics 9.81, then during the upgrade process, the existing configuration files would be renamed to `<component name>.cfg.bak`. After you finish the upgrade, compare it with the new configuration file. If there is any customized change made in the previous .cfg file and it is still valid, manually add it back to the new .cfg file.
- If you upgraded from Smart Analytics 9.40 to Smart Analytics 9.81, you need to re-index the Hot Topic Analytics data. However, do not start training for Smart Ticket.
- If you upgraded from a distributed Smart Analytics from 9.40 to Smart Analytics 9.81, add the following lines before the `[actions]` section in the *<Smart Analytics Installation>*/Content#/Content#.cfg file for the system to clean up expired documents:

  ```
  //--------------------------- Schedules --------------------
  --//
  //expire and compact data everyday
  [Schedule]
  Expire=TRUE
  ExpireTime=23:00
  ExpireInterval=24:00
  ```

  Note With this configured, the system automatically deletes all the expired documents at 23:00 every day. By default, the expiration date for documents is set to 365 days.
- If you upgraded from a standalone Smart Analytics from 9.40 to Smart Analytics 9.81, you need to manually modify the following configuration in the <SmartAnalytics>/IDOL/IDOLServer.cfg file:



```
104  [IDOLServer0]
105  Name=SmartSearch
106  Host=127.0.0.1
107  Port=20010
108  DistributeByFieldsValues=GlobalSearch
109
110  [IDOLServer1]
111  Name=Content1
112  Host=127.0.0.1
113  Port=10010
114  DistributeByFieldsValues=CONTENT1
115
116  [vdb0]
117  dbname=Users
118  type=combinator
119  mapsto=0:Users
120
121  //SSLConfig=SSLOption1
122  //---------------------------SSL Settings ----------------------------//
```

- If you upgraded a distributed Smart Analytics proxy server from 9.40 to Smart Analytics 9.81, the previous configuration files would be renamed to `<component name>.cfg.bak`, and you need to manually modify the following configurations in the *<SmartAnalytics>*/IDOL/IDOLServer.cfg file:

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
VirtualDatabases=1	VirtualDatabases=<x>	VirtualDatabases=<x>
[vdb0] dbname=Users type=combinator mapsto=0:Users	[VDB0] DbName=News Type=Combinator MapsTo=0:News	Do not change

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
	[VDB<x>] DbName=my_database Type=Combinator MapsTo=<y>:my_database Note The number of x starts from 1.	[VDB<x>] DbName=my_database Type=Combinator MapsTo=<y+1>:my_database ◦ You must add the VDB configurations by their number sequence. ◦ There may be other content between two VDB configuration sections. ◦ Make sure the total number of the VDB configuration sections equals to the number specified in the VirtualDatabases parameter. ◦ There is no difference if you use the upper or lower case for the section name of [VDB<x>].

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
[DistributionIDOLServers] Number=2	[DistributionIDOLServers] Number=<x>	[DistributionIDOLServers] Number=<x+1>
[IDOLServer0] Name=SmartSearch Host=127.0.0.1 Port=20010 DistributeByFieldsValues=GlobalSearch		Do not change

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
[IDOLServer1] Name=Content1 Host=127.0.0.1 Port= DistributeByFieldsValues=CONTENT1	[IDOLServer<x>] Host=12.3.4.56 Port=20010 DistributeByFieldsValues=CONTENT<x> Note The number of x starts from 0.	[IDOLServer<x+1>] Name=Content<x+1> Host=12.3.4.56 Port=20010 DistributeByFieldsValues=CONTENT<x+1> Note ◦ You must add the IDOL server configurations by their number sequence. ◦ Make sure the total number of the IDOL server configuration sections equals to the number specified in the [DistributionIDOL Servers] Number parameter.

- After upgrading your Smart Analytics, you need to manually add `*/TITLE_INDEX` to the following files if you have scaled out the Content server:
  - <SmartAnalytics>/Content2/Content2.cfg
  - <SmartAnalytics>/Content3/Content3.cfg
  - <SmartAnalytics>/IDOL/IDOLServer.cfg

  You also need to modify the scaled content to add `*/TITLE_INDEX`.

Existing configurations	You need to modify the configuration to

`[SetTitleFields]`
`// The one field per document that is to be stored`
`as the autn:title value`
`Property=TitleFields`
`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
`*/TITLE_WEIGHT*`
	`[SetTitleFields]`
	`// The one field per document that is to be stored`
	`as the autn:title value`
	`Property=TitleFields`
	`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
	`*/TITLE_WEIGHT*,*/TITLE_INDEX`

Smart Analytics 9.41

If you are upgrading from **Smart Analytics 9.41**, continue with these steps:

1. Specify the parent folder where the current Smart Analytics is installed, and then click **Next**.
2. The installer automatically detects and lists which components have been installed. Review the detected components, and then click **Next**.
3. Specify content folder names as needed. By default, the installer automatically specifies all the content folder names.
4. Specify media server folder names as needed. By default, the installer automatically specifies all the media server folder names.
5. Review the message that Smart Analytics is ready to upgrade, and then click **Next**.
6. Check the pre-installation summary, and then click **Install**. If you want to change your configuration, click **Previous**.

Post-upgrade actions

Perform the following actions after you upgrade your Smart Analytics:

- When the installation is complete, check the Start.bat and Stop.bat files under the <Smart Analytics Installation>/Scripts folder for any commands that start or stop services not on your local machine. Delete or comment out such commands and then add `sc start "[Service Name of your previous content]"` or `sc stop "[Service Name of your previous content]"` as needed.

- If you upgrade from Service Manager 9.41 to Service Manager 9.81, you need to perform full re-index of Catalog_Library after you upgrade to Smart Analytics 9.81.
- If you upgraded from a distributed Smart Analytics from 9.41 to Smart Analytics 9.81, add the following lines before the `[actions]` section in the *<Smart Analytics Installation>*/Content#/Content#.cfg file for the system to clean up expired documents:

  ```
  //--------------------------- Schedules --------------------
  --//
  //expire and compact data everyday
  [Schedule]
  Expire=TRUE
  ExpireTime=23:00
  ExpireInterval=24:00
  ```

  Note With this configured, the system automatically deletes all the expired documents at 23:00 every day. By default, the expiration date for documents is set to 365 days.

- If you upgrade from Smart Analytics 9.41 to Smart Analytics 9.81, you need to manually modify the following configuration sections:

  In the *<SmartAnalytics>*/IDOL/IDOLServer.cfg file:

Existing configurations	You need modify the configuration to

[IndexQueue]
IndexQueueInitialSize=10240
IndexQueueMaxHistorySize=500
IndexQueueMaxPendingItems=10240
[SetDateFields]
// Fields containing the document date
Property=DateFields
PropertyFieldCSVs==*/DREDATE,*/DREDATE_*,*/DATE,*/DATE_*,*/*_DATE,*/*_TIME
[Logging]
LogExpireAction=compress
LogOldAction=move
LogMaxSizeKBs=20480
	[IndexQueue]
	IndexQueueInitialSize=10240
	IndexQueueMaxHistorySize=10240
	IndexQueueMaxPendingItems=0
	[SetDateFields]
	// Fields containing the document date
	Property=DateFields
	PropertyFieldCSVs=*/DREDATE,*/DREDATE_*,*/DATE,*/DATE_*,*/*_DATE,*/*_TIME,*/MODIFIED_EPOCH,*/LASTMODIFIED
	[Logging]
	LogExpireAction=Day
	LogMaxOldFiles=7
	LogOldAction=delete
	LogMaxSizeKBs=102400

In the *<SmartAnalytics>*/Level2proxy/IDOLServer.cfg file:

Existing configurations	You need modify the configuration to

[Server]
[IndexQueue]
IndexQueueInitialSize=10240
IndexQueueMaxHistorySize=500
IndexQueueMaxPendingItems=10240
[Logging]
LogExpireAction=compress
LogOldAction=move
LogMaxSizeKBs=20480
	[Server]
	//store index data on disk only when the index cache is full or when the MaxSyncDelay(120 seconds by default) is reached.
	DelayedSync=TRUE
	[IndexCache]
	IndexCacheMaxSize=102400
	[IndexQueue]
	IndexQueueInitialSize=10240
	IndexQueueMaxHistorySize=10240
	IndexQueueMaxPendingItems=0
	[Logging]
	LogExpireAction=Day
	LogMaxOldFiles=7
	LogOldAction=delete
	LogMaxSizeKBs=102400

In the *<SmartAnalytics>*/Content<x>/Content<x>.cfg file:

Existing configurations	You need modify the configuration to

[IndexQueue]
IndexQueueInitialSize=10240
IndexQueueMaxHistorySize=500
IndexQueueMaxPendingItems=10240
[SetDateFields]
// Fields containing the document date
Property=DateFields
PropertyFieldCSVs=*/DREDATE,*/DREDATE_*,*/DATE,*/DATE_*,*/*_DATE,*/*_TIME
[SetMatchFields]
PropertyFieldCSVs=*/*_MATCH,*/EDK_*,*/*_MATCH_INDEX,*/*_MATCH_WEIGHT*,*/*_BOOLEAN
[Logging]
LogExpireAction=compress
LogOldAction=move
LogMaxSizeKBs=20480
[LanguageTypes]
GenericTransliteration=TRUE
Transliteration=false
	//The amount of memory to allocate to store the most commonly occurring terms. (using 500MB as term cache here, recommended range is 10240 to 2048000.)
	[TermCache]
	TermCachePersistentKB=512000
	//------------------------ Schedules ----------//
	//expire and compact data everyday
	[Schedule]
	Expire=TRUE
	ExpireTime=23:00
	ExpireInterval=24:00
	[IndexQueue]
	IndexQueueInitialSize=10240
	IndexQueueMaxHistorySize=10240
	IndexQueueMaxPendingItems=0
	[SetDateFields]
	// Fields containing the document date
	Property=DateFields
	PropertyFieldCSVs=*/DREDATE,*/DREDATE_*,*/DATE,*/DATE_*,*/*_DATE,*/*_TIME,*/MODIFIED_EPOCH,*/LASTMODIFIED
	[SetMatchFields]
	PropertyFieldCSVs=*/*_MATCH,*/EDK_*,*/*_MATCH_INDEX,*/*_MATCH_WEIGHT*,*/*_BOOLEAN,*/IMPORTMAGICEXTENSION

Existing configurations	You need modify the configuration to

	`[Logging]`
	`LogExpireAction=Day`
	`LogMaxOldFiles=7`
	`LogOldAction=delete`
	`LogMaxSizeKBs=102400`
	`[LanguageTypes]`
	`GenericTransliteration=TRUE`
	`Transliteration=true`

After you modify the configurations, restart all related Smart Analytics components of which such configuration are modified. Then, on the machine where level2proxy is located, reload updated configuration files dynamically and regenerate match index by visiting the following URLs :

- Reload updated configuration files dynamically:

  `http://localhost:20011/DRERESET`

- Regenerate the match fields:

  `http://localhost:20011/DREREGENERATE?Type=Match`

  Wait for a few minutes for the regeneration to complete.

- After upgrading your Smart Analytics, you need to manually add `*/TITLE_INDEX` to the following files if you have scaled out the Content server:
  - <SmartAnalytics>/Content2/Content2.cfg
  - <SmartAnalytics>/Content3/Content3.cfg
  - <SmartAnalytics>/IDOL/IDOLServer.cfg

  You also need to modify the scaled content to add `*/TITLE_INDEX`.

Existing configurations	You need to modify the configuration to

`[SetTitleFields]`
`// The one field per document that is to be stored`
`as the autn:title value`
`Property=TitleFields`
`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
`*/TITLE_WEIGHT*`
	`[SetTitleFields]`
	`// The one field per document that is to be stored`
	`as the autn:title value`
	`Property=TitleFields`
	`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
	`*/TITLE_WEIGHT*,*/TITLE_INDEX`

Smart Analytics 9.50 or later

If you are upgrading from **Smart Analytics 9.50 or later**, continue with these steps:

1. Specify the parent folder where the current Smart Analytics is installed, and then click **Next**.
2. The installer automatically detects and lists which components have been installed. Review the detected components, and then click **Next**.
3. Specify content folder names as needed. By default, the installer automatically specifies all the content folder names.
4. Specify media server folder names as needed. By default, the installer automatically specifies all the media server folder names.
5. Review the message that Smart Analytics is ready to upgrade, and then click **Next**.
6. Check the pre-installation summary, and then click **Install**. If you want to change your configuration, click **Previous**.

Post-upgrade actions

Perform the following actions after you upgrade your Smart Analytics:

- When the installation is complete, check the Start.bat and Stop.bat files under the <Smart Analytics Installation>/Scripts folder for any commands that start or stop services not on your local machine. Delete or comment out such commands and then add `sc start "[Service Name of your previous content]"` or `sc stop "[Service Name of your previous content]"` as needed.

- If you have installed Portal Content in Smart Analytics 9.50 or Smart Analytics 9.51, you need to manually modify the following configuration sections:

  In the *<SmartAnalytics>*/IDOL/IDOLServer.cfg file:

Existing configurations	You need to modify the configuration to
`//The following parameter is required for the DAH if mirrormode is set to false VirtualDatabases=<x>`	`//The following parameter is required for the DAH if mirrormode is set to false VirtualDatabases=<x+1>`

  Add the following configuration:

```
[vdb<x>]
dbname=SearchHistory
type=combinator
mapsto=2:SearchHistory
```

  Note

  - You must add the VDB configurations by their number sequence.
  - There may be other content between two VDB configuration sections.
  - Make sure the total number of the VDB configuration sections equals to the number specified in the VirtualDatabases parameter.
  - There is no difference if you use the upper or lower case for the section name of [VDB<x>].

  In the *<SmartAnalytics>*/Content-SMSP/Content-SMSP.cfg file:

Existing configurations	You need to modify the configuration to
`[Databases] NumDBs=<x>`	`[Databases] NumDBs=<x+1>`

  Add the following configuration:

```
[Database<x>]
Name=SearchHistory
```

- After upgrading your Smart Analytics, you need to manually add `*/TITLE_INDEX` to the following files if you have scaled out the Content server:
  - <SmartAnalytics>/Content2/Content2.cfg
  - <SmartAnalytics>/Content3/Content3.cfg
  - <SmartAnalytics>/IDOL/IDOLServer.cfg

  You also need to modify the scaled content to add `*/TITLE_INDEX`.

Existing configurations	You need to modify the configuration to

`[SetTitleFields]`
`// The one field per document that is to be stored`
`as the autn:title value`
`Property=TitleFields`
`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
`*/TITLE_WEIGHT*`
	`[SetTitleFields]`
	`// The one field per document that is to be stored`
	`as the autn:title value`
	`Property=TitleFields`
	`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
	`*/TITLE_WEIGHT*,*/TITLE_INDEX`

1.2.5.2. Upgrade Smart Analytics on Linux

If you have installed an earlier version of Smart Analytics, follow the instructions in this section to upgrade to Smart Analytics that is shipped with Service Manager 9.81.

To upgrade your Smart Analytics on a Linux system, follow these steps:

1. Before you upgrade your Smart Analytics, complete these tasks:
    1. Make sure that all the index tasks are finished.
    2. Stop all the Smart Analytics services.
    3. Back up all the Smart Analytics files.
2. Obtain the Smart Analytics installer for Linux.
3. Unpack the .zip file and then execute the binary file (`setupSmartAnalyticsLinuxX64.bin`) from the command line.
4. Read the introduction, and then press **Enter**.
5. Read the License Agreement, and then press **Enter** repeatedly until you see **DO YOU ACCEPT THE TERMS OF THIS LICENSE AGREEMENTS? (Y/N)**.
6. Type **Y**, and then press **Enter**.
7. Type **2** to select Upgrade, and then press **Enter**.
8. Choose the version of Smart Analytics installed by typing the corresponding number, and then press **Enter**:
    - 1-Smart Analytics 9.40
    - 2-Smart Analytics 9.41
    - 3-Smart Analytics 9.50 or later
9. Continue with on-screen instructions, which vary based on the Smart Analytics version you selected in the previous step. See the following sections below.
    - Smart Analytics 9.40
    - Smart Analytics 9.41
    - Smart Analytics 9.50 or later
10. Wait for the installation to complete, and then press **Enter** to exit the installer.

Smart Analytics 9.40

If you select **Smart Analytics 9.40**, continue with these steps:

1. Choose the current architecture of the Smart Analytics installed typing the corresponding number, and then press **Enter**:

- 1-Standalone
- 2-Distributed

2. Specify the IP address of the machine on which you have installed Service Manager server, and then press **Enter**.

   You need to specify the IP addresses (or hostnames) of the Service Manager servers that are permitted to send administrative and query actions to the Smart Analytics servers. Use commas to separate multiple addresses (do not use a space before or after a comma).

   Note

   Use a valid FQDN or IP address for the server address. Do not use the localhost or 127.0.0.1.

3. Select the components that you want to upgrade by typing the corresponding numbers. Use commas to separate multiple choices (do not use a space before or after a comma). The servers that are available for upgrade vary depending on the Smart Analytics architecture you specified.

   Note

   - For Smart Analytics components that were installed in the same root folder, upgrade them at the same time in a single upgrade process. Otherwise, components may not be uninstalled completely when you uninstall Smart Analytics.
   - Upgrade the components separately by performing the upgrade process multiple times under the following conditions:
     - Your current version of Smart Analytics is 9.34
     - The Smart Analytics components to be upgraded are installed in different parent folders or physical machines

Architecture	Components Available for Upgrade
Standalone	IDOL Server Media Server
Distributed	Smart Analytics Proxy Content Server Media Server Media Proxy Server

4. Specify the parent folder where the selected Smart Analytics components are installed, and then press **Enter**.

   Note
   - If you are upgrading the Smart Analytics proxy server, the folder should include the IDOL folder.
   - If you are upgrading a distributed content server, the folder should be the parent of the content folder.

5. Follow the configuration steps to configure the components you selected. Press **Enter** after each configuration step.

   Note

   The number of copies (that is, the value of Replicas plus one) must be no more than the number of child servers. If you create more copies than there are child servers, DIH does not start.

6. Check the pre-upgrade summary, and then press **Enter**.

Post-upgrade actions

Perform the following actions after you upgrade your Smart Analytics:

- When the installation is complete, do the following:

    - Check the StartAll.sh and StopAll.sh files under <Smart Analytics Installation>/Scripts folder for any commands that start or stop services not on your local machine. Delete or comment out such commands and then add `source ./Start[Service Name of your previous content]` or `source ./Stop[Service Name of your previous content]` as needed.
    - Delete all the start[Name of Service not on local machine].sh and stop [Name of Service not on local machine].sh files under <Smart Analytics Installation>/Scripts folder.

- If you upgrade from Smart Analytics 9.40 to Smart Analytics 9.81, then during the upgrade process, the existing configuration files would be renamed to `<component name>.cfg.bak`. After you finish the upgrade, compare it with the new configuration file. If there is any customized change made in the previous .cfg file and it is still valid, manually add it back to the new .cfg file.
- If you upgraded from Smart Analytics 9.40 to Smart Analytics 9.81, you need to re-index the Hot Topic Analytics data. However, do not start training for Smart Ticket.
- If you upgraded a distributed Smart Analytics proxy server with version 9.40, the previous configuration files would be renamed to `<component name>.cfg.bak`, and you need to manually modify the following configurations in the *<SmartAnalytics>*/IDOL/IDOLServer.cfg file:

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
VirtualDatabases=1	VirtualDatabases=<x>	VirtualDatabases=<x>
[vdb0] dbname=Users type=combinator mapsto=0:Userss	[VDB0] DbName=News Type=Combinator MapsTo=0:News	Do not change

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
	[VDB<x>] DbName=my_database Type=Combinator MapsTo=<y>:my_database **Note** The number of x starts from 1.	[VDB<x>] DbName=my_database Type=Combinator MapsTo=<y+1>:my_database

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
		Note ◦ You must add the VDB configurations by their number sequence. ◦ There may be other content between two VDB configuration sections. ◦ Make sure the total number of the VDB

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
		configuration sections equals to the number specified in the Virtual Databases parameter. ◦ There is no difference if you use the upper or lower case for the section name of [VDB <x>].

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
[DistributionIDOLServers] Number=2	[DistributionIDOLServers] Number=<x>	[DistributionIDOLServers] Number=<x+1>
[IDOLServer0] Name=SmartSearch Host=127.0.0.1 Port=20010 DistributeByFieldsValues=GlobalSearch		Do not change

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
[IDOLServer1] Name=Content1 Host=127.0.0.1 Port= DistributeByFieldsValues=CONTENT1	[IDOLServer<x>] Host=12.3.4.56 Port=20010 DistributeByFieldsValues=CONTENT<x> **Note** The number of x starts from 0.	[IDOLServer<x+1>] Name=Content<x+1> Host=12.3.4.56 Port=20010 DistributeByFieldsValues=CONTENT<x+1>

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
		Note ◦ You must add the IDOL server configurations by their number sequence. ◦ Make sure the total number of the IDOL server configuration sections equals to the number specified in the

Configurations after upgrade	Configurations in your previous file (<component name>.cfg.bak)	You need to modify the value to
		[Distribution IDOLServers] Number parameter.

- If you upgraded from a distributed Smart Analytics to Smart Analytics 9.81, add the following lines before the [actions] section in the `content[x].cfg file` for the system to clean up expired documents:

```
//--------------------------- Schedules --------------------
--//
//expire and compact data everyday
[Schedule]
Expire=TRUE
ExpireTime=23:00
ExpireInterval=24:00
```



Note

With this configured, the system automatically deletes all the expired documents at 23:00 every day. By default, the expiration date for documents is set to 365 days.

- After upgrade your Smart Analytics, you need to manually add `*/TITLE_INDEX` to the following files if you have scaled out the Content server:
  - <SmartAnalytics>/Content2/Content2.cfg
  - <SmartAnalytics>/Content3/Content3.cfg
  - <SmartAnalytics>/IDOL/IDOLServer.cfg

You also need to modify the scaled content to add `*/TITLE_INDEX`.

Existing configurations	You need to modify the configuration to

`[SetTitleFields]`
`// The one field per document that is to be stored`
`as the autn:title value`
`Property=TitleFields`
`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
`*/TITLE_WEIGHT*`
	`[SetTitleFields]`
	`// The one field per document that is to be stored`
	`as the autn:title value`
	`Property=TitleFields`
	`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
	`*/TITLE_WEIGHT*,*/TITLE_INDEX`

Smart Analytics 9.41

If you select **Smart Analytics 9.41**, continue with these steps:

1. Specify the parent folder where the current Smart Analytics is installed, and then press **Enter**.
2. The installer automatically detects and lists which components have been installed. Review the detected components, and then press **Enter**.
3. Specify content folder names as needed, and then press **Enter**. By default, the installer automatically specifies all the content folder names.
4. Specify media server folder names as needed, and then press **Enter**. By default, the installer automatically specifies all the media server folder names.
5. Review the message that Smart Analytics is ready to upgrade, and then press **Enter**.
6. Check the pre-installation summary, and then press **Enter**.

Post-upgrade actions

Perform the following actions after you upgrade your Smart Analytics:

- When the installation is complete, do the following:
    - Check the StartAll.sh and StopAll.sh files under <Smart Analytics Installation>/Scripts folder for any commands that start or stop services not on your local machine. Delete or comment out such commands and

then add `source ./Start[Service Name of your previous content]` or `source ./Stop[Service Name of your previous content]` as needed.

- Delete all the start[Name of Service not on local machine].sh and stop [Name of Service not on local machine].sh files under <Smart Analytics Installation>/Scripts folder.

- If you upgrade from Service Manager 9.41 to Service Manager 9.81, you need to do full re-index of Catalog_Library after you upgrade to Smart Analytics 9.80.
- If you upgrade from Smart Analytics 9.41 to Smart Analytics 9.81, you need to manually modify the following configuration sections:

  In the *<SmartAnalytics>*/IDOL/IDOLServer.cfg file:

Existing configurations	You need to modify the configuration to

`[IndexQueue]`
`IndexQueueInitialSize=10240`
`IndexQueueMaxHistorySize=500`
`IndexQueueMaxPendingItems=10240`
`[SetDateFields]`
`// Fields containing the document date`
`Property=DateFields`
`PropertyFieldCSVs==*/DREDATE,*/DREDATE_*,*/DATE,*/DATE_*,*/*_DATE,*/*_TIME`
`[Logging]`
`LogExpireAction=compress`
`LogOldAction=move`
`LogMaxSizeKBs=20480`
	`[IndexQueue]`
	`IndexQueueInitialSize=10240`
	`IndexQueueMaxHistorySize=10240`
	`IndexQueueMaxPendingItems=0`
	`[SetDateFields]`
	`// Fields containing the document date`
	`Property=DateFields`
	`PropertyFieldCSVs=*/DREDATE,*/DREDATE_*,*/DATE,*/DATE_*,*/*_DATE,*/*_TIME,*/MODIFIED_EPOCH,*/LASTMODIFIED`
	`[Logging]`
	`LogExpireAction=Day`
	`LogMaxOldFiles=7`
	`LogOldAction=delete`
	`LogMaxSizeKBs=102400`

  In the *<SmartAnalytics>*/Level2proxy/IDOLServer.cfg file:

Existing configurations	You need to modify the configuration to

[Server]
[IndexQueue]
IndexQueueInitialSize=10240
IndexQueueMaxHistorySize=500
IndexQueueMaxPendingItems=10240
[Logging]
LogExpireAction=compress
LogOldAction=move
LogMaxSizeKBs=20480
	[Server]
	//store index data on disk only when the index cache is full or when the MaxSyncDelay(120 seconds by default) is reached.
	DelayedSync=TRUE
	[IndexCache]
	IndexCacheMaxSize=102400
	[IndexQueue]
	IndexQueueInitialSize=10240
	IndexQueueMaxHistorySize=10240
	IndexQueueMaxPendingItems=0
	[Logging]
	LogExpireAction=Day
	LogMaxOldFiles=7
	LogOldAction=delete
	LogMaxSizeKBs=102400

In the *<SmartAnalytics>*/Content<x>/Content<x>.cfg file:

Existing configurations	You need to modify the configuration to

[IndexQueue]
IndexQueueInitialSize=10240
IndexQueueMaxHistorySize=500
IndexQueueMaxPendingItems=10240
[SetDateFields]
// Fields containing the document date
Property=DateFields
PropertyFieldCSVs=*/DREDATE,*/DREDATE_*,*/DATE,*/DATE_*,*/*_DATE,*/*_TIME
[SetMatchFields]
PropertyFieldCSVs=*/*_MATCH,*/EDK_*,*/*_MATCH_INDEX,*/*_MATCH_WEIGHT*,*/*_BOOLEAN
[Logging]
LogExpireAction=compress
LogOldAction=move
LogMaxSizeKBs=20480
[LanguageTypes]
GenericTransliteration=TRUE
Transliteration=false
	//The amount of memory to allocate to store the most commonly occurring terms. (using 500MB as term cache here, recommended range is 10240 to 2048000.)
	[TermCache]
	TermCachePersistentKB=512000
	//------------------------ Schedules ----------//
	//expire and compact data everyday
	[Schedule]
	Expire=TRUE
	ExpireTime=23:00
	ExpireInterval=24:00
	[IndexQueue]
	IndexQueueInitialSize=10240
	IndexQueueMaxHistorySize=10240
	IndexQueueMaxPendingItems=0
	[SetDateFields]
	// Fields containing the document date
	Property=DateFields
	PropertyFieldCSVs=*/DREDATE,*/DREDATE_*,*/DATE,*/DATE_*,*/*_DATE,*/*_TIME,*/MODIFIED_EPOCH,*/LASTMODIFIED
	[SetMatchFields]
	PropertyFieldCSVs=*/*_MATCH,*/EDK_*,*/*_MATCH_INDEX,*/*_MATCH_WEIGHT*,*/*_BOOLEAN,*/IMPORTMAGICEXTENSION

Existing configurations	You need to modify the configuration to

	`[Logging]`
	`LogExpireAction=Day`
	`LogMaxOldFiles=7`
	`LogOldAction=delete`
	`LogMaxSizeKBs=102400`
	`[LanguageTypes]`
	`GenericTransliteration=TRUE`
	`Transliteration=true`

After you modify the configurations, restart all related Smart Analytics components of which such configuration are modified. Then, on the machine where level2proxy is located, reload updated configuration files dynamically and regenerate match index by visiting the following URLs :

- Reload updated configuration files dynamically:

  `http://localhost:20011/DRERESET`

- Regenerate the match fields:

  `http://localhost:20011/DREREGENERATE?Type=Match`

  Wait for a few minutes for the regeneration to complete.

- If you upgraded from a distributed Smart Analytics to Smart Analytics 9.81, add the following lines before the [actions] section in the `content[x].cfg file` for the system to clean up expired documents:

```
//--------------------------- Schedules --------------------
--//
//expire and compact data everyday
[Schedule]
Expire=TRUE
ExpireTime=23:00
ExpireInterval=24:00
```

> **Note**
>
> With this configured, the system automatically deletes all the expired documents at 23:00 every day. By default, the expiration date for documents is set to 365 days.

- After upgrade your Smart Analytics, you need to manually add `*/TITLE_INDEX` to the following files if you have scaled out the Content server:
  - <SmartAnalytics>/Content2/Content2.cfg
  - <SmartAnalytics>/Content3/Content3.cfg
  - <SmartAnalytics>/IDOL/IDOLServer.cfg

  You also need to modify the scaled content to add `*/TITLE_INDEX`.

Existing configurations	You need to modify the configuration to

`[SetTitleFields]`
`// The one field per document that is to be stored`
`as the autn:title value`
`Property=TitleFields PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
`*/TITLE_WEIGHT*`
	`[SetTitleFields]`
	`// The one field per document that is to be stored`
	`as the autn:title value`
	`Property=TitleFields PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
	`*/TITLE_WEIGHT*,*/TITLE_INDEX`

Smart Analytics 9.50 or later

If you select **Smart Analytics 9.50 or later**, continue with these steps:

1. Specify the parent folder where the current Smart Analytics is installed, and then press **Enter**.
2. The installer automatically detects and lists which components have been installed. Review the detected components, and then press **Enter**.
3. Specify content folder names as needed, and then press **Enter**. By default, the installer automatically specifies all the content folder names.

4. Specify media server folder names as needed, and then press **Enter**. By default, the installer automatically specifies all the media server folder names.
5. Review the message that Smart Analytics is ready to upgrade, and then press **Enter**.
6. Check the pre-installation summary, and then press **Enter**.

Post-upgrade actions

Perform the following actions after you upgrade your Smart Analytics:

- When the installation is complete, do the following:
  - Check the StartAll.sh and StopAll.sh files under <Smart Analytics Installation>/Scripts folder for any commands that start or stop services not on your local machine. Delete or comment out such commands and then add `source ./Start[Service Name of your previous content]` or `source ./Stop[Service Name of your previous content]` as needed.
  - Delete all the start[Name of Service not on local machine].sh and stop [Name of Service not on local machine].sh files under <Smart Analytics Installation>/Scripts folder.
- If you upgraded from a distributed Smart Analytics to Smart Analytics 9.81, add the following lines before the [actions] section in the `content[x].cfg file` for the system to clean up expired documents:

```
//--------------------------- Schedules --------------------
--//
//expire and compact data everyday
[Schedule]
Expire=TRUE
ExpireTime=23:00
ExpireInterval=24:00
```



Note

With this configured, the system automatically deletes all the expired documents at 23:00 every day. By default, the expiration date for documents is set to 365 days.

- If you installed Portal Content in Smart Analytics 9.50 or Smart Analytics 9.51, you need to manually modify the following configuration sections:

In the *<SmartAnalytics>*/IDOL/IDOLServer.cfg file:

Existing configurations	You need to modify the configuration to
`//The following parameter is required for the DAH if mirrormode is set to false VirtualDatabases=<x>`	`//The following parameter is required for the DAH if mirrormode is set to false VirtualDatabases=<x+1>`

Add the following configuration:

```
[vdb<x>]
dbname=SearchHistory
type=combinator
mapsto=2:SearchHistory
```

Note

- You must add the VDB configurations by their number sequence.
- There may be other content between the two VDB configuration sections.
- Make sure the total number of the VDB configuration sections equals the number specified in the VirtualDatabases parameter.
- There is no difference if you use the upper or lower case for the section name of [VDB<x>].

In the *<SmartAnalytics>*/Content-SMSP/Content-SMSP.cfg file:

Existing configurations	You need to modify the configuration to
`[Databases] NumDBs=<x>`	`[Databases] NumDBs=<x+1>`

Add the following configuration:

```
[Database<x>]
Name=SearchHistory
```

- After upgrade your Smart Analytics, you need to manually add `*/TITLE_INDEX` to the following files if you have scaled out the Content server:
  - <SmartAnalytics>/Content2/Content2.cfg
  - <SmartAnalytics>/Content3/Content3.cfg
  - <SmartAnalytics>/IDOL/IDOLServer.cfg

  You also need to modify the scaled content to add `*/TITLE_INDEX`.

Existing configurations	You need to modify the configuration to

`[SetTitleFields]`
`// The one field per document that is to be stored`
`as the autn:title value`
`Property=TitleFields`
`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
`*/TITLE_WEIGHT*`
	`[SetTitleFields]`
	`// The one field per document that is to be stored`
	`as the autn:title value`
	`Property=TitleFields`
	`PropertyFieldCSVs=*/DRETITLE,*/TITLE,*/DRETITLE_WEIGHT*,`
	`*/TITLE_WEIGHT*,*/TITLE_INDEX`

1.2.6. Upgrade Service Manager Collaboration

Important:

- Collaboration versions earlier than 9.41 (which is base on the legacy Enterprise Collaboration Server) are no longer supported. If you are upgrading from the legacy EC-based Collaboration, you only need to migrate the EC data. You can use the migration tool in Service Manager 9.62 to migrate the EC data to 9.62 first, and then upgrade to Service Manager 9.80 Collaboration.
- You must upgrade the chat server to version 9.80 if you have upgraded the Service Manager server to 9.80.

Upgrade the chat server

Follow these steps to upgrade the chat server to 9.80:

1. Stop your existing chat server.
2. (Windows Only) Open a DOS command prompt. Navigate to the %OLD_CHAT_SERVER_HOME%\bin directory, and then run the **nssm remove SmChatServer** command to remove the old service.
3. Make a backup of the following items:
    - %OLD_CHAT_SERVER_HOME%\conf*.*
    - %OLD_CHAT_SERVER_HOME%\plugins\lwssoplugin\lwssofmconf.xml
    - %OLD_CHAT_SERVER_HOME%\lib\ojdbc-xxxx.jar (the JDBC driver if you copied it to this folder before)
    - %OLD_CHAT_SERVER_HOME%\resources\security*.*
    - Any other customizations that you made
4. Save the chat server installer from the Service Manager 9.80 installation package to your computer.
5. Extract the chat-server-9.80.zip file to any folder. For example, C:\chatserver980 or /opt/chatserver980/. The following sections refer to this directory as %CHAT_SERVER_HOME%.
6. Copy the files you have backed up in *step 3* (except lwssofmconf.xml, openfire.xml, and security.xml) to the corresponding directories in %CHAT_SERVER_HOME%.

Note

- Open the %CHAT_SERVER_HOME%\conf\lwssofmconf.xml file with a text editor, and then make the same changes.
- Use a diff utility to compare the old out-of-the-box openfire.xml file and the security.xml file to your backed-up version. Make sure to apply your customizations to the new files properly so that you can use these files in your final deployment.
- As Lync Agent is obsoleted in this release, the <lyncIntegration> section is also removed from the openfire.xml file.

7. Windows: Run the **%CHAT_SERVER_HOME%/bin/install-service.bat** command to install the Windows service for the new chat server.

   Linux: Open the /etc/sysconfig/openfire file in a text editor, and then change value of the environment variable `OPENFIRE_HOME` to `%CHAT_SERVER_HOME%`.

8. Start the new chat server.

Upgrade the chat service

Follow these steps to upgrade the chat service to 9.80:

1. Stop your existing chat service.
2. (Windows only) Open a DOS command prompt. Navigate to **%OLD_CHAT_SERVER_HOME%**bin, and then run the **nssm remove SmChatService** command to remove the old service.
3. Make a backup of the following items:
   - %OLD_CHAT_SERVER_HOME%\conf*.*
   - Any other customizations that you made
4. Save the chat service installer from the Service Manager 9.80 installation package to your computer.
5. Extract the chat-service-9.80.zip file to any folder. For example, C:\chatservice980 or /opt/chatservice980/. This directory is referred to as **%CHAT_SERVICE_HOME%** in the sections below.
6. Copy the files that you have backed up in *step 3* (except app.properties and config.yml) to the corresponding directories in %CHAT_SERVICE_HOME%.

> **Note**
>
> Use a diff utility to compare the old out-of-the-box app.properties file and the config.yml file to your backed-up versions. Make sure you have applied your customizations to the new files properly so that you can use these files in your final deployment.
>
> Depending on your deployment type (HTTP or HTTPS), the new files to be deployed are in conf\samples\http_only or conf\samples\https.

7. Windows: Run the **%CHAT_SERVICE_HOME%/bin/install-service.bat** command to install the Windows service for the new chat service.

   Linux: Open the /etc/sysconfig/openfire file in a text editor, and then change value of the the environment variable `CHATMSVC_HOME` to `%CHAT_SERVICE_HOME%`.

8. Start the chat service.
9. Sign in to Service Manager as a system administrator, and then click **System Administration** > **Ongoing Maintenance** > **Collaboration Service Desk Chatbot**.
10. Click **Search**, and then click **Publish AIML** for every virtual agent.

Upgrade the chat UI

Follow these steps to upgrade the chat UI to 9.80:

1. Before deploying the new chat UI, make a backup of the following item:
   - %CHATUI_HOME%\conf\env.js
2. Delete or uninstall the existing chatui.war file.
3. Clear the cache of your web application server.
4. Deploy the new chatui-9.80.war file. For details, see Task 4: Deploy the End User Chat UI.
5. Copy the env.js file that's backed up in step 1 to the corresponding directory in the Chatui folder.
6. Restart the web application server.

1.2.7. Install platform unloads

A platform component is a component other than the applications, such as the Server, Windows Client, Web Tier, and Smart Analytics. New features and enhancements introduced in the latest version of a platform component sometimes require upgrades to both the component and the applications.

Some customers may select to upgrade only certain platform components and still run an old version of the applications. For these customers to take full advantage of the latest version of a platform component, the company provides unload files as needed in an **unloads** directory in the installation package of that component. These unload files contain the application-side changes made for the component, and you can manually install them in your system.



Note

If you plan to upgrade or have already upgraded to the latest applications, ignore these unload files, which are already included in the latest applications.

The following table lists the unload files for different platform components.

Name	Applications version 9.8x	Applications version 9.7x	Component	Description
OCTCR19X1772106_SM980_SM970.unl	9.80	9.70, 9.71, and 9.72	Server	Fixes the issue that the SM-OMi integration fails if the password of the OMi integration user is stored encrypted.
OCTCR19X1778093_SM980_SM970.unl	9.80	9.70, 9.71, and 9.72	Server	Fixes the issue that customers can save incorrect date/time values in date/time fields.
SmartEmail_SM980_SM970.unl	9.80	9.70	Server	Fixes OCTFT19X1150262 and supports OAuth2 Authentication with Delegated permissions when using EWS to connect to a mailbox for Smart Email.

Name	Applications version 9.8x	Applications version 9.7x	Component	Description
OCTCR19X1325632_SM980_SM970.unl	9.80	9.70	Server	Fixes the issue that occasionally the system doesn't process a few eventout records and these records remain in the queue.

Review the description of each unload file to determine which unload files are needed for your system.

For the steps to load an unload into Service Manager, see Load an unload file.

1.3. Upgrade the applications

You can use the Service Manager 9.80 Upgrade Utility to upgrade your Service Manager applications from version 9.3*x*, 9.4*x*, 9.5*x*, 9.6*x*, 9.7*x* to 9.80.



Note

The Upgrade Utility includes not only Upgrade Utility but also Application Patch Manager. It upgrades not only your applications but also your language packs to version 9.80.

Topics in this section include:

Applications upgrade overview

Application upgrade paths

Applications upgrade process

Planning an upgrade

Upgrade tasks on the development environment

Upgrade tasks on the test environment

Upgrade tasks on the production environment

Appendix A: Additional manual migration tasks on Service Manager Hybrid

Appendix B: Data migration

Appendix C: Run the SQL compare utility

Appendix D: Mapping between legacy security profiles and current PD security roles

1.3.1. Applications upgrade overview

The following sections describe how to upgrade your Service Manager applications to 9.80 by using the Service Manager 9.80 Upgrade Utility.

Before you begin an upgrade

Before you begin an upgrade, ensure that you:

- Read through the upgrade documentation to familiarize yourself with the upgrade process and all of the upgrade requirements.
- Are an experienced system administrator who is familiar with Service Manager.

If you don't have the administrative experience necessary to manage the upgrade, you may need assistance from your local applications developers and database administrators. You can also contact Service Manager Customer Support for help with troubleshooting upgrade errors. For additional information and support, contact your software sales representative.

Applications upgrade

You can upgrade your existing Service Manager applications to version 9.80 applications using the Upgrade Utility and resolving the differences between the two versions.

What are applications?

Applications are the Service Manager modules and their related configuration files. For example, Incident Management and Change Management are Service Manager applications.

New features that require an applications upgrade

The following features provided by the release of Service Manager 9.80 require an applications upgrade.

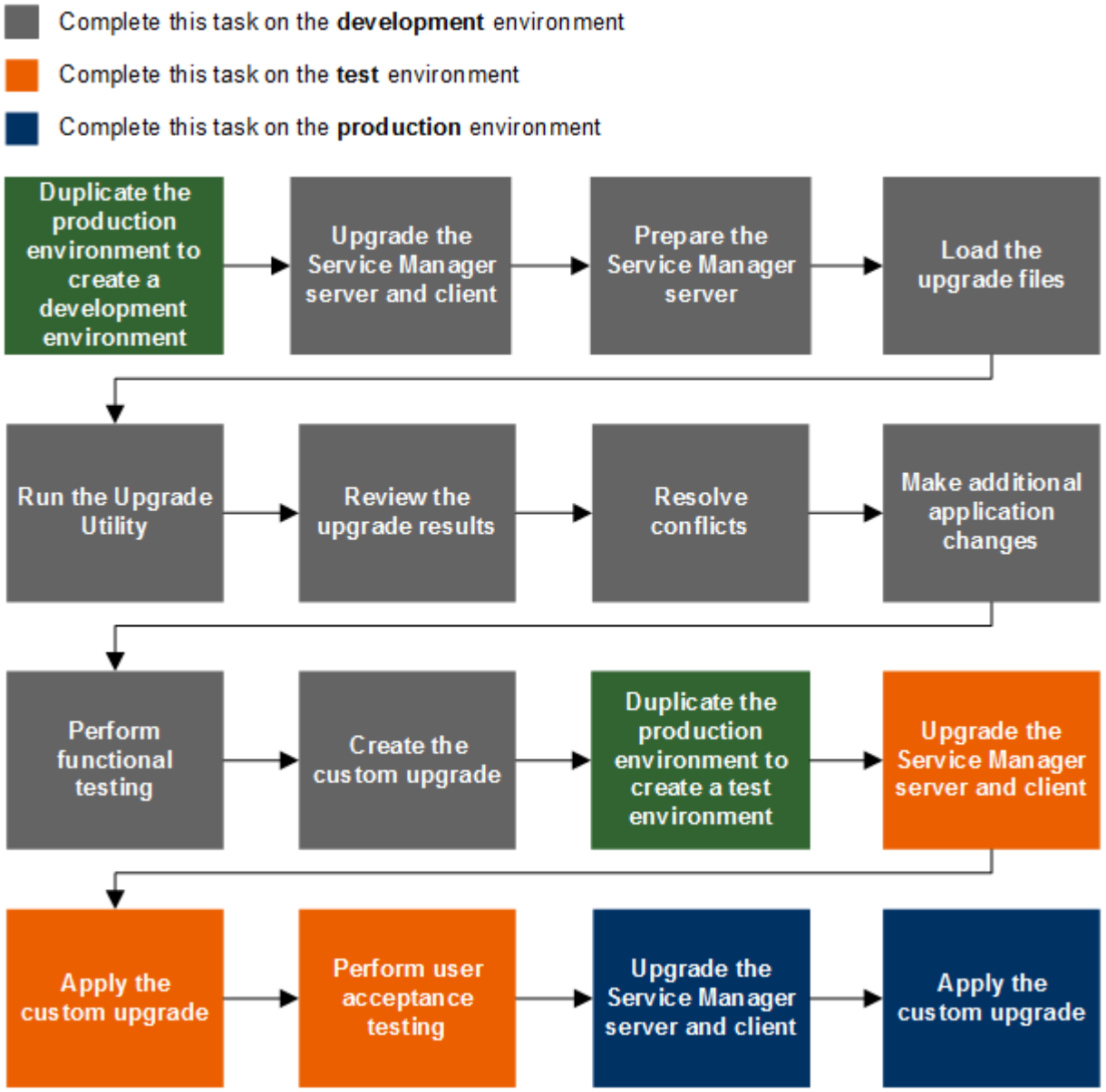
- ALM Octane integration (provided since Service Manager 9.72)
- Microsoft Teams notification (provided since Service Manager 9.72)
- Integrate with UCMDB gateway in SM-UCMDB integration (provided since Service Manager 9.72)
- Microsoft Teams integration (provided since Service Manager 9.71)
- Background process high availability (provided since Service Manager 9.71)
- Feature Tracker (provided since Service Manager 9.70)
- SLM Visualization (provided since Service Manager 9.70)

- Database Report (provided since Service Manager 9.70)
- Scorecard (provided since Service Manager 9.70)
- Native Release Control (provided since Service Manager 9.62)
- Smart Analytics Smart Ticket tuning toolkit (provided since Service Manager 9.60)
- Smart Analytics search accuracy improvement (provided since Service Manager 9.60)
- Process Designer framework
- Mobile Applications
- Case exchange
- Smart email
- Service Portal (provided since Service Manager 9.50)
- Enhanced Service Desk, Incident Management, Problem Management, Change Management, Request Fulfillment Management, Service Catalog Management (provided since Service Manager 9.50), and Service Level Management based on Process Designer (provided since Service Manager 9.40)
- Simplified interaction (provided since Service Manager 9.41)
- Logical Name (provided since Service Manager 9.41)
- Smart Search (provided since Service Manager 9.41)
- Service Manager Collaboration (provided since Service Manager 9.41)
- Accessibility for the embedded Service Manager Calendar
- Service Manager Reports (provided since Service Manager 9.40)
- Service Manager Survey (provided since Service Manager 9.40)
- Smart Analytics (provided since Service Manager 9.40)
- Entity Relationship Diagram (ERD and Data integrity check) (provided since Service Manager 9.40)

Applications upgrade lifecycle

You can use three environments to complete the upgrade process: your current production environment, a new development environment, and a new test environment. You will duplicate your production environment to create the development and test environments. Run the out-of-box upgrade and create the custom upgrade on the development environment, and then run and test the custom upgrade on the test environment before you apply it to your production environment. The figure below provides an overview of the steps in the upgrade process.

The following flow chart illustrates the lifecycle of a typical upgrade of Service Manager applications.



Upgrade phases and sub-phases

The following table describes the phases and sub-phases in the entire applications upgrade lifecycle. These sub-phases are logged in the upgrade log files during the upgrade. When an error occurs, the log files can help you find out during which phase and sub-phase the error occurs.

Phase	Sub-phases
Planning and preparation	• Load Transfer
Running an out-of-box upgrade	• Pre Upgrade Action Check • Pre Upgrade Action Update • Pre Upgrade Action Purge • Pre Upgrade Action • Load Upgrade File • Upgrade Dbdicts • Load Upgrading Data • Upgrade Data • Post Upgrade Action • Post Upgrade Action Prior to SM940 • Post Upgrade Action Prior to SM950 • Post Upgrade Action Prior to SM960 • Post Upgrade Action Auto Merge • Post Upgrade Action Purge • Post Upgrade Action Update • Post Upgrade Action Notification • Post Upgrade Action Restore
Creating a custom upgrade	• Pre Create Action Check • Build Signatures • Build Distribution • Export Data • Transfer Data

Phase	Sub-phases
Applying the custom upgrade	• Pre Upgrade Action Check • Pre Upgrade Action Update • Pre Upgrade Action Purge • Pre Upgrade Action • Load Upgrade File • Upgrade Dbdicts • Load Upgrading Data • Upgrade Data • Post Upgrade Action • Post Upgrade Action Prior to SM940 • Post Upgrade Action Prior to SM950 • Post Upgrade Action Prior to SM960 • Post Upgrade Action Purge • Post Upgrade Action Update • Post Upgrade Action Notification • Post Upgrade Action Restore



Note

If you are upgrading from SM9.6x, there are no such sub-phases.

How does customization affect the upgrade process?

The following explains how customization affects the upgrade process.

Conflicts

Object changes: The Upgrade Utility compares the objects in your database with their out-of-box versions and the corresponding objects provided from the upgrade package. The Upgrade Utility compares objects by their signatures. Each data record in Service Manager has a unique signature, which changes once that data record is updated. When processing object changes, the Upgrade Utility behaves as described in the following tables.

Changed object			
Object in DB has been tailored?	OOB version of object matches upgrade package?	Out-of-box upgrade	Custom upgrade

Object in DB has been tailored?	OOB version of object matches upgrade package?	Out-of-box upgrade	Custom upgrade
Yes	No	The Upgrade Utility tries to merge the object from the upgrade package with your tailored object. If the merge is successful, the Upgrade Utility marks the new merged object as "Auto Merged." If the merge fails, the Upgrade Utility marks the object as "Renamed." In both cases, the Upgrade Utility prefixes the object from the upgrade package with "NEW980," and then copies the object to your database and prefixes that object as "PRE<*version_number*>". For example, an object from applications version 9.31.0022	The Upgrade Utility marks the object from the upgrade package as “Forced,” and then copies the object in its revision.

Object in DB has been tailored?	OOB version of object matches upgrade package?	Out-of-box upgrade	Custom upgrade
		PDCP4 would be prefixed with "PRE9.31.0022_PDCP4".	
Yes	Yes	The Upgrade Utility keeps your local version, even if your version has been tailored and marks your local version as “Kept Customer”.	The Upgrade Utility marks your local version object as “Already Current.”
No	No	The Upgrade Utility overwrites the object in your database with the object from the upgrade package, and marks the object as “Upgraded.”	The Upgrade Utility marks the object from the upgrade package as “Forced,” and then copies the object in its revision.
No	Yes	The Upgrade Utility marks the object as "Already Current" regardless of whether this is the first upgrade or a custom upgrade.	

Added object

Object is added in ...	Behavior	Note
DB	The Upgrade Utility always keeps your local version and does nothing else.	The object in your database doesn't have a corresponding object from the upgrade package.
Upgrade package	The Upgrade Utility adds the object to your database and marks the object as "Added" regardless of whether this the first upgrade or a custom upgrade.	The object from the upgrade package doesn't have a corresponding object from your database.

Dbdict changes: The Upgrade Utility automatically adds new dbdict and merges new fields to existing dbdicts. The Upgrade Utility doesn't delete any existing field. For field and key changes, check "Field mapping changes" and "Key changes."

Field mapping changes: Normally the Upgrade Utility applies field mapping changes automatically, but there may be some exceptions. For example, when a length change is required, the Upgrade Utility automatically expands the length mapping. However, if the field mapping is a LOB-type change, the Upgrade Utility will not the change the type mapping. For detailed exceptions, check SQL Field Compare results before the upgrade and the except.log file after the upgrade.

Key changes: Normally the Upgrade Utility applies key changes automatically, but there may be some exceptions. For example, the Upgrade Utility automatically adds a new key and updates the existing key of the pre-upgrade out-of-box version. However, if a Unique key has been tailored, the Upgrade Utility will not apply the key change. For detailed exceptions, check the SQL Unique Key Compare Results before the upgrade and the except.log file after the upgrade.

Customization during upgrade

If any tailoring changes are made to your production system, for example, by applying an applications "hot fix," after you have initiated the upgrade process, it is highly recommended that you apply those same changes to the development system that's being used for conflict resolution before you create the final custom upgrade package. Or, these tailoring changes may be lost after the custom upgrade has been

applied to production, and any conflict resolution that needs to be done in a production environment may slow down the production upgrade.

Upgrade Utility contents

The following table lists the files that are included in the Service Manager Upgrade Utility.

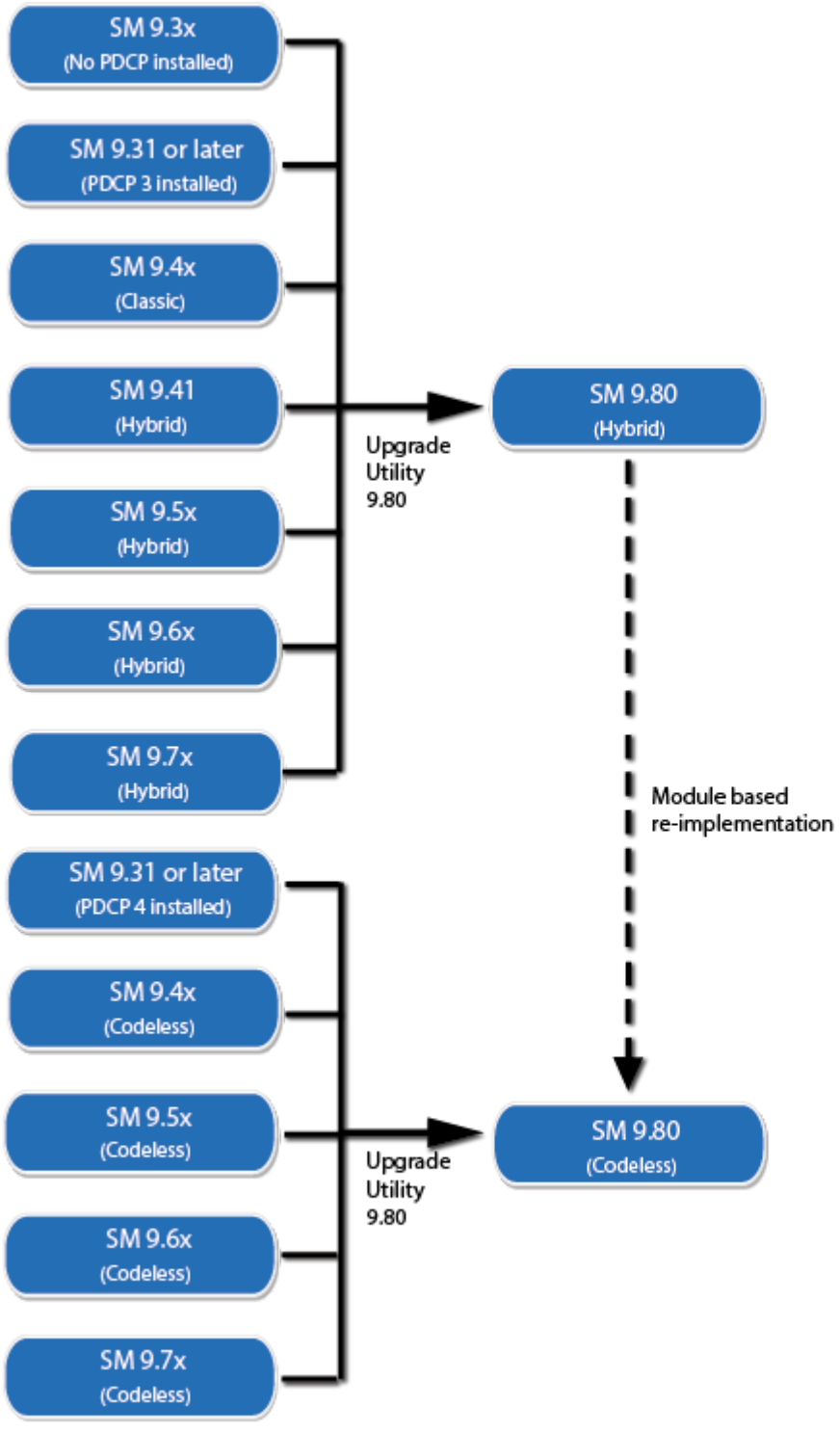
List of Upgrade Utility files

File	Contents
AppUpgVersion.txt	Contains Upgrade Utility version and build number information to help you identify which applications upgrade version you have available. For example: A version of "SM950-9.80.*00xx* v9.80*00xx* Upgrade Build *00xx*" indicates the following: • The Upgrade Utility upgrades Service Manager 9.50 and later releases to Service Manager 9.80. • The Upgrade Utility version number is *9.80.00xx*. • The Upgrade Utility build number for this version is *00xx*. • The string of `Application Version 9.80.00xx Build MF` indicates that Upgrade Utility is released for SM 9.80.
package directory	Files including upgrade data for all supported versions.
preupg.bin	Files that allow you to access the various features of the Upgrade Utility.
transfer.bin (in the package*<version>* folder)	Files that allow for the execution of the upgrade.
upgrade.ver	Version stamp for this upgrade.
upgdbdct.dta	Temporary dbdicts needed for the SQL Compare process.

File	Contents
DeltaMigrationTool.unl	Files that allow you to run Delta Migration Tool.
BusinessPurgeTool.unl	Files to allow you to run Business Data Purge Tool.
fileload.unl	If you are upgrading from version 9.30 or 9.31, you need to import the fileload.unl file that's already in the Upgrade Utility package before loading pregupg.bin.
.zip (in the 3waymerge\oob folder)	Each zip file includes the XML representation of the objects that have been signatured in the pre-upgrade out-of-box version for the Three-Way Merge tool.

1.3.2. Applications upgrade paths

Service Manager 9.80 is available in two modes: Service Manager Codeless and Service Manager Hybrid. The following diagram illustrates the recommended Service Manager 9.80 applications upgrade paths. The paths and arrows symbolize direct upgrades.



Note:
This diagram illustrates the recommended applications upgrade paths only. All paths and arrows symbolize direct upgrades.

Service Manager Codeless

As of version 9.40, Service Manager Codeless incorporates the Process Designer functionality that was available through content packs before, together with further enhancements that aren't available to users of Service Manager Classic. Primarily, this involves implementing Process Designer-based workflows for a number of modules, including Change Management, Service Desk, Incident Management, Problem Management, Request Fulfillment, Knowledge Management, and Service Level Management.



Note

Process Designer is a graphical interface that enables you to develop the workflows that control the flow of a single record throughout its lifecycle within Service Manager, without being an expert in RAD programming.

Service Manager Codeless is available to customers who are installing Service Manager for the first time or who are upgrading from the following scenarios:

- A Service Manager 9.3x system that has Process Designer Content Pack 9.30.3 (PDCP4) applied
- A Service Manager 9.4x system (Codeless mode)
- A Service Manager 9.5x system (Codeless mode)
- A Service Manager 9.6x system (Codeless mode)
- A Service Manager 9.7x system (Codeless mode)

Service Manager Hybrid

Service Manager Hybrid is a mode of Service Manager in which Process Designer technology is fully implemented, yet that allows you to continue using legacy technology such as Format Control. Service Manager Hybrid eases the transition between Service Manager Classic and Service Manager Codeless by enabling you to continue to take advantage of your previous investments in legacy technology. This mode is available to customers who are upgrading from the following scenarios:

- A Service Manager 9.3x system without PDCP installed
- A Service Manager 9.3x system that has Process Designer Content Pack 9.30.2 (PDCP3) applied
- A Service Manager 9.4x system (Classic mode or Hybrid mode)
- A Service Manager 9.5x system (Hybrid mode)
- A Service Manager 9.5x system (Hybrid mode)
- A Service Manager 9.6x system (Hybrid mode)

- A Service Manager 9.7x system (Hybrid mode)

Important

Service Manager 9.80 doesn't support the following upgrades:

- Upgrade from SM9.3x with PDCP4 installed to SM9.80 Hybrid.
- Upgrade from SM9.4x Codeless to SM9.80 Hybrid.
- Upgrade from SM9.5x Codeless to SM9.80 Hybrid.
- Upgrade from SM9.6x Codeless to SM9.80 Hybrid.
- Upgrade from SM9.7x Codeless to SM9.80 Hybrid.

Meanwhile, Service Manager 9.80 doesn't support the following direct upgrades:

- Direct upgrade from SC6.2/SM7.0x to SM9.80 Hybrid/Codeless.
- Direct upgrade from SM7.1x/SM9.2x to SM9.80 Hybrid/Codeless.
- Direct upgrade from SM9.3x without PDCP installed/SM9.3x with PDCP2 installed/SM9.3x with PDCP3 installed to SM9.80 Codeless.
- Direct upgrade from SM9.4x Classic to SM9.80 Codeless.
- Direct upgrade from SM9.4x Hybrid to SM9.80 Codeless.
- Direct upgrade from SM9.5x Hybrid to SM9.80 Codeless.
- Direct upgrade from SM9.6x Hybrid to SM9.80 Codeless.
- Direct upgrade from SM9.7x Hybrid to SM9.80 Codeless.

After you upgrade to Service Manager 9.80 applications, refer to Service Manager support matrix for a list of supported databases. If your existing database isn't supported, upgrade your database by following the instructions in your specific database documentation.

Recommended upgrade order to Service Manager 9.80

1. Install the Service Manager 9.80 server.
2. Install the Service Manager 9.80 Windows and web clients.
3. Upgrade to the Service Manager 9.80 applications using Service Manager 9.80 Upgrade Utility.
4. Install or upgrade other components.

Migration procedure from Service Manager 9.80 Hybrid to 9.80 Codeless

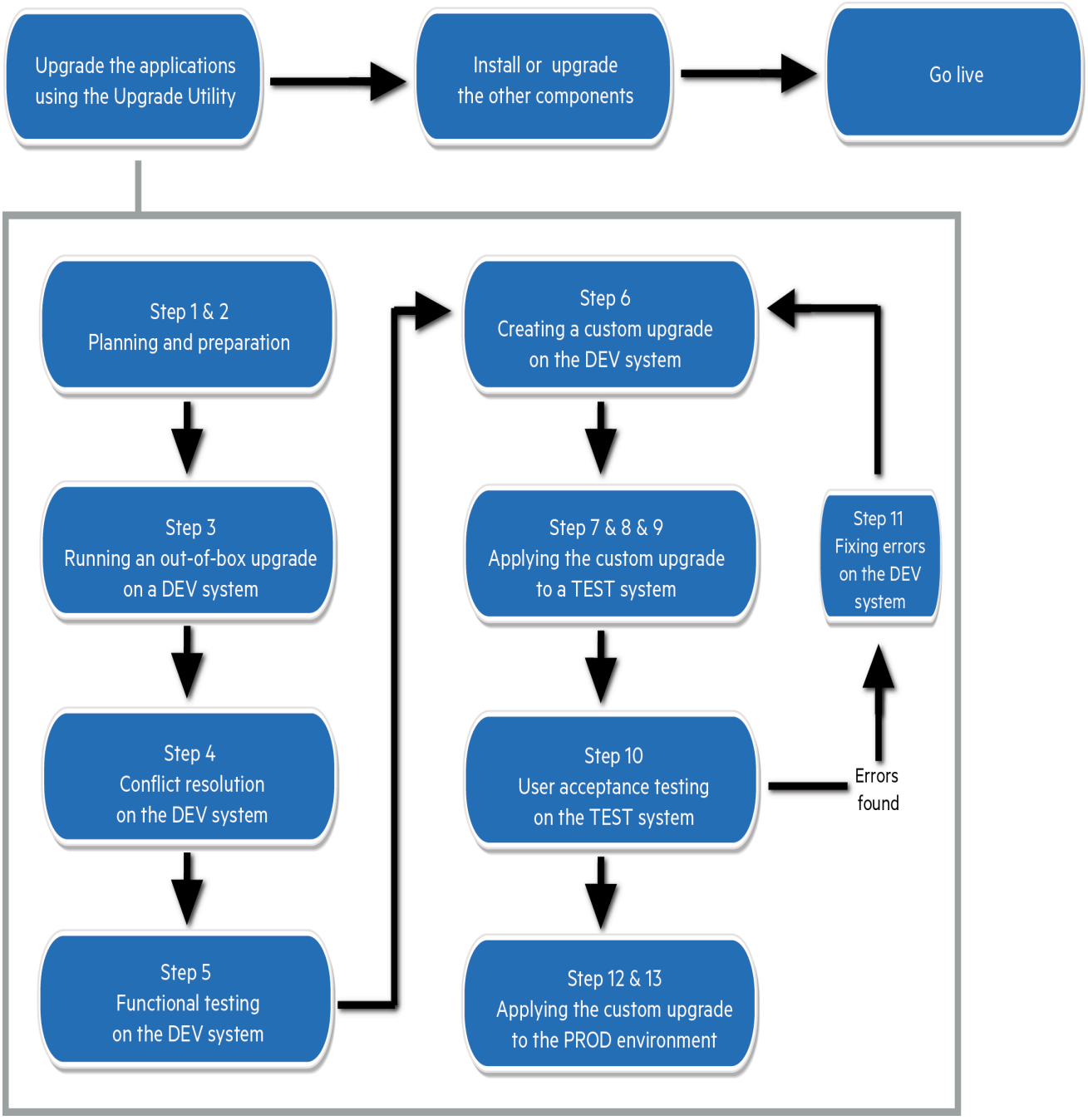
Service Manager Hybrid has all the necessary prerequisites and you can migrate it to Service Manager Codeless.

Note

- If your applications version is earlier than 9.31 and has Process Designer Content Pack installed, you must first upgrade the applications to version 9.31 before you can further upgrade to Service Manager 9.80 Hybrid.
- If your Service Manager system has Process Designer Content Pack 9.30.1 (PDCP2) installed, you must first upgrade to Process Designer Content Pack 9.30.2 (PDCP3) before you can further upgrade to Service Manager 9.80 Hybrid. For more information about how to upgrade from PDCP2 to PDCP3, refer to AppPatchManager_CI.pdf in the release package of Process Designer Content Pack 9.30.2 at Process Designer Content Packs for Service Manager.

1.3.3. Applications upgrade process

The following diagram illustrates the upgrade process.



The high-level steps of the applications upgrade process are explained as follows. For more detailed information of each step, see Applications upgrade overview.

Step 1. Create a development system by duplicating your production system.

You will run an out-of-box upgrade on this system, and then fix errors and resolve conflicts. Make sure the development system meets the requirements in the support

matrix of the target upgrade version, and copy your production data to the development system.

Step 2. Prepare the development system for an upgrade

Prepare the data and binaries (server and client) of the development system for an upgrade.

1. Purge and archive unnecessary data to optimize upgrade performance.

   Purge and archive unnecessary data in the following files (which may contain large amounts of data): mail, msglog, syslog, spool, eventin, eventout, schedule, sysperform, and devaudit (on a production system, development auditing should be turned off).

   For information about purging and archiving records, see the Service Manager Help Center.

2. Meet database requirements.

   1. Keep the table space name same as the production system for an Oracle database.
   2. Convert all column names to uppercase for a case-sensitive database.
   3. Remove some indexes and constraints.
3. Upgrade your Service Manager server and client to the latest version.
4. Update the server configuration files (sm.cfg and sm.ini).

   Before running the Upgrade Utility, you need to make changes to the Service Manager configuration files. Record all changes that you have made so that you can revert them to the original status after the upgrade.

Step 3. Run an out-of-box upgrade on the development system.

1. Check the size of the applications upgrade data, and compare it with the out-of-box upgrade package to make sure the downloaded data has no problems.
2. Load applications upgrade file preupg.bin from the out-of-box upgrade package.
3. Sign out and then sign in again.
4. Apply the out-of-box upgrade package to the development system.
5. Sign out, restart the server, and then sign back in.

Step 4. Conflict resolution on the development system.

1. Fix all exceptions in the except.log file, and verify your fixes.
2. Resolve all conflicts in the upgraderesults table, and verify your conflict resolution results.
3. Perform additional manual tasks.

Step 5. Functional testing on the development system.

Return the development system to normal operation, and test the upgraded development system.

Step 6. Create a custom upgrade package in the development system.

Now you have resolved all the exceptions and conflicts. It is necessary to package all the reconciled objects together into a "custom upgrade" so that they can be moved into the production environment automatically.

Step 7. Create a test system by duplicating your production system.

You need to apply the newly-created custom upgrade to your test system for user acceptance testing. Make sure the test system meets the requirements in the support matrix of the target upgrade version, and then copy your production data to the test system.

Step 8. Prepare the test system for an upgrade.

Follow the same steps for the development system, as described in *step 2*.

Step 9. Apply the custom upgrade package to the test system.

1. Upgrade the Service Manager server and client to the latest version for your test system.
2. Load applications upgrade file preupg.bin from the custom upgrade package.
3. Sign out and then sign in again.
4. Apply the custom upgrade package to the test system.
5. Fix all exceptions in the except.log file, and verify your fixes.

6. Resolve all "Error" results in the upgraderesults table, and verify your resolution results.

Step 10. User acceptance testing on the test system.

Return the test system to normal operation, and test the custom upgrade.

Test all features that your users will access. Pay particular attention to areas that were modified on your system.

Step 11: (Optional) Fixing errors in the development system.

If there are errors found in the test system, fix them in the development system and then go to *step 6*.

Step 12. Prepare the production system for an upgrade.

Follow the same steps for the development system, as described in *step 2*.

Step 13. Apply the custom upgrade package to your production system.

Make sure you have thoroughly tested the custom upgrade, before you apply the newly-created custom upgrade to your production system. This process is identical to the one you followed when applying your upgrade to your test system.



Note

The production system should not be available to users while you are applying the custom upgrade.

1.3.4. Planning an upgrade

Good planning allows your upgrade to run as smoothly and quickly as possible, and helps you to avoid retracing your steps. When preparing for your upgrade, you will need to consider how long each step will take and when users need to be logged off the system so that you can schedule each phase of your upgrade.

Step 1: Identify the upgrade resources

Make sure that you have access to the following resources

- Service Manager tools: The utilities you will use most during the upgrade process include Database Manager and Forms Designer.
- Documentation resources: See Install for client/server installation instructions and Service Manager knowledge.
- Software Support: The Software Support web site has operating system and compatibility information, product documentation, and release notes.

Step 2: Meet the software requirements

Before you start your upgrade, make sure that you meet the following Service Manager system requirements:

- Your RDBMS version, operating system, and client/server environment must meet all criteria listed in the Support Matrix for the target version. See Support matrix.
- To directly upgrade your existing applications to Service Manager 9.80, your existing Service Manager applications release level must be one of the following:
  - Service Manager 9.3x without Process Designer installed
  - Service Manager 9.31 or later with Process Designer Content Pack 9.30.2 or 9.30.3 installed
  - Service Manager 9.4x Classic, Codeless, or Hybrid
  - Service Manager 9.5x Codeless, or Hybrid
  - Service Manager 9.6x Codeless, or Hybrid
  - Service Manager 9.7x Codeless, or Hybrid
- The Service Manager server process (sm) must have read-write access to the database.

Backups

It is highly recommended, at a minimum, that you back up the database at the following strategic points in the upgrade lifecycle:

- After applying an upgrade
- After resolving conflicts

NFS-mounted partitions

Do not install either Service Manager or the Service Manager Upgrade Utility on an NFS-mounted remote partition. This can cause serious performance degradation. The performance of an NFS-mounted partition drops significantly if it reads data in many small pieces instead of one large chunk. Service Manager generates a lot of database read/write activity. An NFS-mounted partition is significantly slower than a local drive when running the Upgrade Utility process.

Step 3: Perform a system health check

A well-maintained production system is the easiest to upgrade. Before starting the upgrade process, perform all regular maintenance on your production system. If necessary, contact Customer Support for recommended actions. Suspend all customization activity on the production system.

Step 4: Create development and test environments

Plan to have at least two copies of your existing production environment:

- A development system that mirrors your current production environment. Use the development system to run the out-of-box upgrade and build a custom upgrade.
- A test system that mirrors your current production environment. Apply the custom upgrade on the test system and verify it there.

Step 5: Develop an upgrade strategy

In standard Service Manager terminology:

- *Customization* refers to changes to RAD applications.
- *Tailoring* refers to changes made by using Service Manager tailoring tools, such as Forms Designer and Format Control.
- *Configuration* refers to local settings (for example, in your environment records and the system information record).

The upgrade process affects different parts of the Service Manager system. Besides upgrading the standard Service Manager applications, an upgrade affects the RDBMS where Service Manager is running and any customized files or RAD applications. For more information, see How does customization affect the upgrade process?.

Tailored systems

A list of tailored files can help you resolve differences quickly between your existing files and new files. You can also use the SQL Compare utility to determine how dbdict files differ.

To list all upgrade files, click **Tailoring** > **Differential Upgrade** > **Patch Records**. Select **UPGRADE** in the drop-down list and then press Enter.

RDBMS-mapped systems

Because Service Manager tables (data files) must be mapped to an RDBMS, you must choose one of the following options before beginning the upgrade:

- Allow the Upgrade Utility to modify your RDBMS tables for you.
- Use SQL Compare utility to generate dbdict difference report and manually update the RDBMS databases before beginning the upgrade process.

The upgrade can affect certain mappings and tables. Contact your database administrator for assistance and to discuss the impact on the RDBMS.

Localized systems

You can upgrade a localized system with the Upgrade Utility. Before you begin to upgrade a localized system, ensure that you have the correct language pack for the language to which you will be upgrading. For more information and instructions on how to install the language pack, refer to Install language packs. The Upgrade Utility detects the presence of a localized system and runs the upgrade just as it would for an English system. You will have to make any customization and tailoring changes, based on the requirements described in this document for each of your system configurations.

Customized RAD applications and ScriptLibrary records

A list of customized RAD applications and ScriptLibrary records and the extent of the customization is useful. If it is unavailable, the programmer who made the changes may be able to supply information. Or, you may need to run a comparison between the existing applications or script and the new version.

Step 6: Meet database requirements

Before upgrading your system, verify that your system is pointing to the correct database.

Convert all tables and fields from lowercase to uppercase

Service Manager doesn't generate lowercase table names or field names. Therefore, if your database is case-sensitive, you must convert all tables and fields from lowercase to uppercase before you can upgrade the server and client or applications.

Note

For Oracle users, you must have a granted role that includes “connect” and “resource” along with a granted system privilege of “select any dictionary” as a minimum in order to avoid errors generated by Oracle.

1.3.5. Upgrade tasks on the development environment

Topics in this section include:

Step 1: Duplicate the production environment

Step 2: Upgrade server and client

Step 3: Perform preparatory tasks

Step 4: Load the applications upgrade files

Step 5: Run the Upgrade Utility

Step 6: View the upgrade results

Step 7: Manage the upgrade result data

Step 8: Resolve exceptions

Step 9: Resolve conflicts

Step 10: Perform additional manual tasks

Step 11: Migrate Process Designer data

Step 12: Return the system to normal operation

Step 13: Test the development environment

Step 14: Back up the system

Step 15: Build a custom upgrade

1.3.5.1. Step 1: Duplicate the production environment

To achieve the best results, develop and test the custom upgrade on a system that resembles your production environment as closely as possible.

To duplicate the production environment, follow these steps:

1. Identify a server to use for the development and test environments.
    - Linux: You can copy the files to a new location on your production machine.
    - Windows: You must create the development system on a different machine from your production system.
2. Ensure that adequate memory and disk space is available and accessible. Frequent backups are necessary.
3. Ensure that your development and test systems meet all upgrade requirements. For more information, see Planning an upgrade.
    - Upgrade your RDBMS to a version compatible with Service Manager 9.80. See Support matrix.
    - Convert your RDBMS code page to Unicode. See your RDBMS vendor documentation.
4. Set up the environment of your development and test machines to resemble your production server as closely as possible. The operating system version and service pack level should match.
5. Copy your existing production system data onto your development system.

    We recommend you use the native RDBMS backup utilities to back up your data. Refer to your RDBMS documentation for backup instructions.
6. Install a Service Manager 9.80 run time environment on the duplicated system. Do not load the Service Manager 9.80 demonstration data files.
7. Install a Service Manager 9.80 client on the duplicated system.

Note

After you duplicate the production environment, you must either disable the integration servers and the KM servers, or change the URL to connect to the integration servers and the KM servers that are in the test environment.

1.3.5.2. Step 2: Upgrade server and client

Make sure that you have upgraded your server and the client to the latest version before running an applications upgrade. This allows the upgrade utility to call the new functions in the latest server and client and to take full advantage of all applications features following an upgrade.



Note

You must upgrade the IDOL server if you are working with IDOL. For details, see Upgrade Smart Analytics.

To upgrade your applications by using this Upgrade Utility, you must first perform a server and client upgrade to the 9.80 platform (server and client).

- Obtain the latest client from the installation package, and then follow the instructions in Upgrade the Windows client.
- Obtain the latest server from the installation package, and then follow the instructions in Upgrade the Service Manager server.

1.3.5.3. Step 3: Perform preparatory tasks

You must prepare your system before upgrade. To do this, perform the following tasks:

1. Check the locale information and make sure it is English, especially when the SM Server is running on the Linux platform. You need to run the **locale** command on the Linux platform to get the locale information, and then set it to **en_US.UTF-8** as necessary.
2. (Optional) You can read and use the sysctl.conf file to control the value of the *shared_memory* parameter. To set proper OS parameters, copy the `<Service Manager server installation path>/RUN/97-sm-sysctl.conf` file into /etc/sysctl.d directory.
3. Use the Windows client to disable the **Client side load/unload** option. To do this, click **Window** > **Preferences**, and then click to clear the option.

   Caution

   Failure to disable this option will cause the upgrade process to fail.
4. Disable Development Audit. To do this, navigate to **Tailoring** > **Audit** in the System Navigator, and then click **Turn Auditing On/Off**. Clear the **Do you want to audit development changes?** option if it is selected, and then save the change.
5. Clean up any records in the following log files:
   - Msglog
   - Syslog
   - Errorlog

   Note

   Back up the table data if you need it later. If the tables contain too many records, the upgrade process will be slow.
6. Clean up the following messaging files:
   - Eventin records
   - Eventout records

> **Note**
>
> Back up the table data if you need it later. If the tables contain too many records, the upgrade process will be slow.

7. Clean up any files in the systemperform table.

> **Note**
>
> Back up the table data if you need it later. The systemperform dbdict will be converted from lob columns to real database columns. Therefore, if there are too many records in the systemperform table, transfer.bin will load very slowly.

8. Update the Service Manager configuration files.

   The following tables list the changes that you need to make to the Service Manager configuration files before running the Upgrade Utility. Record all changes that you have made so that you can revert them to the original status after the upgrade.

   Stop the Service Manager server, apply the required changes to the configuration files, and then restart the Service Manager server.

sm.cfg

Parameter	Changes	Description
wildcardcharacters	If this parameter exists, comment it out by changing it to: `#wildcardcharaters`	Failure to disable this option will cause the upgrade process to fail.
sm system.start	If this parameter exists, comment it out by changing it to: `#sm system.start`	Commenting this parameter out disables the background processes.
sm -sync	Add this parameter to the end of the file if it doesn't exist yet.	This parameter starts the sync process, which identifies and releases locks owned by inactive processes and shared memory that's not in use.
sm -httpPort	If there is more than one instance of the **sm -httpPort** parameter, keep only one instance.	Each **sm -httpPort** parameter starts a Service Manager server process that can handle a certain number of client sessions (see the Service Manager Help Center documentation for more information). Keeping one process alive will be enough for the upgrade process.

Parameter	Changes	Description
Other parameters	Comment out all other parameters except the ones mentioned in this table.	Commenting out those parameters disables all the other Service Manager processes that aren't required during an upgrade.

sm.ini

Parameter	Changes	Description
ir_disable:1	Add this parameter to the end of the file if it doesn't exist.	This parameter disables all IR keys on your existing Service Manager system. This will make the upgrade process run faster.
sessiontimeout:1200	Add this parameter to the end of the file if it doesn't exist. If this parameter already exists, update it to an appropriate value.	This parameter defines the number of minutes that the server waits for a client heartbeat signal before the server assumes that the client session has timed out and closes the connection. A value of 1200 sets the timeout to 20 hours (1200 minutes), a period that should be enough for an upgrade phase to complete in a typical scenario.

Parameter	Changes	Description
heartbeatinterval:120	Add this parameter to the end of the file if it doesn't exist.	This parameter controls the client heartbeat frequency. If the server doesn't receive a heartbeat from the client within the time-out limit as defined by the sessiontimeout parameter, the server terminates the client. All unsaved data is lost and the client must establish a new connection.

Restart the Service Manager server before you perform the upgrade in order for these tasks to take effect.

1.3.5.4. Step 4: Load the applications upgrade files

You must load the `preupg.bin` file into Service Manager before you can use the Upgrade Utility.

If you are upgrading from version 9.30 or 9.31, you need to import the `fileload.unl` file that's already in the Upgrade Utility package before loading `pregupg.bin`. If you are performing a custom upgrade, import the `fileload.unl` file from Upgrade Utility package as well.



Note

If you are performing a custom upgrade on a test or production system, use the `preupg.bin` file included in your custom upgrade instead.

To load the applications upgrade file, follow these steps:

1. On the Service Manager server, create a folder (referred to as the Upgrade folder later in this document).

   Note

   Make sure that the Service Manager server process (`sm`) has write and execute privileges for this folder.

   Suppose you are connecting to the Service Manager server from a client that's installed on a remote client computer. In this case, make sure to create the folder on the Service Manager server instead of the client computer.

2. Extract the Service Manager applications Upgrade Utility files to the Upgrade folder. Make sure the extraction doesn't miss any Upgrade Utility files. Otherwise, the upgrade process will fail.

3. Sign in to the Service Manager Windows client as a system administrator.

> **Note**
>
> Select **English** as the language when signing into the system for an upgrade.

4. Back up the `SYSTEM ADMINISTRATION` menu record.
5. Load the `preupg.bin` file using Service Manager Database Manager.
6. Sign out of Service Manager and then sign in again.

1.3.5.5. Step 5: Run the Upgrade Utility

Now that you have a functional environment, you are ready to run the Upgrade Utility. Follow the steps in this step to run the out-of-box upgrade against the data in your development environment and to run your custom upgrade against your test and production environments. You must perform these steps in a Service Manager Windows client, instead of a web client.



Caution

If the upgrade fails while the Upgrade Utility is running, fix possible issues and rerun the Upgrade Utility, and you should be able to resume the upgrade from the failure point; if the upgrade process can't be resumed, you must restore the database to the last backup point and fix possible issues before you can rerun the Upgrade Utility.

The running of the Upgrade Utility involves the following three primary phases:

1. In the first phase, the Upgrade Utility guides you through several questions and collects information needed for the upgrade.
2. The second phase is the dbdict update phase, where the utility updates dbdicts.
3. The third phase is the data update phase, where the utility updates applications data.

To run the Service Manager Upgrade Utility, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **Apply Upgrade** to launch the Upgrade Utility.
2. In the text box, type the fully qualified path to the Upgrade Utility folder, and then click **Next**.

   Example:

   Windows: `c:\temp\upgrade\`

   Linux: `~/upgrade/`

   Note

   You must have read, write, and execute permission to the folder.

3. The system displays a series of information for your verification, including Applications version upgrading from, Applications version upgrading to, Applications base version, Full path to the Upgrade Utility files and Language(s) to be upgraded. Verify this information and then click **Next** to continue.

   Note
   - If this screen doesn't display the correct information, don't continue with the upgrade. Instead, contact Software Customer Support.
   - If you are upgrading from SM9.6x, the system displays the parameter settings before this step. Review the settings and then click **Next** to continue.

4. (Optional) The system prompts you to migrate legacy categories to the corresponding Process Designer-based modules. To move your selection between the legacy categories lists and the Process Designer-based modules lists, select a category and click the arrow button. Click **Next**.

   Note

   This screen appears only when you are upgrading from the following versions:

   - Service Manager 9.3x without Process Designer Content Pack installed
   - Service Manager 9.3x with Process Designer Content Pack 9.30.2 (PDCP3) installed
   - Service Manager 9.4x Classic

   Based on your business requirements, you may select all categories you are using to migrate.

   Before you proceed, refer to Appendix A: Additional manual migration tasks on Service Manager Hybrid for more information.

5. The upgrade is now ready to start. Click **Next** to start the upgrade.
6. When you are asked whether you want to proceed, click **Yes**.
7. The Upgrade Utility displays the status when the upgrade is being processed. The loading process may take a long time.
8. When you receive an `UPGRADE IS COMPLETE` message, the Upgrade Utility has finished the data processing and you can follow the instructions in the

message to complete the next steps. After you click **Finish**, you are automatically logged out.

9. Restart the server and log back into the client.
10. Open the scversion table in the Database Manager, and verify that the Applications Version field is 9.80.00xx. If this field displays a value other than 9.80.00xx, check the log files to identify the issue that occurred.

Upgrade Utility logs and error messages

The Upgrade Utility creates a set of log files during the upgrade process. These files reside in the log directory.

List of upgrade log files

Log file	Contents
detail.log	This file contains specific information about the upgrade, including the following: • Almost all information in upgrade.log. • Name of the file being purged. For example, “dbdict: upgradestatus is purged.” • Progress of a file loading. For example, “Adding record # 100 from table upgradeobjects”. • Changes made to fields during file processing. For example, “Increasing field length for incidentlib.company in kmquery dbdict from VARCHAR(40) to VARCHAR(70)” • Signature of a file and the action on it. For example, “Processing Format Record : cc.get.dependen, signature=(current=3843738292, oob=NONE, upgrade=3843738292), upgraderesult=current”

Log file	Contents
except.log	This file contains information about any exceptions reported by the upgrade, including the following: • Messages about data type mismatches that failed to be resolved, or database dictionaries failed to be upgraded. For example, "dbdict:FolderRights, field:delete, field type is logical -- expected to be:character" See Data type mismatches. • Messages about the unique key changes that failed to be resolved. For example, "dbdict:Todo, Unique Key is {"record.id", "itemType"} -- expected to be:{"record.id"}" If there are exceptions logged in this file, we recommend you to resolve them in the "Resolving exceptions and conflicts" phase.

Log file	Contents
upgrade.log	This file contains information about where the upgrade is at any point. This file contains only the main steps of the upgrade, including the following: • Starting and ending of each sub-phase. For example, “**** Start Phase [Pre Upgrade Action Update] ****” • Main activities during each sub phase. For example, “Purging upgrade files...” • Number of files to be processed. For example, “There are 608 dbdicts to be processed.” • Names of the files being processed. For example, "Processing dbdict, AdvFilter”

Log file	Contents
pdmigration.log	The log file, which is stored in the Service Manager log directory, includes the following information: • A list of Alert definition records that are created based on settings in your Classic categories • A list of category, subcategory, and producttype records that are migrated to Process Designer-based tables • A list of format level link records that are updated to disable the category, subcategory, and product.type link lines • A list of the Classic probsummary and incident objects that are migrated to Process Designer-based objects • A list of updated process records

1.3.5.6. Step 6: View the upgrade results

While you are applying an out-of-box upgrade on the development system, the Upgrade Utility stores information regarding the upgrade result of each object. You can access this information in the Upgrade Utility through **View Upgrade Results and Merge Conflicts**.

To view the upgrade results, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **View/Merge Results**.
2. If you are upgrading from SM9.3x with PDCP3 installed or non-PD environment or SM9.4x Classic, you can see a wizard page. Select **View Upgrade Results and Merge Conflicts**, and then click **Next**.

   > **Note**
   >
   > This screen appears only after Process Designer Hybrid migration.

3. In the **Result** drop-down list, select the type of results you want to search for.

   Example: Renamed

   When you select a result type from the drop-down list, the description of that result type appears under the drop-down list.
4. Click **Search**.
5. A list is returned that displays all the result records of the specified type.

   > **Note**
   >
   > Some types of results are only informational and don't require any follow-up action.

Description of upgrade results

The search criteria, search results, and a description of the applicable action for each result are described in the table below.

Field	Definition
Object Type	Enter the type of object you want to search for, or leave this field blank to return all object types. The object types you could search for include Applications Cluster, Object, Process, ScriptLibrary, displayoption, format, formatctrl, help, joindefs, link, scmessage, screlconfig, triggers, validity, and wizard.
Object Name	Enter the name of the object you want to search for, or leave this field blank to return objects with any name. The object name is typically the unique identifier in the database table specified for the object type. The unique identifier for some object types (for example, format) may contain concatenate values of multiple fields according to the key setting in the signaturemake signature definition table.
Result: Added	Select this option to search for new objects that the Upgrade Utility added to the system. These objects did not exist in your system before this upgrade. No further action is necessary for these objects.

Field	Definition
Result: Already Current	Select this option to search for objects that were already the latest version. No further action is necessary for these objects.
Result: Auto Merged	Select this option to search for objects that the Upgrade Utility automatically merged by using your local version, the out-of-box version and the upgrade version of the objects. **Note** The result occurs only after applying the first out-of-box upgrade; it no longer occurs after applying the custom upgrade. **Required Action**: If the object was not merged the way you expect, use the **Revert** or **Mass Revert** option from the options menu to revoke the auto-merge and merge the objects manually.
Result: Error	Select this option to search for objects that encountered an error while being updated by the Upgrade Utility. For more information about the error, review the sm.log and except.log files. **Required Action**: Fix the cause of the error, or apply the upgrade again in a copy of your production system if it is needed.

Field	Definition
Result: Forced	Select this option to search for objects that were not only tailored on your Service Manager system but also changed in the upgrade version. After the upgrade, your objects were automatically replaced with the objects in your custom upgrade package. The Upgrade Utility copied your object of the original version to its revision. **Note** This result occurs only after applying the custom upgrade. No further action is necessary for these objects.

Field	Definition
Result: Kept Customer	Select this option to search for objects that were tailored on your Service Manager system but not changed on the upgrade version. These objects were not changed. **Note** The result occurs only after applying the first out-of-box upgrade; it no longer occurs after applying the custom upgrade. **Tip** If an object is Applications Cluster and you want to use the upgrade version, you can select **Choose Upgrade**. The object <*object name*> is then renamed to PRE<*old version number*><*object name*> and the object NEW980<*object name*> is renamed to <*object name*>. No further action is necessary for these objects.

Field	Definition
Result: Kept Customer Non-OOB	Select this option to search for objects that did not exist in the original version but were added on your Service Manager system, and also added on the upgrade version. **Note** • The result occurs only after applying the first out-of-box upgrade. It no longer occurs after applying the custom upgrade. • There is no such result when upgrading from SM9.7x. The corresponding result is **Renamed**.

Field	Definition
	 **Tip** If you want to use the upgrade version for the object, you can select **Choose Upgrade**. If the object is Application Cluster, the object <object name> is then renamed to PRE<old version number><object name> and the object NEW980<object name> is renamed to <object name>. If the object is not Application Cluster, the object NEW980<object name> is copied to replace <object name>. The result is then set to **Replaced**. **Required Action**: Choose one of the following solutions for each object with this result. • Keep the old version — No further action is necessary. • Keep the new version — Select the object in the merge view and click **Copy all from left to right** on the toolbar. • Merge new and current versions — Determine which of the new features need be incorporated into your tailored object, and then modify the tailored object. When finished, delete the new object NEW980<object name> and the

Field	Definition
	copied object PRE<old version number><object name>.
Result: Merged	Select this option to search for dbdict objects that were tailored on your Service Manager system, which Upgrade Utility has merged with the version in this upgrade. **Required Action**: Test these objects, and when satisfied change their result to **Reconciled**.

Field	Definition
Result: Previously Reconciled	Select this option to search for objects that were tailored on your Service Manager system, that were marked as Reconciled during a previous upgrade or patch release, or where your object was not changed and the Upgrade Utility added a new object NEW980<object name>. **Note** The result occurs only after applying the out-of-box upgrade; it no longer occurs after applying the custom upgrade. **Required Action**: Choose one of the following for each object with this result. • Keep the old version — No further action is necessary. • Keep the new version — Select the object in the merge view and click **Copy all from left to right** on the toolbar. • Merge new and old versions — Determine which of the new features should be incorporated into your tailored object, and then make the changes in your tailored object. When finished, delete the new object NEW980<object name> and the copied object PRE<old version number><object name>.

Field	Definition
	Tip If you want to use the upgrade version for the object, you can select **Choose Upgrade**. If the object is Application Cluster, the object <object name> is then renamed to PRE<old version number><object name> and the object NEW980<object name> is renamed to <object name>. If the object is not Application Cluster, the object NEW980<object name> is copied to replace <object name>. The result is then set to **Replaced**.
Result: Reconciled	Select this option to search for objects that you have already marked as Reconciled. **Note** The result occurs only after applying the out-of-box upgrade; it no longer occurs after applying the custom upgrade. No further action is necessary for these objects.

Field	Definition
Result: Renamed	Select this option to search for objects that were not only tailored on your Service Manager system but also changed on the upgrade version. After upgrade, your tailored object was not changed, the Upgrade Utility added a new object NEW980<object name> and copied your tailored object as a backed up object PRE<old version number><object name>. **Note** The result occurs only after applying the out-of-box upgrade; it no longer occurs after applying the custom upgrade. **Required Action**: Choose one of the following for each object with this result. • Keep the old version — No further action is necessary. • Keep the new version — Select the object in the merge view and click **Copy all from left to right** on the tool bar. • Merge new and old versions — Determine which of the new features should be incorporated into your tailored object, and then make the changes in your tailored object. When finished, delete the new object NEW980<object name> and the copied object

Field	Definition
	PRE<old version number><object name>.  **Tip** If you want to use the upgrade version for the object, you can select **Choose Upgrade**. If the object is Application Cluster, the object <object name> is then renamed to PRE<old version number><object name> and the object NEW980<object name> is renamed to <object name>. If the object is not Application Cluster, the object NEW980<object name> is copied to replace <object name>. The result is then set to **Replaced**.

Field	Definition
Result: Upgraded	Select this option to search for objects that were automatically replaced with the upgrade version objects. These are objects that were not tailored on your Service Manager system, but changed on the upgrade version. **Note** The result occurs only after applying the out-of-box upgrade; it no longer occurs after applying the custom upgrade. No further action is necessary for these objects.

Field	Definition
Result: Replaced	Select this option to search for objects that were not only tailored on your Service Manager system but also changed on the upgrade version and automatically replaced with the upgrade version objects. These objects have 3 types: RAD Application, ScriptLibrary with labels of Proprietary, and the objects that are necessary for Process Designer functionality. After upgrade, your tailored object was renamed to PRE<old version number> <object name> and the Upgrade Utility added a new object <object name>. **Note** The result occurs only after applying the out-of-box upgrade; it doesn't occur after applying the custom upgrade. **Required Action**: Do the following for the relevant object with this result. • ScriptLibrary - check the new and old versions — Determine which of the old tailoring should be incorporated into the upgraded object, and then make the changes in your upgraded object. When finished, delete the new object NEW980 and the copied object PRE. • Others - No further action is necessary for these objects.

Field	Definition
	Tip If you don't want to replace the objects, you may select **Revert**. If the object is Application Cluster, the object <object name> is then renamed to NEW980<object name> and the object PRE<old version number><object name> is renamed to <object name>. If the object is not Application Cluster, the object PRE<old version number><object name> is copied to replace <object name>. The result is then set back to **Renamed**, **Previously Reconciled**, **Kept Customer** or **Kept Customer Non-OOB**.

See the following table for some examples about how the upgrade result types are marked for an existing record.

OOB record (9.30)	OOB record (9.33)	Customer record (9.33)	OOB record (9.80)	Upgrade Utility check sequence	Upgrade Utility check condition	Conflict flag	Upgrade result type	Upgrade result value
111	112	112	112	1	Customer record (9.33) = OOB record (9.80)	No	Already Current	112

OOB record (9.30)	OOB record (9.33)	Customer record (9.33)	OOB record (9.80)	Upgrade Utility check sequence	Upgrade Utility check condition	Conflict flag	Upgrade result type	Upgrade result value
111	112	112	113	2	Customer record (9.33) != OOB record (9.80) and Customer record (9.33) = OOB record (9.33)	No	Upgraded	113

OOB record (9.30)	OOB record (9.33)	Customer record (9.33)	OOB record (9.80)	Upgrade Utility check sequence	Upgrade Utility check condition	Conflict flag	Upgrade result type	Upgrade result value
111	112	113	112	3	Customer record (9.33) != OOB record (9.80) and Customer record (9.33) != OOB record (9.33) and OOB record (9.33) = OOB record (9.80)	No	Kept Customer	113

OOB record (9.30)	OOB record (9.33)	Customer record (9.33)	OOB record (9.80)	Upgrade Utility check sequence	Upgrade Utility check condition	Conflict flag	Upgrade result type	Upgrade result value
111	112	113	114	4	Customer record(9.33)!=OOB record(9.80) and Customer record(9.33)!=OOB record(9.33) and OOB record(9.33)!=OOB record(9.80)	Yes	Renamed	113

OOB record (9.30)	OOB record (9.33)	Customer record (9.33)	OOB record (9.80)	Upgrade Utility check sequence	Upgrade Utility check condition	Conflict flag	Upgrade result type	Upgrade result value
		113	112	5	OOB record (9.33) doesn't exist and Customer record (9.33) != OOB record (9.80)	Yes	Kept Customer Non-OOB	113

Note

When upgrading from SM9.6x, the upgrade result for **Kept Customer Non-OOB** is **Renamed** instead.

1.3.5.7. Step 7: Manage the upgrade result data

To manage the upgrade result data more easily, do either of the following:

- Open the Upgrade Results list, click **More** or the More Actions icon, and click **Export to Excel** to manage the data in a Microsoft Excel document, or
- Open the Upgrade Results list, click **File** > **Print** > **List View** to print the list of records.

After you complete an applications upgrade on your development system, you are ready to resolve the exceptions and conflicts that originate from tailoring on an upgraded object. Before resolving conflicts, Service Manager features may not function as expected.

1.3.5.8. Step 8: Resolve exceptions

Exceptions are logged if the Upgrade Utility can't add or update an object. After running an upgrade, you can identify exceptions by viewing error messages in the except.log or sm.log file. These exceptions are reported in the Upgrade Results list as "Error."

Data type mismatches

If the data type of a field in your dbdict doesn't match the data type of the like-named field defined in the dbdict provided by the upgrade package, the Upgrade Utility can't merge these dbdicts. For example, if an existing dbdict has a scalar field and the Upgrade Utility attempts to add a structure field with the same name, this discrepancy prevents the dbdict from being updated.

To fix this issue, you can change the data type and the SQL type of the field, and use Complex Update to migrate existing data on that field to the target data type. Follow these steps to modify field data type by using the Dbdict Utility:

1. Navigate to **System Navigator** > **Tailoring** > **Database Dictionary**.
2. Type the dbdict name in the File Name field, and then click **Search**.
3. Double-click the field you want to change the data type, and then make updates as expected.
4. Save your changes.

The following is an example that shows the process of fixing a typical data type mismatch.

Note

The error messages in the except.log file identify each data type by an index number:

Index number	Data type
1	number
2	character
3	date/time
4	logical
8	array
9	structure
11	expression

The following table displays a list of data type mismatches that may appear in the except.log file for an Oracle database. Only some examples are listed for each category of data type mismatch. If you are using an MSSQL database, the actual error message may vary slightly. For example, the following list highlights the different errors in the different databases:

- Oracle Error : dbdict:ApprovalDef, field:appr.condition, SQL type is CHAR(1) -- expected to be:RAW(255)
- MSSQL Error: dbdict:ApprovalDef, field:appr.condition, SQL type is CHAR(1) -- expected to be:VARBINARY(255)

Error message in Oracle:	Solution:
dbdict:incidents, field:svc.options, SQL type is VARCHAR2(90) -- expected to be:BLOB dbdict:SMISTaskQueue, field:failedReason, SQL type is VARCHAR2(60) -- expected to be:BLOB dbdict:Workflow, field:description, SQL type is VARCHAR2(255) -- expected to be:BLOB	To fix these issues, change the SQL type to RAW(255) or BLOB by using the Dbdict utility. Additionally, you will need to set the “SQL RC” to true to allow the field to store RAD expressions. Note that the stored value of the field in the database is encoded by Service Manager. **Note:** If you change the value of "SQL RC", we suggest changing this value back to the original value after updating the SQL type.
dbdict:cm3eventack, field:number, field type is number -- expected to be:character	To fix this issue, change the field type to character by using the Dbdict utility.
dbdict:Rule, field:tablename, SQL type is VARCHAR2(60) -- expected to be:CLOB	Ignore the error message. No further action is necessary.
dbdict:svcItemCount, field:access.filter, SQL type is RAW(255) -- expected to be:CLOB	To fix these issues, change the SQL type to CLOB by using the Dbdict utility. Additionally, you need to switch the value of “SQL RC” to update the type. **Note:** If you change the value of "SQL RC", we suggest changing this value back to the original value after updating the SQL type.

Error message in Oracle:	Solution:
dbdict:eventin, field:evnumber, field type is number -- expected to be:character dbdict:eventout, field:evnumber, field type is number -- expected to be:character	These fields reside in the descriptor structure field of BLOB SQL type. To fix this issue, change the field type to character by using the Dbdict utility.
dbdict:svcCatLanguage, field:catalogid, field type is character -- expected to be:number	This field resides in the catalog structure field of BLOB SQL type. To fix this issue, change the field type to number by using the Dbdict utility.
dbdict:licenseinfo, field:id, field type is character -- expected to be:number	The licenseinfo table is used to track license information by Service Manager server. This issue should be ignored.
dbdict:svcCatalog, field:id.attach, field type is character -- expected to be:number	This id.attach field is an alias of id field in svcCatalog table. To fix the issue, change the field type to by using the Dbdict utility.
dbdict:FolderRights, field:close, field type is logical -- expected to be:character	The close field is an alias of delete field in FolderRights table. To fix the issue, change the field type to character by using the Dbdict utility.
dbdict:FolderRights, field:delete, field type is logical -- expected to be:character	This issue can be fixed by following the steps in the **Fixing the FolderRights delete field** section below.

Error message in Oracle:	Solution:
dbdict:cm3r, field:assets, SQL type is CLOB -- expected to be: VARCHAR2(200) dbdict:cm3r, field:assets, field type:character, SQL type is CLOB -- expected to be:VARCHAR2(200), SQL table is m1 -- expected to be:a3	This kind of error message is generated because the Upgrade Utility doesn't remap the field of array type from the main table to the alias table and the subsequent full-table copy may slow down the upgrade. Follow these steps to fix the issue: 1. Double-click the field of array type which contains this field in the related table, and then update the value in “SQL Table” to a<*number*> which is not used in the alias table. For example, a3. 2. Double-click the field of character type in the related table. Update the value in “SQL TYPE” to the expected value, and then update the value in “SQL Table” to the same a<*number*> which is not used in the alias table. For example, a3. 3. Click **OK** to save your changes.  **Caution** If the related table contains more than 100,000 records, fixing the data type mismatches in the field may take more than one hour.

Error message in Oracle:	Solution:
dbdict:inbox, Primary Key is inbox.id -- the primay key can't be added. Since SM doesn't support operation on table with primary key when applications version is older than 9.32, then add inbox.id as unique key.	To fix this issue, change the unique key to primary key by using the Dbdict utility.
Update field type for record.number in kmreindex dbdict from number to character -- You need to update the relevant data in dbdict kmreindex manually.	The field type is updated from number to character. You need to check whether the data values are updated and then update the data values to the correct ones.
Failed to add Primary key: {"mailno"} to table mail. You must add it manually.	Check whether the primary key is added to dbdict mail. If not, please check whether the values of this field are filled for all records in this table. You need to manually fill this field and add the primary key by using the Dbdict utility.

Note

Ignore the message if the child fields of an array of structure field have been mapped to the database columns. For example:

```
dbdict:computer, field:video, SQL type is EMPTY --
expected to be:BLOB
dbdict:computer, field:vid.card.id, SQL type is
VARCHAR2(30) -- expected to be:EMPTY
dbdict:computer, field:vid.card.bios, SQL type is
VARCHAR2(30) -- expected to be:EMPTY
dbdict:computer, field:vid.manufacturer, SQL type is
VARCHAR2(30) -- expected to be:EMPTY
dbdict:computer, field:vid.model, SQL type is
VARCHAR2(30) -- expected to be:EMPTY
dbdict:computer, field:vid.chip.type, SQL type is
VARCHAR2(30) -- expected to be:EMPTY
dbdict:computer, field:vid.memory, SQL type is NUMBER -
- expected to be:EMPTY
dbdict:computer, field:vid.resolution, SQL type is
VARCHAR2(30) -- expected to be:EMPTY
dbdict:computer, field:vid.resolution.x, SQL type is
VARCHAR2(30) -- expected to be:EMPTY
dbdict:computer, field:vid.resolution.y, SQL type is
VARCHAR2(30) -- expected to be:EMPTY
```

Fixing the FolderRights delete field

Example: The dbdict for the FolderRights table has a delete field with the "logical" data type. The Upgrade Utility tries to update the delete field with the "character" data type, which has possible values of "always," "never," "workgroup," and "assigned."

Note

The field type of delete field in FolderRights dbdict may be updated to character. There is another information in except.log:

`Update field type for delete in FolderRights dbdict from logical to character -- You need to update the relevant data in dbdict FolderRights manually.`

The value of the field also need to be updated following the steps manually.

1. In the dbdict for the FolderRights table, add a field named delete.tmp with a data type of character, and update the dbdict.
2. Log out of the system and log back in.
3. Make sure that the Complex Update feature is enabled for the FolderRights table.

   Note

   To enable the Complex Update feature, open the FolderRights Format Control record, go to **Privileges** and open the **Advanced** tab, and then update the values in the Simple Mass Update field and in the Complex Mass Update field to `true`.

4. In the Database Manager, search for all records in the FolderRights table.
5. From the More Actions menu, click **Mass Update**.
6. When you are asked whether you want to update all records, click **Yes**.
7. Click **Complex Update** on the toolbar.
8. In the statements area under **Instructions for action on EACH RECORD**, add statements using standard RAD expressions to migrate data from the delete field to the delete.tmp field.

   Example:

   `if delete in $file=true then delete.tmp in $file="always" else delete.tmp in $file="never"`

9. In the dbdict for the FolderRights table, edit the **delete** field and add the **Type** (character) and **SQL Type** (same as the SQL Type automatically assigned for delete.tmp).
10. Log out of the system and log back in.

11. In the Database Manager, search for all records in the FolderRights table.
12. From the More Actions menu, click **Mass Update**.
13. Click **Complex Update** on the toolbar.
14. In the statements area under **Instructions for action on EACH RECORD**, add statements using standard RAD expressions to migrate data from the delete.tmp field to the delete field and empty the delete.tmp field.

    Example:

    ```
    delete in $file=delete.tmp in $file; delete.tmp in $file=NULL
    ```

15. In the dbdict for the FolderRights table, follow these steps to remove the delete.tmp field:

    1. In the Database Manager, search for all records in the dbdict table.
    2. Select the dbdict format.
    3. Type FolderRights in the **Name** field, and then click **Search**.
    4. Locate the row for the delete.tmp field. Remove the values in the Name, Type, Index, and Level fields respectively.
    5. Click **Save** and **OK**.
16. Log out of the system and log back in.
17. Test the change by updating records in the FolderRights table and populating the **delete** field with "always," "never," "workgroup," or "assigned."

Add new object record failure

All the upgraded objects are saved in the upgradeobjects table, and you need to add some objects from this table to the current upgradeobjects table. If the target table contains more than one unique type key, some errors may occur when adding new objects during the upgrade process. Refer to the following error message as an example.

```
2014-03-20 14:04:29 file:inbox, updated the field inbox.id value from 10000313 to 14042954310000313
```

The value of the inbox.id field is 10000313 in this example, which is used in the current upgradeobjects table. If the Upgrade Utility adds an inbox record that contains the same value for this field, the current value of the inbox.id field is automatically prefixed with a timestamp to avoid duplication errors. The timestamp is composed of hour (two bits), minute (two bits), second (two bits) and millisecond (three bits). In this example, the timestamp is 140429543.

After upgrading, you can update the temporary value of the inbox.id field as necessary.

Handle key change failure

If the Upgrade Utility fails to apply certain key changes, error information is logged into the except.log file. Review the log file and make appropriate operations.

Important

If you are upgrading from applications version earlier than 9.32, we recommend you to enable primary keys after the applications upgrade and the main steps are as follows:

1. Apply the out-of-box upgrade
2. Build a custom upgrade
3. Apply the custom upgrade
4. Enable primary keys

You can review the except.log file and then refer to Working with primary keys for conversion instructions.

Error message 1: Failed to add <key_type> key: <field_name> to table <table_name>. You must add it manually.

Follow these steps to handle this error:

1. Type dbdict in the Service Manager command line, and press **Enter**.
2. In the **File Name** field, type the table name indicated by the error message, and click **Search**.
3. Click the **Keys** tab.
4. Position the mouse point in the key name part of an empty key structure, and click **New Field/Key**.
5. Select the appropriate key type from the combo list, and add the appropriate field to the key, as indicated by the error message.
6. Click **Add**, and click **Yes** to confirm.
7. Click the **Keys** tab, and click **OK** to save the change.
8. Continue to follow *steps 1* through *step 7* for each key that failed to be added.

Error message 2: Failed to update <key_type> key: <old_field_name> to <new_field_name> in table <table_name>. You must update it manually.

Follow these steps to handle this error:

1. Type dbdict in the Service Manager command line, and press **Enter**.
2. In the **File Name** field, type the table name indicated by the error message, and click **Search**.

3. Click the **Keys** tab.
4. From the key list, select the key name that's indicated by the error message, and click **Edit Field/Key**.
5. Update the fields in the key according to the field names indicated by the error message.
6. Click **OK**, and click **Yes** to confirm.
7. Click the **Keys** tab, and click **OK** to save the change.
8. Continue to follow *steps 1* through *step 7* for each key that failed to be updated.

Error message 3: `Failed to remove <key_type> key: <field_name> from table <table_name>. You must remove it manually.`

Follow these steps to handle this error:

1. Type `dbdict` in the Service Manager command line, and press **Enter**.
2. In the **File Name** field, type the table name indicated by the error message, and click **Search**.
3. Click the **Keys** tab.
4. From the key list, select the key name that's indicated by the error message, and click **Edit Field/Key**.
5. Click **Delete**, and click **Yes** to confirm.
6. Click the **Keys** tab, and click **OK** to save the change.
7. Continue to follow *steps 1* through *step 6* for each key that failed to be removed.

In addition, you may encounter the following Unique Key errors:

Error: `dbdict:Approval, Unique Key is {"unique.key", "file.name", "name"} -- expected to be:{"unique.key", "file.name", "name", "component"}`

Error: `dbdict:ApprovalLog, Unique Key is {"counter", "file.name", "unique.key"} -- expected to be:{"counter", "file.name", "unique.key", "component"}`

Unexpected errors

If the except.log file or the Upgrade Results list reports any errors other than data type mismatches, review the sm.log file for more information, and if needed, contact Customer Support for assistance.

1.3.5.9. Step 9: Resolve conflicts

In this step, you are going to resolve the conflicts between your tailored objects and the new objects provided in the upgrade package.

Standard conflict resolution process

The standard process of resolving conflicts are listed as follows:

1. To view the conflicts, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **View/Merge Results**.
2. (Optional) Select **View Upgrade Results and Merge Conflicts**, and then click **Next**.

   Note

   This screen appears only after Process Designer Hybrid migration.

3. Search for records with a status of "Renamed."
4. For each renamed object, you can choose one of the following options.
   - **Option 1:** Use your customized object instead of the new object.

     In this case, delete the new object that's prefixed with NEW980 and the copied object that's prefixed with PRE<*version_number*>.
   - **Option 2:** Use the new object instead of your customized object.

     In this case, delete your customized object and the copied object that's prefixed with PRE<*version_number*> and then rename the new object by removing the NEW980 prefix, or you can select **Choose Upgrade** to replace the object with the new object that's prefixed with NEW980.

     The Mass Choose Upgrade feature can help you replace multiple customized objects with new objects that are prefixed with NEW980. For more information, see *Using the Mass Choose Upgrade feature* below.
   - **Option 3:** Merge the changes shipped with the new object into your customized object.

     In this case, find out what changes the new object includes, manually apply those changes to your customized object, and then delete the new object

that's prefixed with NEW980 and the copied object that's prefixed with PRE<*version_number*>.

> **Note**
>
> The Merge tool, Auto-Merge and Revert options, and third-party three-way compare and merge tools can assist you in comparing the objects and merging the code during the out-of-box upgrade. See *Using the Merge tool* and *Using the Auto Merge and Revert options* below for more information.

5. After resolving each conflict, you must mark the object as "Reconciled." Marking an object as reconciled will remove it from the current conflict list and provide the version with which the object was last reconciled.

> **Note**
>
> The Mark as Reconciled feature can assist you to mark the object as Reconciled. For more information, see *Using the Mass Mark as Reconciled feature* below.

For more specific guidelines and examples for conflict resolution, see the Conflict Resolution for Upgrade to Service Manager (SM) 9.3x white paper.

Display components

The Upgrade Results list also includes conflicts in display components. You can follow the standard conflict resolution process to resolve these conflicts. However, you must pay special attention to following types of display components:

- displayscreen records
- displayoption records
- displayevent records

Display screen records

Display screens are individual records identified by a unique screen ID. The displayscreen records define the attributes of a screen and provide access to the individual records for options and events. A display screen is not the same as a form.



Caution

There are triggers attached to the displayscreen file. Changes to the records in this file affect their associated display options and events.

If any displayscreen objects appear in your upgrade results list, perform conflict resolution on those records by following the standard conflict resolution process.

Display options and display events

The Upgrade Utility upgrades the display components in the same manner as all other components. The displayoption table and the displayevent table have a unique identifier, stored in the ID field. The upgrade process assigns an ID to every display option following the pattern: **<screen id>_<action>_<number>**, where the screen ID and action are from the display option (or event), and the number is an optional field added when multiple options have the same screen ID and action.

If options that have been added to your system have the same action as others in the same display screen, the upgrade process assigns a <number> in the order of the options' GUI option number.

If an added option was not the last option in terms of GUI option number, the upgrade process doesn't add the additional numbers in the ID field in the same order as they would have for an out-of-box system. The upgrade process renames the added option and any option after it (in GUI option order), it doesn't upgrade them automatically.

To ensure that this type of renaming doesn't happen in future upgrades, when performing conflict resolution on these options, use the ID of the renamed option, **NEW980<screen id>_<action>_<number>** and manually change the identifier of the added options. Rename all other options to match the ID of the renamed ones.

When renaming an option, use an identifier to specify that this is a customized option, added for your installation. For example, an ID might look like: “**apm.edit.problem_don'thing_ACME1**”.

This table gives an example of part of the display screen conflict resolution for **apm.edit.problem**.

Example conflict resolution for the apm.edit.problem display option

Screen ID	GUI Action	Upgrade Action	
300	do nothing	update	**Name:** apm.edit.problem_do_nothing_1 **Result:** This item was updated correctly. **User Action:** No action necessary.
400	do nothing	update	**Name:** apm.edit.problem_do_nothing_2 **Result:** This item was updated correctly. **User Action:** No action necessary.

Screen ID	GUI Action	Upgrade Action	
450 This is an option you added.	do nothing	rename	**Name:** apm.edit.problem_do_nothing_3 **Result:** This item was renamed. It is your customized option. **User Action:** Rename this object to give it a unique new name, such as: apm.edit.problem_do_nothing_ACME1 **Name:** NEW980apm.edit.problem_do_nothing_3 **Result:** This is the new SM 9.80 option. **User Action:** Perform conflict resolution. To perform conflict resolution, open apm.edit.problem and look at the options. Compare this option with apm.edit.problem_do_nothing_3

Screen ID	GUI Action	Upgrade Action	
			and NEW980apm.edit.problem_do_nothing_3
500	do nothing	The upgrade ignores this option.	**Result:** This option doesn't appear in the reports. **User Action:** Perform conflict resolution. To perform conflict resolution, open apm.edit.problem and look at the options. Compare this option with apm.edit.problem_do_nothing_3 and NEW980apm.edit.problem_do_nothing_3

Using the Merge tool

For each object marked as "Renamed" in the Upgrade Results list, the Upgrade Utility generates XML objects for the three versions of the object: base, customer, and upgrade.

Version	Location	Description
base	*Upgrade*\3waymerge\work\base	An XML representation of every object that has been signatured in the pre-upgrade out-of-box version.
customer	*Upgrade*\3waymerge\work\customer	An XML representation of all objects that were tailored in the customer version and resulted in a conflict during the upgrade.
upgrade	*Upgrade*\3waymerge\work\upgrade	An XML representation of the object provided by the upgrade package of all objects resulted in a conflict.

Each of the three folders described above contains a sub-folder for each signatured table. You can find the XML representations of the objects in the table within these sub-folders.

The built-in, Two-Way/Three-Way Merge tool allows you to examine the upgrade and customer versions of a record in a side-by-side view as well as the base, upgrade, and customer versions of a record in a three-way view. This will help you to determine which changes to include in the final record.

This tool assists the conflict resolution process in these two ways:

- It allows you to identify where changes are located before you can visually compare the objects and to make changes manually, such as in format records.
- It allows you to identify and merge changes directly between objects, such as ScriptLibrary records.

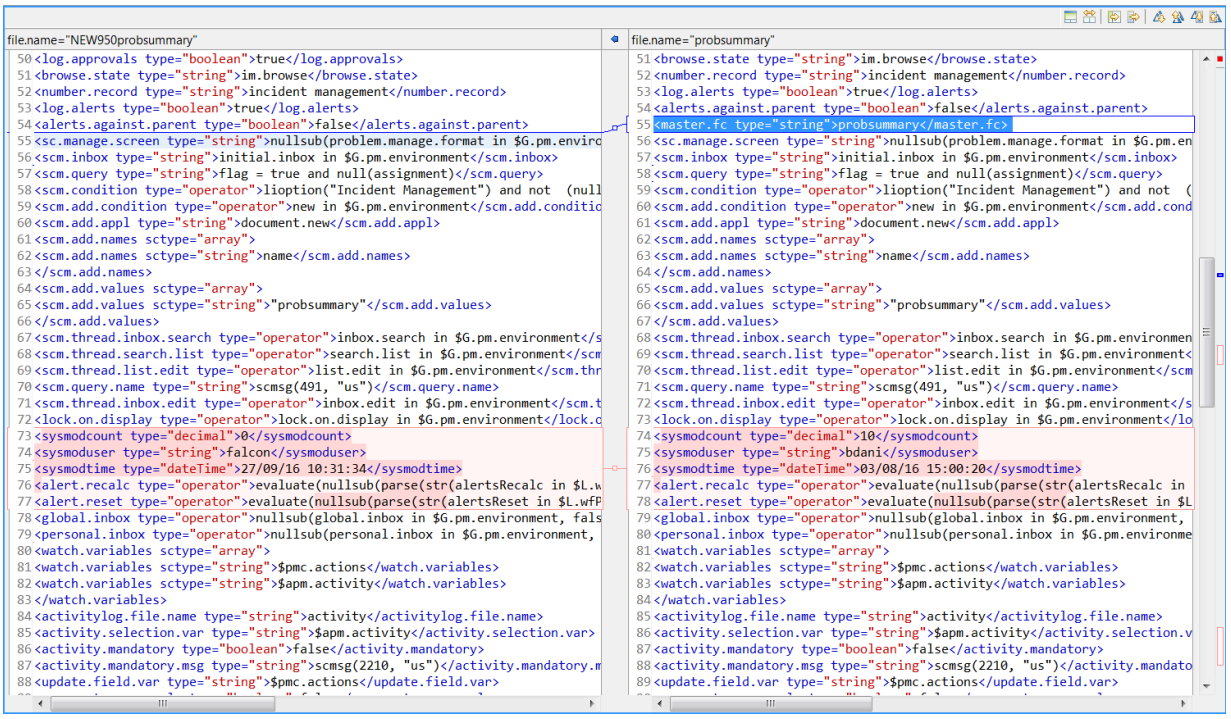
To use the Merge tool, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **View/Merge Results**.

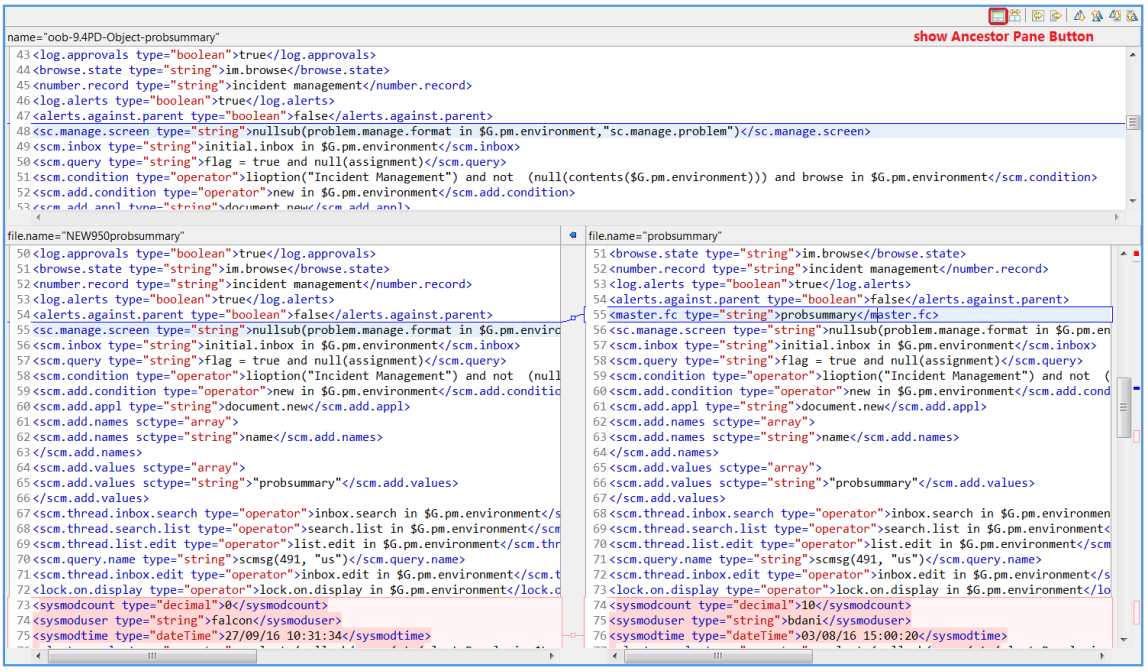
2. In the **Result** drop-down list, select **Renamed**.

> **Note**
>
> The Two-way/Three-way Merge tool is also available for "Previous Reconciled", "Kept Customer Non-OOB", "Auto Merged" and "Replaced" records.

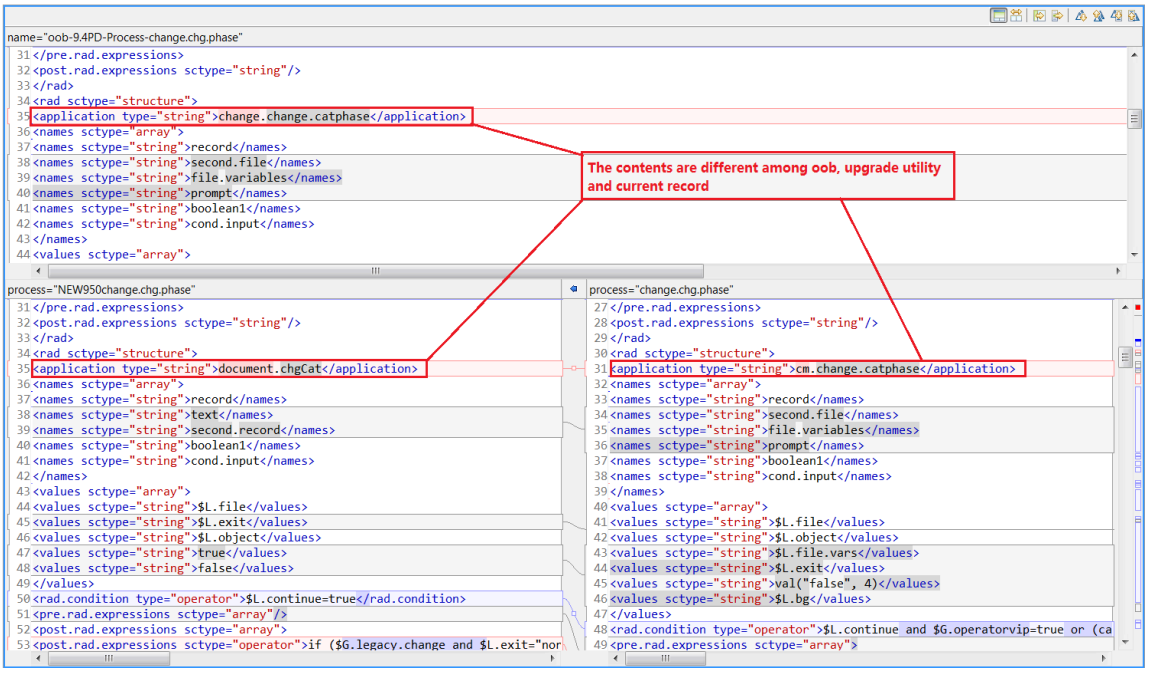
3. Click **Search**.
4. A list is returned that displays all the result records of the “Renamed” type. Select a record that you want to examine, and then click **Merge** on the toolbar.
5. The current default merge mode is Three-Way Merge mode. The merge option is unavailable for format records. Instead, you can click **Compare** option from the **More Actions** menu to start the merge tool in read-only mode. You can use this mode to identify the differences in the side-by-side view or three-way view and then merge records manually in Forms Designer.



1. In Three-Way Merge mode, click the **Show Ancestor Pane** button to show the base record reference. Click the button again to hide the base record reference.



2. In Three-Way Merge mode, if the records of base, upgrade, and customer versions are all different, the background color is red. If you put the cursor into the records with the red background and then click the **Copy Current Change from Left to Right** button, the selected records from the left pane will be appended to selected records in the right pane.



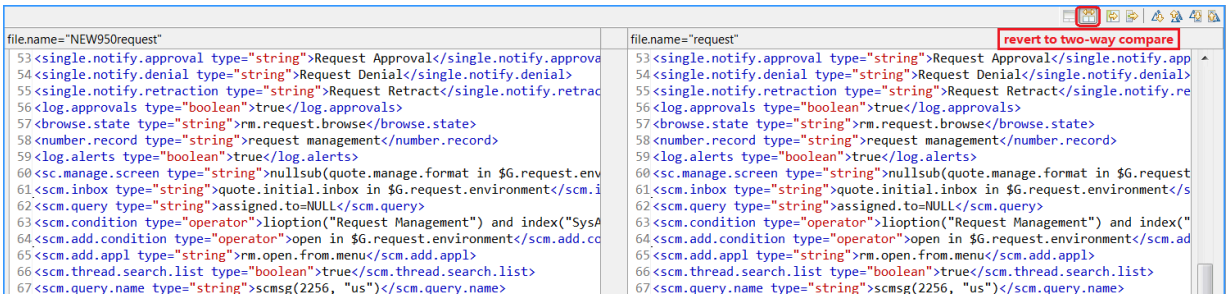
3. In Three-Way Merge mode, if the records of the customer version are identical to the base version, but different from the upgrade version, the background color is blue. If you put the cursor into the records with the blue background, and then click the **Copy Current Change from Left to Right** button the selected records from the right pane will be replaced by the selected records from the left pane.

4. In Three-Way Merge mode, if the records on the left contain red background records, and you then click the **Copy All from Left to Right** button, the records in the right pane with blue background will be replaced with selected records from the left pane.

> **Note**
>
> The right records with red background will not be changed. To do this, you will need to switch to Two-Way Merge mode to use Copy All from Left to Right feature.

6. To enter Two-Way Merge mode, click the **Two-Way Compare** (Ignore Ancestor) button. In this mode, the Show Ancestor Pane button is disabled.

   When you do this, the button label changes to Three-Way Compare. Click this again to revert to Three-Way Compare mode.

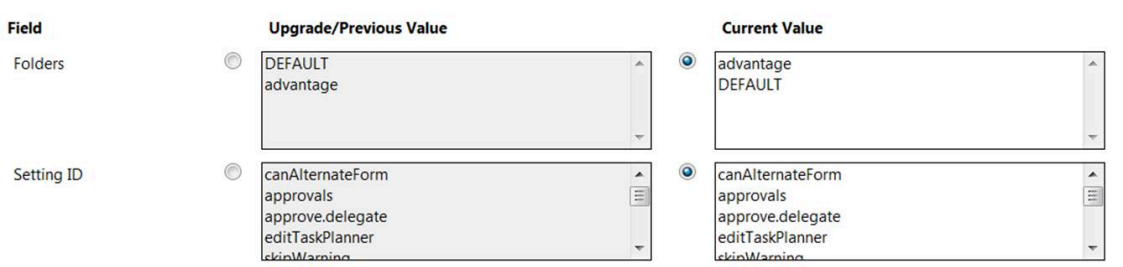


1. In Two-Way-Merge mode, if one record differs upgrade and customer versions, you can click **Copy Current Change from Left to Right** button to replace a selected record from the left pane to the right pane.

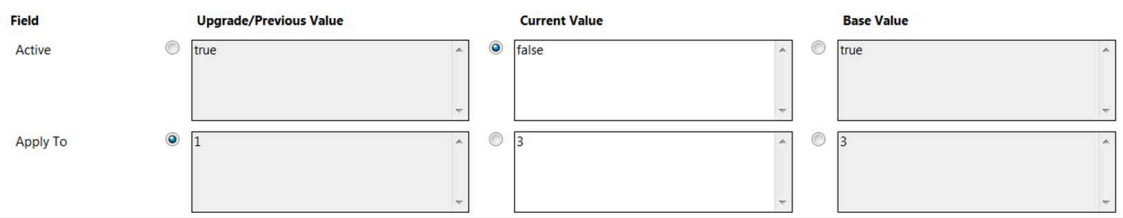
2. In Two-Way-Merge mode, click **Copy All from Left to Right** button to replace all records in the right pane with the records from the left pane.

Note

There is a Field-Level Merge mode for some simple structure records. This field compare mode displays either Two-Way-Merge or Three-Way-Merge. You can select the value you want to keep. Refer to the following screenshots:



The field compare mode displays Two-Way-Merge by default if there is no related out-of-box xml file in the <Upgrade Utility root folder>/3waymerge/oob/<base version>/<dbdict name> folder. Otherwise, the field compare mode displays Three-Way-Merge.



To switch between the field merge mode and the XML merge mode, click **XML Merge**.

Using a third party tool to visually compare objects

You may also visually compare the three versions of each object using a three-way compare and merge tool outside Service Manager, and then merge them manually in Service Manager. For example, you can use a tool, such as KDiff3 for Windows, to compare and merge objects.

To download and learn about KDiff3, visit the KDiff3 Web site.

A brief example of using KDiff3 to compare the three versions of an object is provided in the following steps:

1. Install KDiff3 on the Service Manager server host.
2. Open KDiff3. For more information, refer to the KDiff3 documentation.
3. For the A(Base) parameter, specify the path to the Upgrade\3waymerge\work\base folder.

4. For the B parameter, specify the path to the Upgrade\3waymerge\work\customer folder.
5. For the C parameter, specify the path to the Upgrade\3waymerge\work\upgrade folder.
6. Click **OK**.
7. Navigate the folder structure and double-click the file that's named after the object you want to merge.
8. Compare the three versions of the object in the 3-way compare view.
9. Manually apply the changes you have identified to the object in Service Manager.

Using the Auto Merge and Revert options

During upgrade processing, the Upgrade Utility attempts to merge your objects with the corresponding objects provided with the upgrade. You can search for records with a result type of "Auto Merged" in the Upgrade Results list to review these objects.



Note

An auto-merged object needs to be thoroughly tested for syntax and functional integrity.

If you encounter issues with objects that have been auto-merged or reconciled, you can click the **Revert** button on the toolbar to restore "Auto Merged" or "Renamed" objects, or "Replaced" Applications Cluster and ScriptLibrary objects to their previous states. Use **Revert** to restore one object and **Mass Revert** to restore multiple objects. After restoring an object, you can also re-attempt to auto-merge that object using the **Auto Merge** or **Mass Auto Merge** option from the More Actions menu.

Using the Mass Choose Upgrade feature

During the Upgrade process, you can use the Mass Choose Upgrade feature to overwrite your systems old objects with the newer versions from the upgrade utility. You can use this feature to quickly update the objects of the following statuses, which are generated during the upgrade:

- Auto Merged
- Renamed
- Previously Reconciled
- Reconciled
- Kept Customer Non-OOB

You can use this feature to quickly update Applications Cluster objects of the following statuses:

- Kept Customer
- Kept Customer Non-OOB
- Renamed
- Previously Reconciled

To use the Mass Choose Upgrade feature, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **View/Merge Results**.
2. (Optional) Select **View Upgrade Results and Merge Conflicts**, and then click **Next**.

   Note

   This screen appears only after Process Designer migration.

3. In the **Result** drop-down list, filter the set of objects (Auto Merged; Renamed; Previously Reconciled; Reconciled) on which you wish to use the Mass Choose Upgrade feature and then click **Search**.
4. If more than two objects exist in the resulting search, click the **Mass Choose Upgrade** button from **More Actions** menu in the returned list and then click **Yes**.

After you click **Yes**, the objects that you selected will be updated with the contents of the newer versions from the upgrade utility.

Using the Mark as Reconciled feature

During the Upgrade process, you must mark conflicting objects as “Reconciled” after resolving each conflict. To help with this process, you can use the Mass Mark as Reconciled feature to mark multiple objects as “Reconciled.” You can use this feature on objects with the following statuses:

- Auto Merged
- Renamed
- Previously Reconciled

To use the Mass Mark as Reconciled feature, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **View/Merge Results**.

2. If you are upgrading from SM9.3x with PDCP3 installed or non-PD environment or SM9.4x Classic, you can see a wizard page. Select **View Upgrade Results and Merge Conflicts**, and then click **Next**.

   > **Note**
   >
   > This screen appears only after Process Designer migration.

3. In the **Result** drop-down list, filter the set of objects (Auto Merged; Renamed; Previously Reconciled) on which you wish to use the Mass Mark as Reconciled feature and then click **Search**.

4. If more than two objects exist in the resulting search, click the **Mass Mark as Reconciled** button from the **More Actions** menu and then click **Yes**. After you click **Yes**, all objects that you selected will be marked as “Reconciled” and removed from current conflict list.

   > **Note**
   >
   > If only one object exits in the resulting search, or if you want to resolve conflicts for the selected objects individually, use the **Mark as Reconciled** button on the toolbar instead.

Resolve Process Designer related data

To resolve the Process Designer related data, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **View/Merge Results**.

2. If you are upgrading from SM9.3x with PDCP3 installed or non-PD environment or SM9.4x Classic, you can see a wizard page. Select **View Upgrade Results and Merge Conflicts**, and then click **Next**.

   > **Note**
   >
   > This screen appears only after Process Designer Hybrid migration.

3. Click the **Expert Search** button from the More Actions menu, type version.reconciled#"PD Enable" in the **Query** field, and then click **Search**.

4. In the search result, resolve the data by following the actions described in the table below.

Reconciled Version	Result	Action
PD Enable: Updated	Renamed	Merge the newer object from the Upgrade Utility to the object.
PD Enable: Replaced	Renamed	Overwrite your systems old object with the newer version from the Upgrade Utility.
PD Enable: Need to be removed	Any	Remove the objects manually.

Sample user scenarios

You need to manually resolve the conflicts when your custom functions are conflicted in the ScriptLibrary. The following five scenarios shed some light on how to solve your conflict when performing an upgrade.

1. An out-of-the-box function that exists in the old version conflicts with the same function in the new version.
   In this scenario, choose the function in the new version to resolve the conflict.
2. A customized out-of-the-box function that exists in the old version conflicts with the same function in the new version.
   In this scenario, merge these two functions to resolve the conflict.
3. A function doesn't exist in the old version but is available out-of-the-box in the new version.
   In this scenario, choose the function in the new version to resolve the conflict.
4. You have added a function that is unavailable in the out-of-the-box functions of the old version. However, this function is available out-of-the-box in the new version.
   In this scenario, compare the function between the two versions to resolve the conflict.
   1. If you use these functions to achieve the same result, you can merge them from a business perspective. We recommend using the function codes in the new version since it has a higher priority.

2. If you use these functions to achieve different results, choose the function from the new version to resolve the conflict. Meanwhile, rename the old function and then update the corresponding and related functions so that the old function still works.

5. An out-of-the-box function exists in the old version but no longer exists in the new version.
   In this scenario, we recommend keeping this old function. Of course, you may remove it if it's no longer used.

1.3.5.10. Step 10: Perform additional manual tasks

This section lists changes that can't be automated by the Upgrade Utility and changes that are required only for certain customers. Make these changes before testing and backing up your system.

Adopt the logical name solution

In previous versions of Service Manager, the logical.name field in the device table was used as both the unique identifier and display name for a CI, causing loss of CI data integrity across Service Manager modules and complexity of CI reconciliation between integration products (for example, Service Manager doesn't allow duplicate CI names while UCMDB does).

Service Manager 9.41 and later versions solves the problems described above with the logical name solution. Once you have upgraded to Service Manager 9.80 from an applications version prior to 9.41, you need to perform additional manual tasks to adopt the solution in your production environment. For a summary of this solution and details of these manual post-upgrade tasks, see the Service Manager Logical Name Solution white paper.

Adopt Smart Search

Service Manager 9.41 introduces Smart Search which enables you to search among a variety of content, including Service Manager records, SharePoint documents, static web pages, and KM documents. You can integrate multiple knowledge libraries by configuring different search connectors, so that users can search among the information that they can access.

Once you have upgraded to Service Manager 9.80 from an applications version prior to 9.41, follow these steps to make sure the login.DEFAULT format control record has the expected code for the Smart Search feature:

1. Sign in to your Service Manager server as a System Administrator.
2. Click **Tailoring** > **Format Control**. Service Manager opens a blank Format Control Maintenance form.
3. Type login.DEFAULT in the Name field, and then click **Search**.
4. Click **JavaScript**.
5. Check whether the following codes exist:

```
var idolserver = new SCFile('idolserverinfo');
var rc = idolserver.doSelect('true');
if (rc === RC_SUCCESS) {
                vars['$lo.idol.enabled'] =
idolserver['enable'];
                vars['$lo.idol.img.enabled'] =
idolserver['image.enable'];
                if (vars['$lo.idol.enabled']) {
                        vars['$G.kmsearchengine'] = 'IDOL';
                 }
}
```

If these codes don't exist, you must type true in the Add field and then add these codes manually.

6. Click **Save** and **OK**.

Adopt Chat

Perform the manual steps described in the Deploy and configure the chat server task.

Update the support.groups fields in the device table

Once you have upgraded to Service Manager 9.80, follow these steps to update the support.groups fields in the device table:

1. Sign in to your Service Manager server as a System Administrator.
2. Click **Tailoring** > **Database Dictionary**.
3. Type device in the File Name field, and then click **Search**.
4. Click **support.groups** of the array type.
5. Update the value in the SQL Table field to a1, and then click **OK** to save your changes.

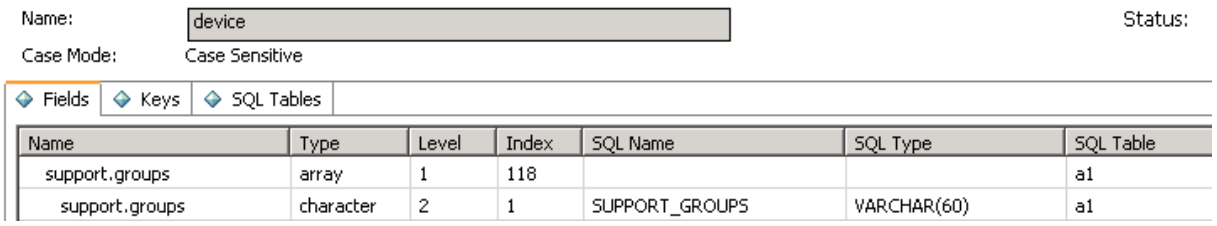


Note

If a1 is already in use, use other values instead. For example, a2 or a3.

6. Click **support.groups** of the character type.

7. Update the value in the SQL Table field to a1, update the value in the SQL Type field to VARCHAR(60), and then click **OK** to save your changes. See the following screenshot as an example:



Note

- VARCHAR is the SQL type of a SQL server. Use VARCHAR2 for an Oracle server.
- If a1 is already in use, use other values instead. For example, a2 or a3.
- If the system displays the following error message, update VARCHAR(60) to a bigger size such as VARCHAR(100).

  `Length (61 bytes) of data for field support.groups in SMSQLF3E5E54522D0397D0D08F6074 exceeds max (60 bytes), truncated (record.update,start)`

8. Click **OK** to save your changes and close the device table.

Caution

If the device table contains more than 100,000 records, updating the support.groups fields may take more than one hour.

Initialize CI descendant relationships

Follow these steps to initialize CI descendant relationships if you have upgraded to Service Manager 9.80 from an applications version prior to 9.64:

1. If you are upgrading from version 9.6x, go to the database and run **delete from CIRELATIONBSGM1** to remove all records from the table.
2. Create a ScriptLibrary and then add the following codes:

   ```
   lib.BSGRelationsCalculation1.run();
   ```

3. Click the **Execute** button to initialize the data of CI relationship to improve your system performance.

> **Note**
>
> The reference time to execute the codes is about 2 minutes per 10000 CI relationship records in the dbdict cirelationbsg for CI relationships. The actual execution time depends on your computer performance and the CI relationship complexity.
>
> It may take a very long time or even up to multiple days in graph generating for a huge CI graph (such as more than 100K cirelationship or device records). In order to lower the effect on the production system, you may consider restoring a backup of the production system as a test environment, running the script in the test environment, and then moving the cirelationbsg table contents back to the production system.

Update Smart Email Message ID for EWS

> **Note**
>
> Skip this task if you are upgrading from SM9.6x.

You need to complete the following steps only when you use Smart Email with the EWS protocol:

1. Create a ScriptLibrary and add the following codes:

   ```
   lib.SmartEmailUtil.convertSmartEmailEWSStore(true);
   ```

2. Click the **Execute** button to update the Smart Email Message IDs for the mails received within the previous 24 hours.

> **Note**
>
> If you want to update the Smart Email Message IDs for all the received mails, you can execute the following codes:
>
> ```
> lib.SmartEmailUtil.convertSmartEmailEWSStore();
> ```
>
> However, doing this may take a long time.

Translate customized inbox names

Service Manager upgrade attempts to enable localization for the customized inbox names. If you have customized inbox names, search the scmessage record for active languages other than English and then translate the content of Text for the language:

Language Code: <language code>

Class: local:inbox

Message Number: <inbox id>

Synchronize affected CIs in Problem Management

Follow these steps to synchronize affected CIs in Problem Management:

1. Create a ScriptLibrary and add the following codes:

   ```
   lib.problemManagement.synchronizeAllAffectedCiNames();
   ```

2. Click the **Execute** button to synchronize all the problem records.



Note

The actual execution may take some time depending on the amount of the Problem records.

Reimplement Feature Tracker

If you have installed the DevOps Support Content Pack to use Feature Tracker, you need to reimplement Feature Tracker. See Prepare Git for detailed instructions.

Additional manual migration tasks on Service Manager 9.80 Hybrid

After you have run the Service Manager 9.80 Applications Upgrade Utility to migrate your system to Service Manager Hybrid, you must complete a number of manual configuration tasks. Some tasks are optional, depending on how you want to configure your Service Manager Hybrid system. For detailed instructions, refer to Appendix A: Additional manual migration tasks on Service Manager 9.80 Hybrid.



Note

Perform these manual migration tasks in the development environment only.

1.3.5.11. Step 11: Migrate Process Designer data

Note: Skip this task if you are upgrading from Service Manager 9.52 or later versions.

You can use the data migration tool to migrate data from legacy process records to new process records within the same Service Manager system. It's not used to migrate data across different Service Manager systems. You can configure the migration tool from the GUI, and it requires administrator privileges.

The migration tool allows an operator to configure the field/value mapping between two files. It also covers customized fields. The out-of-box Process Designer provides several sample migration scripts that you can choose to use according to your migration scenario.



Caution

During data migration, there are no dependencies between the modules, however the scripts for each specific module must be executed in the listed order.

Module	Script
PD Framework	1. Legacy change model taskplanner data to new taskplanner 2. Legacy change instance taskplanner data to new taskplanner 3. Legacy change/task data to new taskplanner
Problem	1. Legacy problem to new problem 2. Legacy problem task to new problem task 3. Legacy known error to new known error 4. Legacy Process Designer known error to new Process Designer known error 5. Legacy known error related record migration 6. Legacy known error attachment to new known error attachment
Service Level Agreement	1. Legacy SLA to new Agreement
Service Catalog	1. Legacy Catalog Items to new Catalog Items
Configuration Item	1. Legacy configuration item to new configuration item

If further migrations are needed, an administrator can use these samples to define their own. Considering that some tables typically have a large data volume, the Data Migration tool supports an SQL batch update mode to ensure good performance.

Note

We recommend that you back up the production data before migration.

Run the scripts for specific modules as follows.

	PD Framework module					
Script	**9.3x non PD**	**9.3x PD3**	**9.3x PD4**	**9.40 Classic / 9.41 Classic**	**9.40 Codeless / 9.41 Hybrid / 9.41 Codeless**	**9.50 Codeless / 9.50 Hybrid / 9.51 Codeless / 9.51 Hybrid**
Legacy change model taskplanner data to new taskplanner		√	√			
Legacy change instance taskplanner data to new taskplanner		√	√			
Legacy change/task data to new taskplanner	√			√		

Problem module

Script	9.3x non PD	9.3x PD3	9.3x PD4	9.40 Classic / 9.41 Classic	9.40 Codeless / 9.41 Hybrid / 9.41 Codeless	9.50 Codeless / 9.50 Hybrid / 9.51 Codeless / 9.51 Hybrid
Legacy problem to new problem	√	√		√		
Legacy problem task to new problem task	√	√		√		
Legacy known error to new known error	√	√		√		
Legacy PD known error to new PD known error			√			

Script	9.3x non PD	9.3x PD3	9.3x PD4	9.40 Classic / 9.41 Classic	9.40 Codeless / 9.41 Hybrid / 9.41 Codeless	9.50 Codeless / 9.50 Hybrid / 9.51 Codeless / 9.51 Hybrid
Legacy known error related record migration	√	√		√		
Legacy known error attachment to new known error attachment	√	√		√		

Service Level Agreement, Service Catalog, and Configuration Item

Module	Script	9.3x non PD	9.3x PD3	9.3x PD4	9.40 Classic / 9.41 Classic	9.40 Codeless / 9.41 Hybrid / 9.41 Codeless	9.50 Codeless / 9.50 Hybrid / 9.51 Codeless / 9.51 Hybrid
Service Level Agreement	Legacy SLA to new Agreement	√	√	√			
Service Catalog	Legacy Catalog Items to new Catalog Items	√	√	√	√	√	
Configuration Item	Legacy configuration item to new configuration item	√	√	√	√	√	√

How to run a migration script

> **Note**
>
> If you are upgrading an existing Service Manager system to Process Designer, you need to customize the legacy out-of-box migration script samples for your specific environment.

To run a migration script, follow these steps:

1. Type db in the command line to open Database Manager.
2. Type **migrationSetting** in the **Table** field, and then click **Search**.
3. In the Migration Script setting detail form, click **Migrate Data** to run the migration script.
4. When using the Batch Update mode, the Data Migration pop-up window shows the RDBMS SQL Statements that will be run on the database. Check that the SQL statements are correct, and change these SQL statements if needed. Click **Next**, and then click **Finish** to start the migration in Batch Update mode.

   > **Note**
   >
   > If there is no data to migrate, or if the data is already migrated, you receive the following notification:
   >
   > "No records match the filter condition. At least one matching record is required."

1.3.5.12. Step 12: Return the system to normal operation

After the upgrade, the system may exhibit abnormal behavior until you return it to its normal operating environment.

Note

Remove all files related to upgrade from the system after successful completion of the upgrade. For information on removing these files, see Step 10: Perform additional manual tasks.

To return to a normal operating environment, follow these steps:

1. Sign out.
2. Stop the server.
3. Remove the comments from the `system.start` entry from the `sm.cfg` file.
4. Restore all the parameters that you updated in `sm.ini` and `sm.cfg` to their original state.
5. Add all parameters that are documented as necessary for your upgraded system.
6. Start the Service Manager server.
7. Sign in.
8. Wait for the background processes to start up.

Note

Service Manager doesn't recompile indexes in your RDBMS. If your RDBMS isn't configured to recompile indexes automatically after index changes, you must recompile your indexes manually.

1.3.5.13. Step 13: Test the development environment

After you resolve all conflicts, test the upgraded system and verify that the conflicts resolutions function properly. If there are problems that you can't resolve, contact Customer Support.

1.3.5.14. Step 14: Back up the system

Make a checkpoint backup of the data files to enable you to restore from this point, if necessary. Refer to the documentation for your RDBMS for backup instructions.

1.3.5.15. Step 15: Build a custom upgrade

If you have followed all the steps to this point, you have already run the upgrade against the duplicate system you created of your production system, and you have performed reconciliation for that system. This section describes how to build a custom upgrade, which is then applied to your test system and production system.

Recommendation

Before you begin to build the custom upgrade, ensure back up (see Step 14: Back up the system) or duplicate (see Step 1: Duplicate the production environment) the current upgraded environment.

To build the custom upgrade, follow these steps:

1. Sign in to the reconciled development system.
2. (Optional) You may need to exclude certain records on the upgraded system from being built to the custom upgrade package. To do so, follow these steps:

   a. Click **Tailoring** > **Differential Upgrade** > **Patch Records** to open the Patch Maintenance page.

   b. Select **UPGRADE** from the drop-down list, and then press Enter.

   c. You can update the query definition as required. If removing an entry, ensure that you clear the Dbdict field, uncheck the Add Only checkbox, and delete the corresponding query.

   d. Save your changes.
3. Create a folder for the custom upgrade on the Service Manager server. You can assign any name you like to this folder. However, for documentation purposes, this folder will be referred to as the *CustomUpgrade* folder. Ensure that the *CustomUpgrade* folder is empty.

   Note

   If you are connecting to the Service Manager server from a client that's installed on a remote client computer, make sure that the folder is created on the Service Manager server instead of the client computer.

4. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **Create Custom Upgrade**.
5. The Upgrade Utility automatically checks for duplicate records prefixed with "PRE", "NEW" or "OLD".
6. The system displays the patch file name and all the supported languages that are installed on your Service Manager system. Review this information, and then click **Next**.
7. Type a name for the custom upgrade package in the **The name of the release** field. For example, SM202210.
8. Type the fully-qualified path to the folder where the Upgrade Utility should create files in the **The path to export the upgrade files** field. This should be the path to the empty *CustomUpgrade* folder that you created in *step 3*. For example, **C:\OpenText\Service Manager\CustomUpgrade**.

   Note

   You must have read, write, and execute permission to the folder.

9. Click **Next**.
10. The system displays **Service Manager Upgrade Builder is now ready to build the upgrade**. Click **Next** to continue.
11. When you receive a "Finished creating the transfer files for the upgrade" message, the Upgrade Utility has finished the data packaging.

Note

If you are upgrading from SM9.6x, you only need to type the fully-qualified path to the folder to create files from *Step 6* to *Step 9*.

Upgrade Utility logs and error messages

When building the custom upgrade, the Upgrade Utility creates a set of log files, which reside in the log directory. The following table describes the contents of the log files during this step.

List of upgrade logs

Log file	Contents
detail.log	This file contains specific information about the upgrade, including the following: • All information in transfer.log and upgrade.log • Name of the file being purged. For example, "2022-09-20 13:58:32 dbdict: upgradestatus is purged." • Progress of signature creation. For example, "2022-09-20 14:47:36 Created 100 signatures for Object on version SM202308" • Building distribution information. For example, "2022-09-20 14:55:07 Building Distribution object for Applications Cluster Action.run"
transfer.log	This file contains information about the object being transferred by the upgrade. For example, “2022-09-20 15:12:27 Initiating an export of scmessage on query "((class="error" and message.id isin {"10"})) and syslanguage~="xxx"""

Log file	Contents
upgrade.log	This file contains information about where the upgrade is at any point, including the following: • Starting and ending of each sub-phase. For example, "2022-09-20 14:47:32 **** Start Phase [Pre Create Action Check] ****" • Main activities during each sub-phase. For example, "2022-09-20 14:47:35 Signaturing records." • Name of the file being exported. For example, "2022-09-20 15:11:37 Exporting: table = Object query = 'true'"

1.3.6. Upgrade tasks on the test environment

Step 1: Apply the custom upgrade to the test environment

You need to apply the newly-created custom upgrade to your test system for user acceptance testing.



Note

If you experience problems, such as a power failure or a network connection error while upgrading the system, you need to restore the database before attempting to run the upgrade again.

To upgrade the custom upgrade, follow these steps:

1. On the test system, which is a copy of your production system, complete all the preparation tasks in Upgrade tasks on the development environment.
2. Disable the **Client side load/unload** option from **Window** > **Preferences** > **Service Manager**.
3. Load the preupg.bin file using Service Manager Database Manager, and then restart the client.

   Instead of using the file that you extracted from the product installation package, you must use the preupg.bin file in your CustomUpgrade folder.

   Note

   The AppUpgVersion.txt file doesn't exist in the CustomUpgrade folder. Do not copy the file to this folder.

4. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **Apply Upgrade** to launch the Upgrade Utility.
5. In the text box, type the fully qualified path to the CustomUpgrade folder, and then click **Next**.

   Instead of using the file that you extracted from the product installation package, you must use your *CustomUpgrade* folder.

Example:

Windows: `c:\temp\upgrade\`

Linux: `~/upgrade/`

> **Note**
> - You must have read, write, and execute permission to the folder.
> - If you are upgrading from SM9.6x, the system displays the parameter settings before this step. Review the settings and then click **Next** to continue.

6. The loading process may take a long time. Wait until the file is loaded and the system displays the `Transfer files loaded` message.
7. The system displays a series of information for your verification, including Applications version upgrading from, Applications version upgrading to, Applications base version, Full path to the Upgrade Utility files and Language(s) to be upgraded. Verify this information and then click **Next** to continue.

> **Note**
>
> If this screen doesn't display the correct information, don't continue with the upgrade. Instead, contact Software Customer Support.

8. When you are asked whether you want to proceed, click **Yes**.
9. The Upgrade Utility displays the status when the upgrade is being processed.
10. When you receive an "UPGRADE IS COMPLETE" message, the Upgrade Utility has finished the data processing and you can follow the instructions in the message to complete the next steps. After you click **Finish**, you are automatically logged out.
11. Restart the server and log back into the client.
12. Open the **scversion** table in the Database Manager, and verify that the **Application Version** field is 9.72.00xx. If this field displays a value other than 9.72.00xx, check the log files to identify the issue that occurred.

Note how long it takes to apply the custom upgrade, so you will know how long the production system will be unavailable during the production upgrade. You can check the log file for an estimate.

Upgrade Utility logs and error messages

When applying the custom upgrade to the test system, the Upgrade Utility creates a set of log files, which reside in the log directory.

The contents of these log files are similar to those in the log files when running an out-of-box upgrade. See Upgrade Utility logs and error messages.

Tables and records that aren't upgraded by the Upgrade Utility

The Upgrade Utility doesn't automatically upgrade all tables and records. The patches record lists the tables and records that are packaged into the custom upgrade. Customizations made to any other tables or records will not be part of the custom upgrade. To make sure that the objects that you have reconciled are moved to the production system, verify the following scenarios:

- If an object is in the patches record, and the Result field of its related “upgraderesults" record displays "Kept Customer” or “Kept Customer Non-OOB”, change the Result field to “Reconciled”.

  For example, follow these steps after you have modified the cm.open.display_newphase displayoption record:

  1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **View/Merge Results**.
  2. Type displayoption in the Object Type field, type cm.open.display_newphase in the Object Name field, and then click **Search**.

     Note

     Some object names consist of multiple key fields, you can find the definitions in the signaturemake record.

  3. If the Result field in the search results is “Kept Customer” or “Kept Customer Non-OOB”, manually change the Result field to “Reconciled”
- If an object is in the patches record, and the Result field of its related “upgraderesults" record is not "Kept Customer” or “Kept Customer Non-OOB”, no additional task is needed.
- If an object is not in the patches record, do one of the following:
  - Create an unload file containing those objects by adding them to an unload script or using the standard Service Manager Unload/Export Facility,

- Make the same changes manually by directly modifying the objects on the production system. For records that you might have deleted, you can either build a purge script for those records or delete the records manually on the production system.

Resolve Process Designer related data

To resolve the Process Designer related data, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **View/Merge Results**.
2. Click the **Expert Search** button from the More Actions menu, type version.reconciled#"PD Enable" in the **Query** field, and then click **Search**.
3. In the search result, resolve the data by following the actions described in the table below.

Reconciled Version	Result	Action
PD Enable: Need to be removed	Any	Remove the objects manually.

Step 2: Perform additional manual tasks

This section lists changes that can't be automated by the Upgrade Utility and changes that are required only for certain customers. Make these changes before testing and backing up your system. For detailed instructions, refer to Step 10: Perform additional manual tasks.

Step 3: Migrate Process Designer data

See Step11: Migrate Process Designer data for more information.

Step 4: Test the test environment

After you apply the custom upgrade on the test environment, perform user acceptance testing and load testing for all features, especially customized applications. Test the upgraded system with the new Service Manager client to verify any changes you have made in reconciliation. If the upgrade process has any problems, you need to contact Customer Support. After you complete testing of the upgraded system, you can use it to upgrade your production system.

To test the test environment, follow these steps:

1. Return the system to a normal operating environment.
2. Install and configure the Service Manager client for the target version (see instructions in *Service Manager Installation*).
3. Use the new Service Manager client to log on.
4. Review the new features.
5. Use the new Service Manager client to thoroughly test the upgraded system. Test all features that your users will access. Pay particular attention to areas that were modified on your system.

1.3.7. Upgrade tasks on the production environment

Step 1: Apply the custom upgrade to the production environment

After you test the custom upgrade on the test system, you need to apply the newly-created custom upgrade to your production system. This process is identical to the one you followed when applying your upgrade to your test system.

> **Note**
>
> Do not apply an upgrade to your production system if it has not been thoroughly tested. If issues were found in the custom upgrade, we recommend that you fix the issues in the development system and re-create the customer upgrade.

To apply the custom upgrade to the production system, follow these steps:

1. Make sure the production system is not available to users while you are applying the custom upgrade.
    1. Have all users sign out of the server.
    2. Prevent users from signing into your Service Manager server by running the **sm -quiesce:1** quiesce command from the operating system's command prompt.
2. Ensure the upgrade files you created are accessible to the production system (the files are located on the same server).
3. If you transfer the files to your production system by FTP, set FTP to binary mode.
4. Complete all the preparation tasks in Upgrade tasks on the development environment.
5. Apply the customer upgrade to the production system.

    > **Note**
    >
    > If you experience problems, such as a power failure or a network connection error while upgrading the system, you need to restore the database before attempting to run the upgrade again.

6. Sign out from your Service Manager server, and then sign in again.
7. Allow users to sign in to the server by running the **sm -quiesce:0** quiesce command from the operating system's command prompt.

Step 2: Perform additional manual tasks

This section lists changes that can't be automated by the Upgrade Utility and changes that are required only for certain customers. Make these changes before testing and backing up your system. For detailed instructions, refer to Step 10: Perform additional manual tasks.

Step 3: Migrate Process Designer data

See Step11: Migrate Process Designer data for more information.

Step 4: Test the production environment

Test the upgraded environment and verify that it functions properly. If there are problems that you can't resolve, contact Software Support.

1.3.8. Appendix A: Additional manual migration tasks on Service Manager Hybrid

After you have run the SM 9.80 Applications Upgrade Utility to migrate your system to Service Manager Hybrid, you must complete a number of manual configuration tasks. Some tasks are optional, depending on how you want to configure your Service Manager Hybrid system.

Mandatory manual migration tasks

Optional manual migration tasks

1.3.8.1. Mandatory manual migration tasks

Note: Some manual migration tasks are only mandatory under certain conditions. See each task for further information.

Review the migrated validity records

After you migrate to Service Manager Hybrid, validity records are validated against the Process Designer category tables instead of the Classic category tables. Therefore, a manual review of the validity records is required.

A Process Designer migration report that resembles the following is included in the pdmigration.log file:

The following validity records are updated to perform validation against Process Designer-based category tables (such as category, subcategory, and producttype) instead of legacy category tables. Backups of the original validity records (identified by the suffix "_disabled") have been created. Unique ID:BP;Field Name:category;Sequence:1 Unique ID:BP;Field Name:category;Sequence:3 Unique ID:BP;Field Name:incident.category;Sequence:3 Unique ID:BP;Field Name:product.type;Sequence:1 Unique ID:BP;Field Name:product.type;Sequence:2 Unique ID:BP;Field Name:product.type;Sequence:3 Unique ID:BP;Field Name:product.type;Sequence:5 Unique ID:BP;Field Name:product.type;Sequence:7 Unique ID:BP;Field Name:product.type;Sequence:10 Unique ID:BP;Field Name:product.type;Sequence:13 Unique ID:BP;Field Name:subcategory;Sequence:1 Unique ID:BP;Field Name:subcategory;Sequence:2 Unique ID:BP;Field Name:subcategory;Sequence:3 Unique ID:BP;Field Name:subcategory;Sequence:5 Unique ID:BP;Field Name:subcategory;Sequence:11 Unique ID:BP;Field Name:subcategory;Sequence:12

The number of updated validity records depends on your system configuration. If you have not customized any validity records, you can ignore this step.



Note

If you have added dbdict fields to the category tables and if you use the validity table to validate against these fields, you must manually change the validation logic for these fields.

Review the category-based alerts migration

Stage 1, 2, and 3 alerts and deadline alerts

The Applications Upgrade Utility migrates the Classic Stage 1, Stage 2, Stage 3, and Deadline alerts and Deadline Alert Group configurations from Classic categories to the workflow phases of the automatically-generated Incident workflow.

The migrated alerts are named according to their source. For example, "Incident Alert Stage 1 for complaint" indicates that the alert is converted from stage 1 configuration of the complaint category. The following types of alert are created:

- **Incident alert stage 1 for <***category name***>**. This alert updates the incident alert status to "alert stage 1" if the category has a stage 1 interval or expression specified.
- **Incident alert stage 2 for <***category name***>**. This alert updates the incident alert status to "alert stage 2."
- **Incident alert stage 3 for <***category name***>**. This alert updates incident alert status to "alert stage 3."
- **Incident Deadline for <***category name***>**. This alert updates the incident alert status to "DEADLINE ALERT."

The migrated alerts are located in the alerts tab of each workflow phase.

Gap and limitations

The migrated alerts implement the main functionality of the Classic alerts; however, there are some gaps and limitations.

- In Classic category-based alert expressions, "$file" represents the incident. The Applications Upgrade Utility automatically modifies "$file" to "$L.file" for compatibility with the migrated alerts. If you use other variables in your Classic system, you must manually modify the expression in the migrated alerts.
- The timing for the generation of stage 2 and 3 alerts is different. In the Classic Incident module, the schedule for stage 2 is generated after the schedule for stage 1 is executed and the incident alert status is updated to stage 1. Likewise, the schedule for stage 3 is generated after the schedule for stage 2 is executed and the incident alert status is updated to stage 2. In Hybrid mode, however, all schedule records for stage 1, stage 2, stage 3, and deadline alerts are generated or re-generated at the same time, when the incident record is updated.

  Therefore, if you are using an expression that uses the content of the current record ($file) as either the condition or the calculation for the alert time in stage 2, stage 3, or deadline alerts, issues may occur because the content of the current record can differ when the schedule is generated.

- The migrated alerts are moved to the alerts list of the "In Progress" phase of the migrated Incident workflow. If you add new phases, you need to also add these alerts to new phases so that the alerts can work for incident in the new phases.
- The alert reset and recalculation conditions in objects are changed in Hybrid mode. Therefore your original alerts may not work. For example, the out-of-box alert reset condition in the Classic probsummary object is `category in $L.file="incident" and not (null(1 in external.process.reference in $L.file))`. If you use this condition, alerts only work for incidents that are assigned to the incident category. In the migrated probsummary object, the condition is `evaluate(nullsub(parse(str(alertsReset in $L.wfPhase), 2), false)) or not (same(current.phase in $L.file, current.phase in $L.file.save))`, and the alert reset condition of the "In Progress" phase is set to "true." Therefore, all alerts configured in the phase will work for all incidents rather than only for incidents that are assigned to the incident category.

  In this case, you can modify the schedule condition and the alert condition in the existing alert definition of the Incident Management module by appending the original reset condition that's configured in the Classic object.

Reassignment threshold

The reassignment interval and expression and the reassignment count threshold in the Classic categories aren't migrated automatically.

If you still need reassignment threshold functionality, you can add the relevant fields to the new Process Designer-based category file and form, and then copy the configuration to the new Process Designer categories. The following table lists the fields that you will need to add.

Field name in dbdict	Data type	Label in form/description
count	number	Reassign Count Threshold
reassign	date/time	Reassignment Interval
reass.expression	expression	Reassignment Expression

Review Change Management processes

Note

The following step applies only to customers who upgrade to Service Manager Hybrid from a system that doesn't have PDCP3 installed.

Renamed processes

The following two Classic processes in DocEngine are renamed by the Applications Upgrade Utility. The old Classic names are now used for Codeless processes.

Process name in Classic mode	Process name in Hybrid mode
cm.open	cm.open.classic
cm.open.save	cm.open.save.classic

Therefore, if you have made any customizations to the processes, a conflict with the new name will be reported in the upgrade results (for example, if you customized "cm.open", this process will be renamed to "cm.open.classic" and will conflict with the out-of-box "cm.open.classic".) Additionally, if you have made any customizations that call the Classic processes, you must update these to call the new process names.

Review "Close Phase"

In the migrated Change Management workflow, the "Close Phase" functionality is moved to transitions between phases. In Classic versions of Service Manager, the "close" display option closes intermediate phases and closes the change or change task in the last phase. In the migrated Change Management workflow, the "close" display option closes only changes or change tasks in the last phase in the workflow.

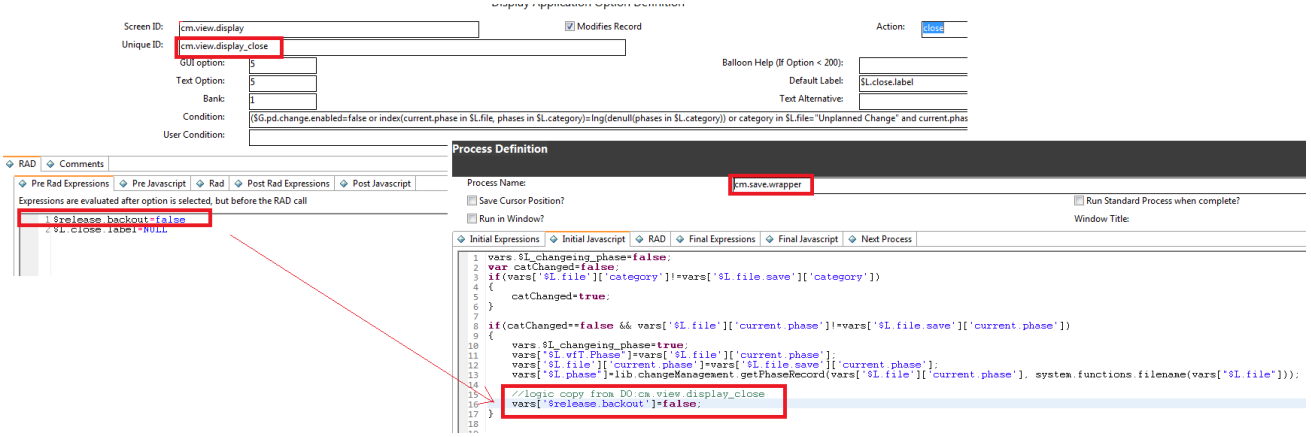
The following condition is added to the original condition of the "cm.view.display_close" display option:

```
($G.pd.change.enabled=false or index(current.phase in $L.file, phases in $L.category)=lng(denull(phases in $L.category)) or category in $L.file="Unplanned Change" and current.phase in $L.file="Discovery Assessment") and
```

If you have tailored this display option, or if you have configured other display options to perform the "close" action, you must make equivalent modifications to ensure that the close action only works for last phase.

In Hybrid versions of Service Manager, display options aren't called during the "close" phase. Therefore, the "cm.view.display_close" display option code is migrated

to the "cm.save.wrapper" process, as illustrated by the following image. If you have tailored this display option, you must make equivalent changes to the process.



In out-of-box Hybrid versions of Service Manager, the transition between phases is named "Close Phase". However, you can modify the label to suit your needs by using the localization functionality in the Workflow Editor.

Modify $L.env to $G .cm3r.environment/$G.cm3t.environment

In the Classic Change module, $L.env is a "cm3profile" record, whereas in Hybrid Change it is a "tableAccess" record. Therefore, references to $L.env are updated to refer to $G.cm3r.environment/$G.cm3t.environment, in case a "cm3profile" record is required (the $G.cm3r.environment/$G.cm3t.environment global environment variable contains the same content as the Classic version of the $L.env record).

In out-of-box Hybrid versions of Service Manager, all the related display screens, display options, and processes in the Change Management module are updated. For example, in "cm.view.display_reopen", `evaluate(reopen in $L.env)` is updated to `(filename($L.file)="cm3r" and evaluate(reopen in $G.cm3r.environment) or filename($L.file)="cm3t" and evaluate(reopen in $G.cm3t.environment))`.

However, you must make equivalent updates for any tailoring you have made that uses $L.env as a "cm3rprofile" record.

Revert to status-based SLM for Change Tasks

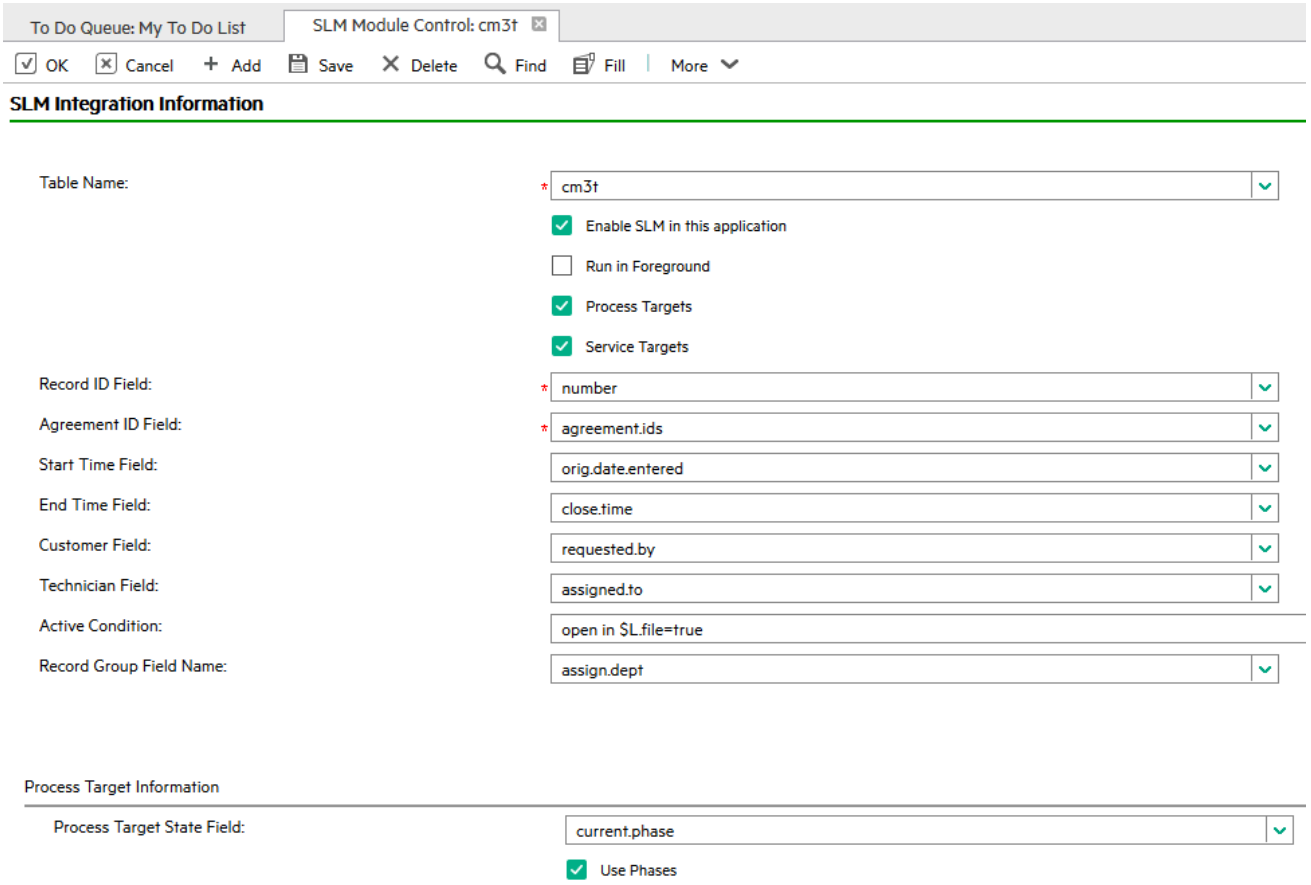
Note

The following step applies only to customers who upgrade to Service Manager Hybrid from a system that doesn't have PDCP3 installed.

Service Level Management (SLM) for Change Tasks in Codeless versions of Service Manager is based on phases, whereas in Classic versions it is based on statuses. Therefore, you must revert to status-based SLM for Change Tasks in order for your existing SLAs to work with the migrated workflows.

To do this, follow these steps:

1. In the System Navigator, click **Service Level Management** > **Administration** > **Configure Application**.
2. In the **Table Name** drop-down list, select **cm3t**.
3. In the Process Target Information section, click to clear the **Use Phases** option, and then configure the **Process Target State Progression** and **Standard Alerts** fields as required.



Review the "form edit" condition in workflow phases

The "form edit" condition in Process Designer-based workflows determines whether a form is editable when a user views a record. In Service Manager Classic, this control condition may be located different places. When the Applications Upgrade

Utility creates the migrated workflows for Hybrid mode, the condition settings are migrated from the default location of the condition in Service Manager Classic:

- Service Desk: The condition is copied from the state input condition
- Change: The condition is copied from the legacy change phase

If you have configured the condition in any other location, it will not be automatically migrated. In this situation, you must manually combine the Classic condition settings into the "form edit" condition.



Note

Whilst we recommend that you use the "form edit" condition in workflow phases in Service Manager Hybrid and Codeless, the "IO control" condition for the display screen still works. If you set both conditions, the form is editable only when both condition are met.

Merge the status lists

The Applications Upgrade Utility implements the Process Designer search form and the Process Designer status list. Therefore, you must merge the Classic status list into the Process Designer-based status list.

The following table describes the table in each module that you must update, along with information about the Globallist, the Globalvariables, and where the data comes from.

We recommend that you modify the data in the ModuleStatus database according to your needs. For example, you must update the ModuleStatus list for the Incident module to add or remove statuses according to your requirements.

Module	Global list name	Process Designer statuses global list variable	Details
Incident	Incident Local Statuses	$G.imStatuses	Retrieve from file ModuleStatus with Limiting SQL module="probsummary"
Service Desk	Interaction Local Statuses	$G.sdStatuses	Retrieve from file ModuleStatus with Limiting SQL module="incidents"
Problem	Problem Local Statuses	$G.pbmStatuses	Retrieve from file ModuleStatus with Limiting SQL module="rootcause"
Problem	Problem Task Local Statuses	$G.pbmTaskStatuses	Retrieve from file ModuleStatus with Limiting SQL module="rootcausetask"

Note

Both the Process Designer-based and Classic Change modules use the same status list.

Update the eventin service

If you use the eventin service to create Incidents, and if the RAD applications of your Event Register for creating the Incidents is set to "axces.apm," you must check whether the "add" action in the document engine's state records action blocks externally-created Incidents.

To do this, first identify which process maps to the "add" action in the Classic Open State. The following illustration uses the im.first process as an example from an out-of-box system.

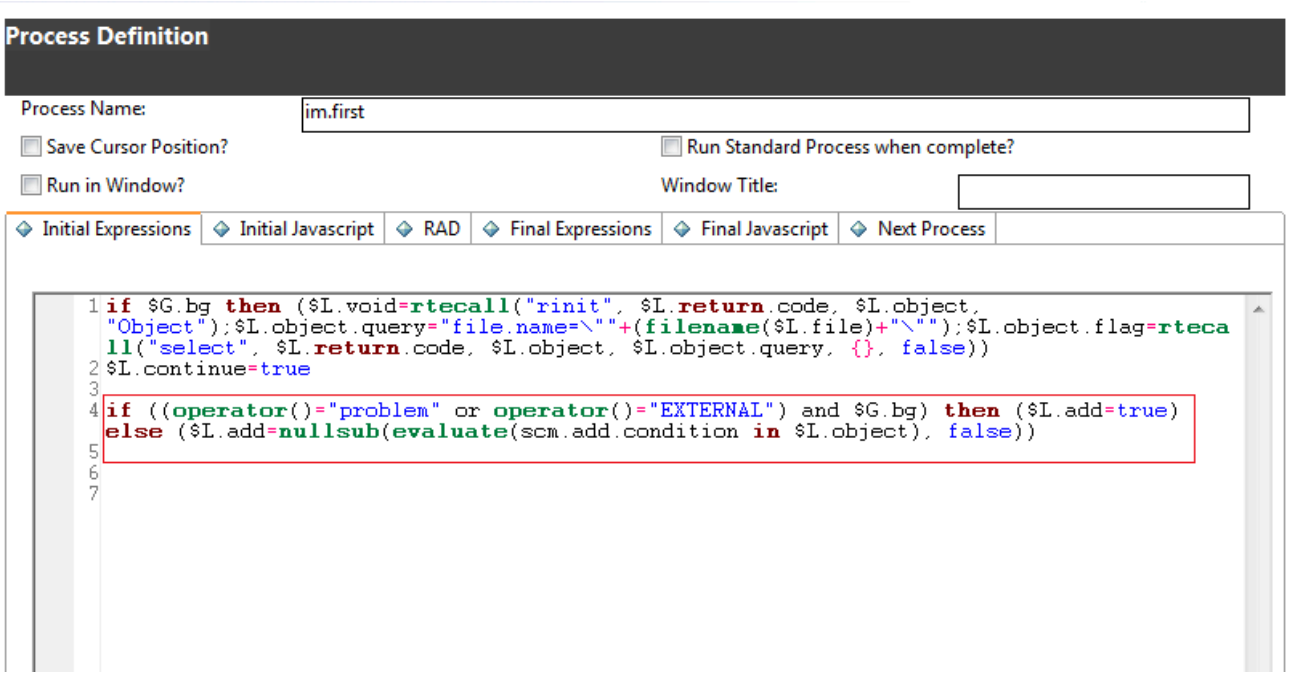
State Definition

State: im.open
Display Screen: apm.first
Initialization Process: im.open.setup
Format: nullsub(format.name in $G.pm.environment, "IM.open.incident")
Input Condition (view state only): true

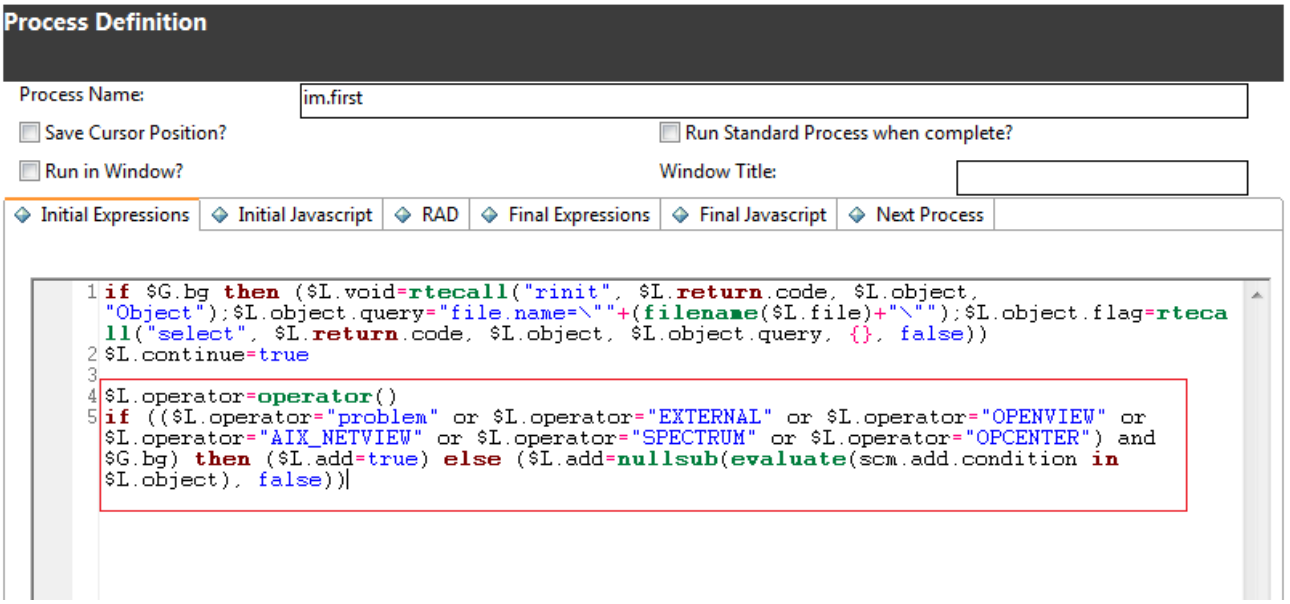
Non-base methods

Display Action	Process Name	Condition	Save First
save	im.first.save	true	
resolve	im.first	true	
new	im.first.new	true	
closeme	im.first	true	
cause	im.lookup.cause	true	
getsla	im.get.sla	true	
find.solution	im.find.solution	true	
getans.search	getans.search.solution	true	
getans.retrieve	getans.retrieve.solution	true	
getans.open	getans.open	true	
getans.create	getans.create.solution	true	
retrieve.iknow	iKnow.getsolution	true	
add	im.first	true	
kmsearch	kmmappedsearch	true	
view.biz.services	view.biz.services	true	
fill	context.pre.fill	false	
runcontext	run.context.wizard	true	
saveandexit	im.first.exit	true	
menu	im.open.cancel	true	

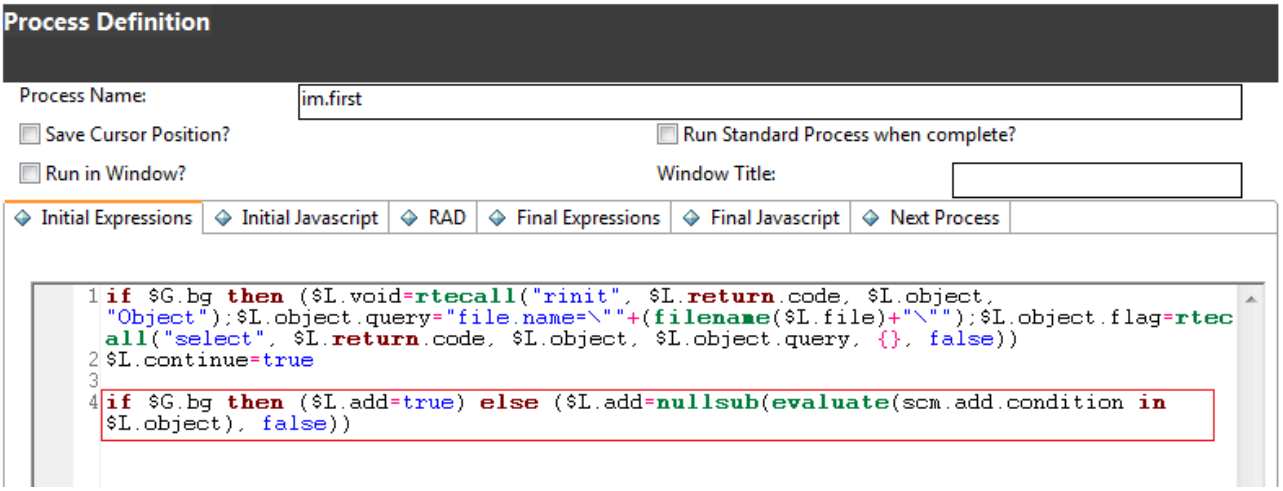
In out-of-box systems earlier than Service Manager 9.41, only the "problem" and "EXTERNAL" operators have permission to add Incident records from the backend.



Make sure that the User IDs of your eventin services are configured as demonstrated in the following image.



Alternatively, make sure the User IDs of your eventin services aren't blocked. To do this, you can change your code to allow all backend operations, as illustrated by the following image.



If you use the following applications in your event register, the relevant eventin services are now allowed to trigger the Process Designer rule sets.

RAD applications	File	Out-of-box event registers
axces.apm	probsummary	pmu: problem update pmr: problem reopen pmo: problem open pmm: problem mibile checkout/in pmc: problem close ... (total 19 eventregisters)
axces.apm.epmosmu	probsummary	epmosmu: e problem open smu
axces.sm	incidents	esmin: e service management smin: service management
axces.cm3	cm3r cm3t	cm3rin cm3tin ... (total 14 eventregisters)

Note

- The Change module eventin services (such as eventregister cm3rin and cm3tin) can already trigger Process Designer rule sets.
- If you use the axces.database RAD applications (the behavior of which is not changed by the Process Designer migration) as the eventregister application, the eventin services will not trigger the Process Designer rule sets.

Replace the environmental variables

In Service Manager Classic, the `$L.env` variable stores the profile record. However, in Service Manager Codeless, the `$L.env` variable stores the tableAccess record. Therefore, all code in the Change Management module that uses the `$L.env` variable (mostly, this relates to Display Options) must be updated to use the `$G.cm3r.environment` or `$G.cm3t.environment` variables.

For example, the display condition of the **cm.view.display_approve** display option must be updated as follows:

Original:

```
evaluate(approvals in $L.env)
```

After update:

```
evaluate(approvals in $G.cm3r.environment) or evaluate(approvals in $G.cm3t.environment)
```

1.3.8.2. Optional manual migration tasks

Configure the Change Management module to use the new Task Planner



Note

The following step applies only to customers who upgrade to Service Manager Hybrid from a 9.3x system that has PDCP3 or PDCP4 installed.

If you are upgrading to Service Manager 9.80 Hybrid from a Classic version of Service Manager, you must add the Task Planner form to your Change format, and then run the "Legacy change/task data to new taskplanner" data migration script, if you want to use the new Task Planner.

An improved version of the Task Planner was introduced in Service Manager 9.40. To use it, make sure that the following code elements are successfully implemented (this disables the previous version of the Task Planner). To do this, you can use conflict resolution or manually change the code elements according to following table.

Record type	Record name	Modifications	Comments
Object	changeModel	Uses the "Change Model" workflow, the workflow location is "In Object," and the file-level rule sets are configured to use the new Task Planner.	

Record type	Record name	Modifications	Comments
Object	cm3t	The following object-level rule sets are configured to use the new Task Planner: • After successful add: `.common.taskplan.noneplantasktochangeplan` • On enter: `.common.taskplan.validateoutputontask` • After successful enter: `.common.taskplan.sync.task.titleandstatus.tochangeplan` `.common.taskplan.updatedependenttaskstatus` • Initialization:	

Record type	Record name	Modifications	Comments

		`.common.taskplan.taskticket.init`
		• On display:
		`.common.workflow.init.vars`
		• On update:
		`.common.taskplan.savecontextconfig`
		• After successful update:
		`.common.taskplan.sync.task.titleandstatus.tochangeplan`	

Process	changeModel.buildtable	The following initial JavaScript is commented: `lib.changeModel.saveFromXML(true);`	This record was used by optionid 441 to save taskxml files to the changeModel table for the Classic Task Editor.

Record type	Record name	Modifications	Comments
Process	changeModel.init	The following initial JavaScript is commented: `lib.changeModel.createXML(vars.$L_file);`	This record was used to initialize a taskxml file to be displayed by the Classic Task Editor.
Process	changeModel.save	The following initial JavaScript is commented: `lib.changeModel.saveFromXML();`	This record was used to save taskxml files to the changeModel table.

Record type	Record name	Modifications	Comments
Process	change.open.save	• RAD expressions are evaluated before the change.openPlanTasks RAD call. • The following expression is removed: `if ($L.exit="added" and filename($L.file)="cm3r") then ($L.changePlan=jscall("changePlan.createPlan", $L.file, changeModel in $L.file);$L.void=rtecall("copycurrent", $L.errcode, $L.phase.save, $L.phase))` • Disabled calling the change.ope	

Record type	Record name	Modifications	Comments
		nPlanTasks RAD.	

Record type	Record name	Modifications	Comments
Process	change.update.save	• RAD expressions are evaluated before the apm.mb.ok RAD call. • The following expressions are removed: ◦ if (filename($L.file)="cm3r") then ($L.changePlan=jscall("changePlan.getPlan", $L.file)) ◦ $L.task.list = "";if (filename($L.file) = "cm3r" and not	

Record type	Record name	Modifications	Comments
		(same(current .phase in $L.file, $L.orig .phase))) then ($L.tas k.list = jscall(" change Plan.ge tNeedT oClose TaskLis t", $L.orig .phase, $L.cha ngePla n, nullsub (allow. open.ta sks in $L.pha se, false), workflo wName in $L.wfP hase, current .phase in	

Record type	Record name	Modifications	Comments
		$L.file)) ◦ if ($L.task.list~="") then ($L.continue = false;$L.exit= "bad.val"; $L.opentask.msg = scmsg(550, "cm3", {$L.task.list})) ◦ if ($L.continue=false and not (null($L.opentask.msg)) and $G.bg) then ($L.void=rtec	

Record type	Record name	Modifications	Comments
		all("msg", $L.rc, $L.opentask.msg)) • Disabled calling the apm.mb.okRAD • RAD expressions are evaluated before the change.openPlanTasks RAD call. • The following expression is removed: if (filename($L.file)="cm3r") then ($L.success.flg=rtecall("refresh", $L.errcode, $L.changePlan)) • Disabled calling the change.ope	

Record type	Record name	Modifications	Comments
		nPlanTasks RAD	

Record type	Record name	Modifications	Comments
Process	changePlan.buildtable	• RAD expressions are evaluated before the change.openPlanTasks RAD call: • The following expression is removed: ◦ if (filename($L.file)="cm3r") then ($L.changePlan = jscall("changePlan.getPlan", $L.file)) • Post RAD expressions: the following expressions are removed: • ◦ $L.void=jscal	This record is used by optionid 441 to save taskxml files into the changeModel table for the legacy Task Editor.

Record type	Record name	Modifications	Comments
		l("changePlan.createXML", $L.file) ◦ $L.exit="refresh" • Disabled calling the change.openPlanTasks RAD • The following initial JavaScript is commented: lib.changePlan.saveFromXML(true); 	
ScriptLibrary	ApplyChangeModel	The following script is commented: addTasksToChangePlan(change, changeModel);	

Additionally, you must check that the following code elements are correctly implemented before you use the new Task Planner. To do this, you can use conflict resolution or manually change the code elements according to following table.

Record type	Record name	Modifications	Comments
format	chm.cm3r.tasks.horz	The old Task Editor widget is replaced with the new Task Planner widget.	These are the out-of-box formats. Customize your own formats accordingly if you're not using out-of-box formats.

Record type	Record name	Modifications	Comments
format	chm.task chm.standard.registration chm.standard.pln.sch chm.standard.execution chm.standard.pir chm.standard.backout chm.normal.registration chm.normal.rsk.imp chm.normal.tcab.approval chm.normal.build.test chm.normal.dcab.approval chm.normal.deployment chm.normal.backout chm.normal.pir chm.emergency.registration chm.emergency.rsk.imp	A new Task tab is added to dynamically display context-related record information that's planned during the task plan.	These are the out-of-box formats. Customize your own formats accordingly if you're not using out-of-box formats.

Record type	Record name	Modifications	Comments
	chm.emergency.approval chm.emergency.build.test chm.emergency.implmt chm.emergency.backout chm.emergency.pir		
Triggers	trigger.name="trigger.changeModel.remove.taskplan" and table.name="changeModel"	A new trigger is added to remove task plan-related information once a changeModel record is deleted.	

Update your customized formats

If you don't plan to use the out-of-box ChangeModel or Task formats, you must configure the Task Planner widget in your tailored ChangeModel and Task formats as described in the following table.

Task Planner property	Value in ChangeModel format	Value in Task format
Editor type	Model	Ticket
Category	Category	Header/category
ID	id	Header/number
Editable	Yes	Yes
Parent file	cm3r	cm3r

Additionally, you must manually add the "Additional Properties" tab to the Change and Task details forms, in order to display the Additional Properties information for Changes and Tasks. To do this, manually add references to the following subformats in the relevant formats:

- Change format

  `common.taskplan.parent.InputOutput.subform`

- Task format

  `common.taskplan.task.InputOutput.subform`

Data migration

For more information about how to migrate your data, refer to Appendix B: Data migration.

Upgrade to the Process Designer related record subform

The functionality of related records in Process Designer-based systems and in Classic systems is different. However, Classic related record functionality still works in Process Designer-based environments, and it is therefore not mandatory for you to replace this functionality.

Caution

If you decide not to use the Process Designer-based related record functionality, rule types such as "Run Action" will not work.

To use the Process Designer-based related record functionality, simply update the "Related Records tab" in the corresponding forms to implement the Process Designer-based related record subform (for example, `im.incident.subform.related` for the Incident module). Then, migrate the related record data.

Use the Process Designer workflow viewer in your Change Management forms

When you view a record in a Process Designer-based module, the form uses the new Workflow Viewer widget to display the workflow history of the record. The Applications Upgrade Utility updates your forms during the upgrade process to enable this feature.

However, if you previously customized Service Manager so that the cm3r.sub.workflow.g subform was not used to hold the Classic workflow widget in the Change Management module, you must manually implement the new Workflow Viewer widget in your Change Management forms.

To do this, follow these steps:

1. In Forms Designer, open the "chm.cm3r.sub.workflow.g" out-of-box Process Designer workflow subform. Then, use design mode to copy the content of this subform.
2. In Forms Designer, open the form to which you want to add the new Workflow Viewer widget. Then, overwrite the content for the existing Classic workflow widget subform with the content that you copied from the "chm.cm3r.sub.workflow.g" subform.

Configure the Search form

Search forms in Process Designer-based systems and in Classic systems are different. For example, the Process Designer-based Incident module uses the "im.advFind.incident.search" search form, instead of the "advFind.incident.search" search form. If you had previously customized some fields in a search form, you must manually add these fields to the Process Designer-based version of the search form.

We recommend that you use the Process Designer search form. However, if you want to continue to use the Classic search form, you must update the SearchConfig record to revert the changes applied by the Applications Upgrade Utility. To do this, set the value of the **Search Format** field to the appropriate Classic search form for that table.

The following table lists the mapping between Process Designer-based search forms and their Classic equivalents.

Table name	Classic search form	Process Designer search form
probsummary	advFind.incident.search	im.advFind.incident.search
cm3r	advFind.search.change	advFind.search.chm
cm3t	advFind.search.task	advFind.search.task
incidents	advFind.SD.search	sd.advFind.search
rootcause	advFind.search.problem	pbm.advFind.search.problem
rootcausetask	advFind.search.problem.task	pbm.advFind.search.problem.task

Migrate the solution matching configuration from security profiles to security settings

The Incident solution matching functionality in Process Designer-based systems and in Classic systems is different.

We recommend that you use Process Designer solution matching. However, if you want to use Classic solution matching, you must manually migrate the security configuration from Classic security profiles to Process Designer security roles. This is necessary because Classic solution matching is configured in security profiles (for example, pmenv), which aren't used in Process Designer security roles.

The following example uses the "Check similar problems" Incident solution matching setting to demonstrate how to perform the migration.

1. The field name of the "Check similar problems" setting is "check.rc." To add a setting with the same name in the "Incident" security area, search for the "Incident" security area. From the **More** menu, click **Administration** > **Add New Setting**, and then add a new setting.
2. Set the security rights value for this new Setting.
3. Check all your Incident Management security profile records (in an out-of-box system, there are 49) to identify all the records in which "Check similar problems" is selected. For example, if "Check similar problems" is selected in

the sysadmin profile, you need to set the area of the sysadmin security role to "Incident," and select the "Check similar problems" setting.

4. After you manually migrate the "Check similar problems" setting, perform the same tasks for the following settings:
    - Check similar incidents
    - Check incident duplicates on configuration item
    - Check incident duplicates on related configuration item
        - Max levels
        - Max hits

Update reports to use Process Designer-based category tables

If you plan to use Process Designer-based Incident, Service Desk, and Problem category tables in your reports, you must update your reports to use the new tables.

The following table describes the mapping between category tables in Process Designer-based and Classic systems.

Incident module

Classic table	Process Designer-based table
Category	imCategory
subcategory	imSubcategory
producttype	imArea

Service Desk module

Classic table	Process Designer-based table
Category	sdCategory
subcategory	sdSubcategory
producttype	sdArea

Problem module

Classic table	Process Designer-based table
Category	pbmCategory
Subcategory	pbmSubcategory
producttype	pbmArea

Make the Category field in the Interaction and Incident forms read-only

In Classic systems, the category field in the Interaction and Incident forms aren't read-only; users can change the category of an interaction. However, in Process Designer-based systems, this field is used to determine the specific workflow that the interaction or incident follows. If the category changes, the interaction may need to use a new workflow and forms, but this will not happen if the field is modified directly in the form. Therefore, we recommend that you make the Category field read-only.

To correctly change the category of an interaction or incident, use the new **Change Category** menu option that's automatically created by the Applications Upgrade Utility. This ensures that the correct workflow is used.

Revert to using the "escalate to Change" wizard

The escalation to change process in Process Designer-based systems and in Classic systems is different. After you upgrade to Service Manager 9.80 Hybrid, the escalation wizard no longer appears when you click **Escalate** in an Interaction, and the Change is not created in the background. Instead, the Change form is launched for you to input the details.

To revert to using the Classic escalation wizard, follow these steps:

1. Open the "escalate.interaction" process, and modify the condition so that it evaluates to "true" for escalation to change. This ensures that the escalation wizard is displayed.

   For example, modify the condition from `$L.use.legacy.sd.solution.matching=true and ($L.string="create" and $exit.code.fc="normal")` to `($L.use.legacy.sd.solution.matching=true or category in $L.file="request for change") and ($L.string="create" and $exit.code.fc="normal")`.

2. Open the "cc.createchange" process, and modify the expression that's evaluated before the RAD call from `$escalation.wizard.call=false` to `$escalation.wizard.call=true`. This ensures that the change is created in the background.

3. If it doesn't already exist, add the following expression to the "number" link line of the "screlate.incidents.cm3r" link:

   `requestedDate in $L.related=nullsub($requested.end.date, tod())`

   Note

   You can further tailor "escalate to Change" wizard by adding additional expressions or field mapping to copy data to change records.

Remove the button in the Incident form that cancels the Interaction escalation process

The Interaction escalation process in Process Designer-based systems and in Classic systems is different. In Process Designer-based systems, the interaction is added to database before the Process Designer solution matching form is displayed, and the Incident form is displayed when you select to create a new Incident (there is no longer an Escalate button on the Interaction form). When you save the Incident, the Incident is created and associated with the Interaction. If you don't want to escalate the Interaction to an Incident, you can click **Cancel** in the Incident form. In Classic systems, the Incident is created in the background without displaying the Incident form to the end user.

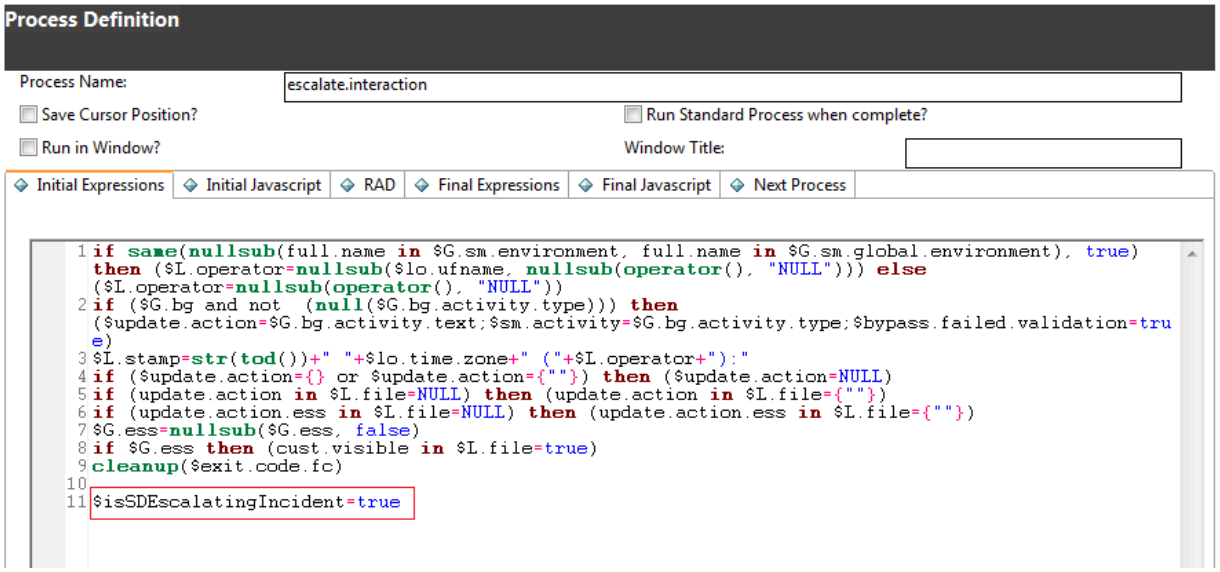
To revert to the old behavior, you must add the **Escalate** button back to the Interaction form and remove the **Cancel** button from the Incident form. To do this, follow these steps:

1. To add the **Escalate** button back to the Interaction form, update the user condition in the "cc.edit.incident_escalate" display option to the following:

   `evaluate($L.tableAccess.view) and (nullsub($G.ess, false)=false and ess.entry in $L.filed and nullsub($isnewweb, false)=true or jscall("sdUtil.isInteractionEscalatedOrFulfilled", $L.file)=false) and nullsub(category in $L.filed, "unknown")~="service catalog" and open in $L.file~="Closed"`

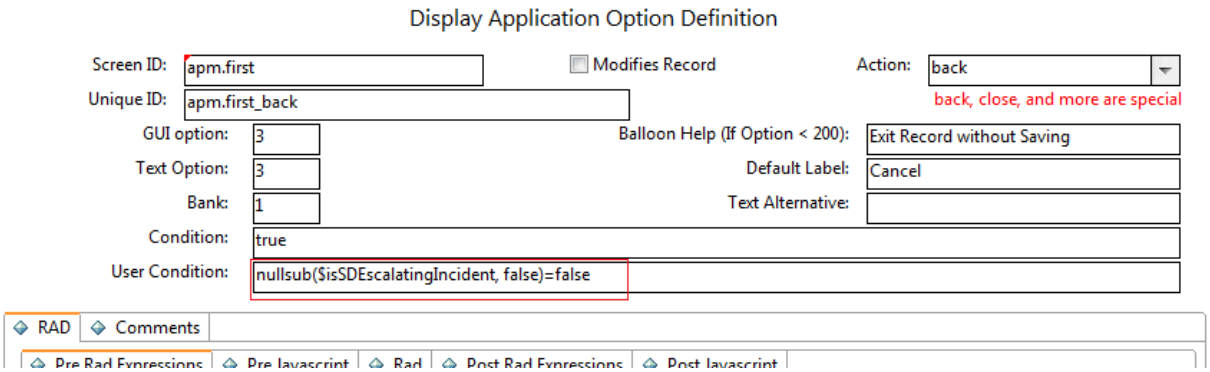
2. Remove the **Cancel** button from the Incident form. To do this, set a flag variable in the "escalate.interaction" process. For example, set the following flag:

   `$isSDEscalatingIncident=true`



   Then, check the following flag in the condition of the **Cancel** button ("apm.first_back" display option):

   `nullsub($isSDEscalatingIncident, false)=false`



Migrate any customization of the knownerror table

The Process Designer-based Problem module stores Known Error records and Problem records in the same table (rootcause), whereas the Classic Problem module stores Known Error records in the knownerror table. Therefore, if you have made any customizations to the Classic knownerror table that you want to retain, you must manually reapply these customizations to the rootcause table.

Remove the Classic Request Management dashboard and report definitions

After you upgrade to Service Manager Hybrid, the Classic Request Management dashboard and report definitions remain in the system. If you don't want to run the Classic Request Management and Process Designer-based Request Fulfillment modules in parallel, you can remove the Classic Request Management dashboard and report definitions.

To remove the Classic Request Management dashboard definition record, follow these steps:

1. In the System Navigator, click **Reporting** > **Search Dashboard**, and then search for ID "10000468."
2. Click **Delete**, and then click **Yes** to remove the Dashboard record.

To remove the Classic Request Management report definition records, follow these steps:

1. In the System Navigator, click **Reporting** > **Search Report**, and then search for ID "10000460."
2. Click **Delete**, and then click **Yes** to remove the Report record.
3. Repeat steps 1 and 2 for the following IDs:
    - 10000461
    - 10000462
    - 10000463
    - 10000466
    - 10000467

Enable Classic Request Management and Process Designer-based Request Fulfillment to run in parallel

Migration from Classic Request Management to Process Designer-based Request Fullfillment is a manual process that can be time-consuming if you have heavily tailored the Request Management module. In order to ease this transition Service Manager Hybrid enables you to run the Classic Request Management and the Process Designer-based Request Fulfillment modules in parallel. To do this, follow these steps.

Step 1: Set the value of the global flag to "true"

In the ProcessDesignerEnablement Script Library, locate the **isNonPDRequestInParallel** function, un-comment the line `$G.nonpd.request.in.parallel variable`, and then set its value to "True."

Step 2: Restore the non-Process Designer Request menu items

Request module menu items

To restore these menu items, search for menu records that have the parameter value "RM PD." In out-of-box systems, there are three such records. Add the Classic "Request Management" menu item back to the menu.

Request environment menu items

These menu items are located at **System Administration** > **Ongoing Maintenance** > **Environment Records**. To restore these menu items, search for the "ENV RECORDS" menu, and then add the Classic request menu items.

Request profile menu items

These menu items are located at **System Administration** > **Ongoing Maintenance** > **Profiles**. To restore these menu items, search for the "MODULE PROFILES" menu, and then add the Classic request menu items.

Models menu item

This menu item is located at **System Administration** > **Base System Configuration**. To restore this menu item, search for the "SYSTEM ADMINISTRATION" menu, and then add the Classic Request menu items.

Step 3: Update the queries in Inbox views

1. Search all Approval inboxes andcheck the query condition in each record to determine whether it uses only Classic global variables (`$G.ocmq.environment` , `$G.ocmo.environment` , and `$G.ocml.environment`) or Process Designer-based global variables (`$G.request.environment` and `$G.requestTask.environment`). If so, modify the queries to use both sets of global variables.

   For example, this query in the "All my Approvals" inbox contains a condition that resembles the following:

   `(file.name="request" and (current.pending.groups isin approval.groups in $G.request.environment or current.pending.groups isin {$lo.user.name})`

   In this situation, you need to change the query to the following:

   `(file.name="request" and (current.pending.groups isin approval.groups in $G.request.environment or current.pending.groups isin {$lo.user.name}) or`

`(file.name="ocmq" and (current.pending.groups isin approval.groups in $G.ocmq.environment or current.pending.groups isin {$lo.user.name})`

2. In the "My Group's To Do List" Inbox view, restore `itemType="ocmq" or itemType="ocml"` to the query, and make sure that the "request" and "requestTask" item types are included in the query.
3. In the "My Pending Delegated Approvals" Inbox view, append the following string to the query:

   `or (file.name="ocmq" and current.pending.groups isin $G.delegated.ocmq.groups`

   In the "My Pending Approvals" Inbox view, append the following string to the query:

   `or (file.name="ocmq" and (current.pending.groups isin approval.groups in $G.ocmq.environment or current.pending.groups isin {$lo.user.name})`

Step 4: Restore the "OCM Create Order" schedule record

To restore the "OCM Create Order" schedule record, you must unload the record from your system before you upgrade to Service Manager Hybrid, and then load it to your system after the upgrade process is complete.

You also can unload this record from a previous version of Service Manager.

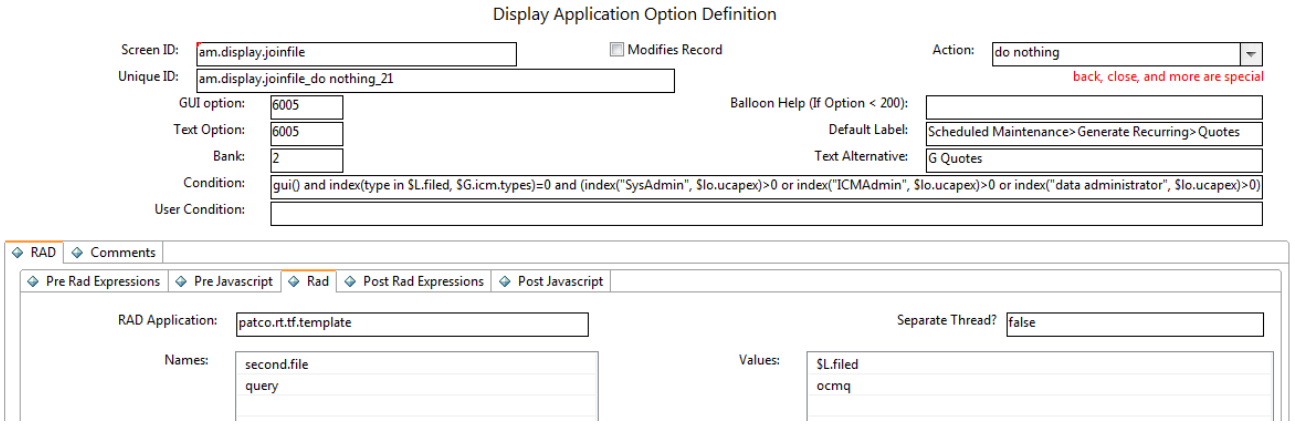
Step 5: Configure search

Search for SearchConfig records that have a table name that begins with "ocm," and then remove the "false" condition in the Allow Advanced Find query.

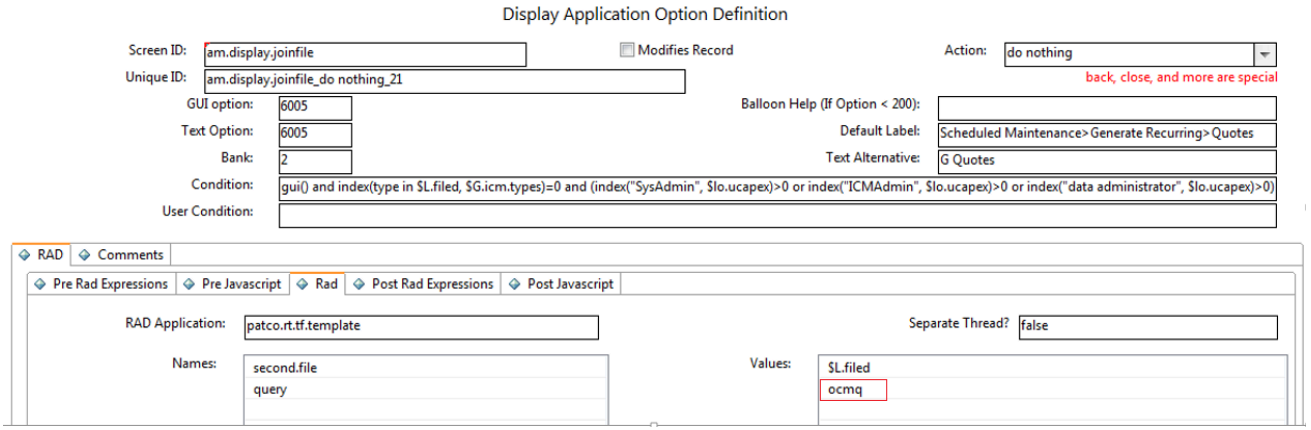
Step 6: Add display options

These display options are located at **More** > **Scheduled Maintenance** > **Generate Recurring** > Request in CI record display forms.

The sub-menu in the following illustration refers to Request table records.



Create a copy of the display option for this sub-menu, and then modify it to refer to the OCMQ table records, as demonstrated in the following image.



When you have done this, two sub-menus are displayed in the CI form. One sub-menu refers to the Request table records, and the other sub-form refers to the OCMQ table records.

Step 7: Rename the connector

Rename the Process Designer catalog connector. For example, name the connector "Open New PD Request."

If you customized the "Open New Request" catalog connector, the out-of-box Process Designer catalog connector is named "NEW980Open New PD Request" after the migration process.

If you did not customize the "Open New Request" catalog connector, the out-of-box Process Designer catalog connector is named "Open New Request" after the migration process. In this case, you need to rename the catalog connector to "NEW980Open New PD Request," and then restore the non-Process Designer catalog connector.

Step 8: Modify the connector wizard

1. In the "svcCat Select Connector" wizard, click **File Selection** > **JavaScript**, and then change the connector name for the bundle catalog in the connector list from "Open New Request" to "Open New PD Request" (or to the name that you used in step 6).
2. Click the **Next Wizard** tab, and then change the connector name for Process Designer Request from "Open New Request" to "Open New PD Request" (or to the name that you used in step 6).
3. Restore the Next wizard for Request Management.

Step 9: Restore the Classic Request profile setting to the Operator form

Update the JavaScript of the "operator.view" and "operator.search" display screens as follows.

Before update

```
if(lib.ProcessDesignerEnablement.isRequestEnabled())
{
vars['$L.showRMProfile']=false;
}
```

After update

```
if(lib.ProcessDesignerEnablement.isRequestEnabled() &&
vars['$G.nonpd.request.in.parallel']!==true)
{
vars['$L.showRMProfile']=false;
}
```

Step 10: Use the model table for Classic Request Management

After you run the Applications Upgrade Utility, Process Designer-based Request Fulfilment uses the new productCatalog table instead of the model table. Additionally, the tool updates all references to the model table to refer to the productCatalog table.

In order for the Classic Request Management to work in parallel with Process Designer-based Request Fulfilment, the two modules must use the two tables

separately.

To achieve this, first restore the links that reference the model table.

Links that must be updated

Link name	Field
"ocmlrec"	part.no
"ocml.view.summary"	part.no
"ocml.view.space"	model
"ocml.view.space"	part.no
"ocml.view.sap"	part.no
"ocml.view.detail"	parts.part.no
"ocml.view.detail"	part.no
"ocml.view.default.workorder"	parts.part.no
"ocml.view.default.workorder"	model
"ocml.view.default.workorder"	part.no
"ocml.view.default.telecom"	parts.part.no
"ocml.view.default.telecom"	part.no
"ocml.view.default.office.move"	parts.part.no
"ocml.view.default.office.move"	part.no
"ocml.view.default.internal"	parts.part.no
"ocml.view.default.internal"	part.no
"ocml.view.default"	parts.part.no
"ocml.view.default"	model
"ocml.view.default"	part.no
"ocml.search"	part.no

Link name	Field
"ocml.copy.model"	number
"ocml"	part.no
"ocmco"	part.no
"ocmco"	parent.parts
"model"	component.part.no
"model"	parent.parts
"model"	model
"advFind.search.line.item"	part.no

Links to update (optional)

Link name	Field
"Survey.CFG.addFilter"	part.no
"stock"	part.no
"software.counter"	installation.models
"software.counter"	license.models
"problem"	failing.component
"problem"	model
"pc.software.files"	part.no
"modelvendor.lkup"	part.no
"modelvendor"	part.no
"model.icm"	model
"IM.summary"	model
"IM.master.update"	model
"IM.master.open"	model
"IM.master.link"	model
"IM.master.close"	model
"furnishings"	model
"device2"	model
"device.template"	bios.model
"device.template"	part.no
"device.telecom"	part.no

Link name	Field
"device.storage"	part.no
"device.softwarelicense"	part.no
"device.officeelectronics"	print.model
"device.officeelectronics"	part.no
"device.networkcomponents"	part.no
"device.mainframe"	part.no
"device.handhelds"	part.no
"device.furnishings"	part.no
"device.example"	bios.model
"device.example"	part.no
"device.displaydevice"	part.no
"device.display"	part.no
"device.computer"	bios.model
"device.computer"	part.no
"device"	part.no
"DEFAULT ICM"	model
"dec.device.pc"	part.no
"contract.template"	part.no
"contract.template"	model
"contract"	part.no

Link name	Field
"contract"	model
"configurationItemNode"	bios.model
"configurationItemNode"	part.no
"configurationItem"	bios.model
"configurationItem"	part.no
"cm3t"	part.no
"cm3t"	product.no.
"cm3r"	part.no
"cm3r"	product.no.
"advFind.search.contract"	part.no
"advFind.search.contract"	model
"advFind.search.ci"	part.no

Next, configure synchronization between the "model" and "productCatalog" tables. Classic Request Management and Process Designer-based Request Fullfillment use the two tables separately, but you must ensure that the data in the two tables is consistent.

Step 11: Modify the KM knowledge base

After you run the Applications Upgrade Utility, the "Request_Library" KM knowledge base is updated to use the request table. You must add another library that uses the OCMQ table, so that the KM knowledge base can support both Classic and Process Designer-based request.

For example, you might create the following knowledgebase:

- Knowledgebase Name: Quote_Library
- Display Name: Quote
- Type: sclib

- Search Server Name: <*your search server name*>

Then, in the **Field Definitions** tab, configure the following field definitions.

Field name	Alias	Type	Hitlist	Index weight	Match	Sort
number	id	String	true	Level 4	false	
status	problem status;kmstatus	String	true	No Index	true	
current.phase	phase	String	true	No Index	true	
brief.description	title	String	true	Level 4	false	
description	description	String	false	Level 3	false	
requested.for	requestedfor	String	true	Default	false	
requestor.name	requestorname	String	true	Default	false	
requested.date	requesteddate	Date	true	No Index	false	
company	company	String	true	No Index	false	
assigned.dept	assignment	String	true	Default	true	
assigned.to	assignee	String	true	Default	false	
priority	priority	String	true	No Index	true	
justification	justification	String	false	Default	false	

Field name	Alias	Type	Hitlist	Index weight	Match	Sort
sysmodtime	sysmodtime	Date	true	No Index	false	

Step 12: Ensure the related record functionality works correctly

For more information about how to update your forms to use the new related record functionality, refer to *Upgrade to the Process Designer related record subform* above.

If you want your system to allow the creation of related records between the Classic Request Management and Process Designer-based Request Fullfillment modules, you must configure the screlationtype table appropriately. For example, if you want to allow the creation of related quotes from Incident records, you must add a new record in the screlationtype table.

For more information about how to perform data migration for screlation records, refer to Related Records data migration.

After you have done this, you can create related quotes from incident records.

Step 13: Restart the server

After you have configured Request Management and Request Fulfillment to run in parallel, you must restart the Service Manager server.

1.3.9. Appendix B: Data migration

This appendix describes the following topics that are related to the data migration tool.

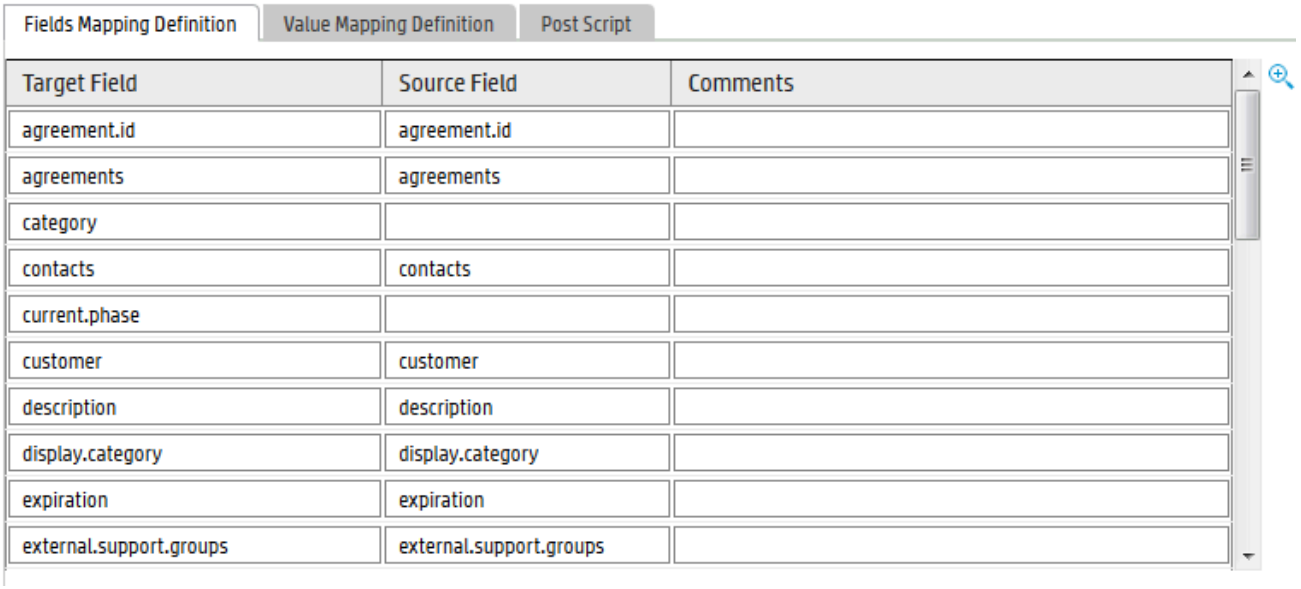
Field mapping and value mapping

Batch Update mode

Out-of-box data migration settings

1.3.9.1. Field mapping and value mapping

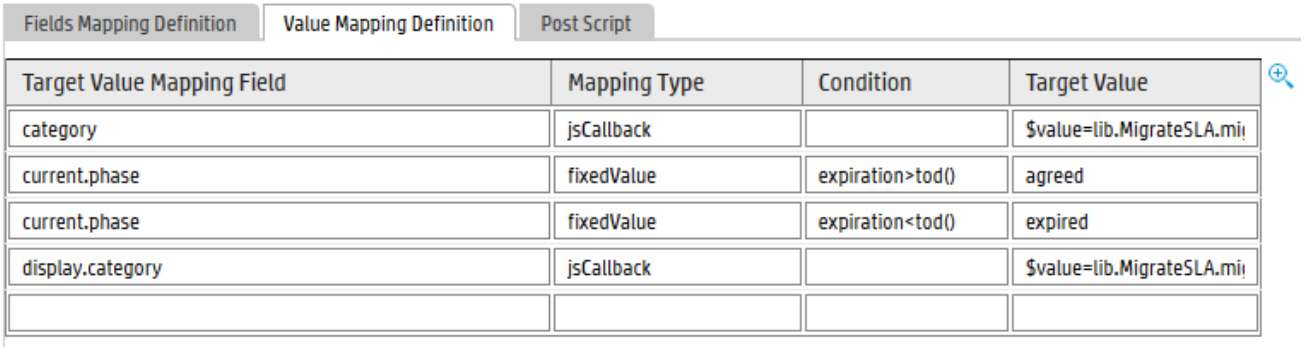
Field mapping defines the mapping between the fields in the source file and target file, as shown in the following figure.



Note

- The Target field can't be empty.
- The Source and Target fields must already exist in the table.
- If there is a structure field mapping and you're not in Batch Update mode, both fields must have the exact same structure definition.
- Array of structure and structure fields aren't supported when in Batch Update mode.
- The M2 and alias table aren't supported in Batch Update mode.
- You can't use jscallback in Batch Update mode.

Value mapping defines the value mapping for each mapped field.



Note

- For the fields that don't have a value mapping defined, the default is direct value copy.
- You can define multiple value mappings for the same field.

The following table describes each column in the Value Mapping Definition tab.

Column	Description
Target value mapping field	The field from target field that needs to define value mapping.

Column	Description
Mapping Type	The following three types are supported: 1. **fixedValue:** target field value = the fixed value defined in “Target Value” column. The supported fixed value are number, string, datetime, and Boolean. You can't specify an expression in a fixed table, nor a fixed value for structure and array field type. 2. **sourceField:** target field value = source field value defined in “Target Value” column. 3. **jsCallback:** target field value = a JavaScript script to dynamically set the value. Three keywords ($sourceTable, $targetable, $value) can be used if the direct script code is written in the Target Value column: ◦ $sourceTable, $targetTable reference the field value in the source table field and the target table field, respectively as shown in the following example: `var status = $sourceTable[“rcStatus”],$targeTable[“rcStatus”] = “open”` ◦ $value is used to specify the current target field value as

Column	Description

	shown in the following example:
	`$value=vars.$G_user_role retrieves the value from another variable.`
	jsCallback can also support directly calling a JavaScript function defined in Script library, such as the following:
	`lib.UpgradeScript.migrateData($sourceTable,.$targetTable);`

Condition	If the condition is true, then the value mapping is applied during the migration. The condition expression supports the following: 1. Arithmetic operators: >, <, >=, <=, =, ~= 2. Logical operators: or, and 3. The “null” keyword can be used as a null value in the formula.
Target Value	Depending on the Mapping Type setting, Target Value can be a fixed value string, or a source field name or a JavaScript script.

1.3.9.2. Batch Update mode

When the target file equals the source file, the migration tool executes the update operation on the same table. In addition, an additional SQL batch update mode is automatically enabled to allow the migration tool to directly update the underlying database.

Note

You can run the migration in Batch Update mode to get better performance when the source and target files are the same. The Batch Update mode can't be used when you migrate data between two different files, such as from Legacy known error to Problem.

Note

Some of the out-of-box migration scripts can't be run in Batch Update mode. For more information, see Appendix B: Data migration.

Batch update mode can only be enabled when the target file equals the source file. If it is enabled, a batch SQL update is generated in a popup wizard instead of using the Service Manager API to update the data records one by one. After confirmation, the batch SQL update is sent to the underlying database.

If Batch Update mode is enabled, you should note the following:

- You should update the data records directly via the database without triggering Service Manager. Otherwise, updating the data records will trigger additional background process.
- The Filter Condition field should follow the RDBMS SQL grammar for your database. Otherwise, Filter Condition should follow query statement grammar of Service Manager.
- The “Condition” field in the Value Mapping Definition should follow the RDBMS SQL grammar for your database. Otherwise, Filter Condition should follow query statement grammar of Service Manager.


T30 - Activate Patch

1.3.9.3. Out-of-box data migration settings

The Data Migration Tool provides the following out-of-box data migration settings.

Problem data migration

Considering that Problem doesn't need to copy data from two different tables, and that it typically has a large volume of data, we recommend that you run the migration by using the batch update mode (especially for migrating a large volume of closed tickets).

Note

After you migrate the Problem data, remove the “Problem_Library_disabled_by_PDHD” knowledge base as this legacy library is no longer used.

The out-of-box example data migration is based on the following three key fields:

- status
- category
- current.phase

Note

You can add additional fields if needed, but don't remove these three fields.

Problem

Default settings:

Source Table	rootcause
Target Table	rootcause
Filter Condition	category="BPPM"

Field Mapping:

Target Field	Source Field
category	category
current.phase	current.phase
rcStatus	rcStatus

Value Mapping:

Target Value Mapping Field	Mapping Type	Condition	Target Value
category	fixedValue	category="BPPM"	problem
current.phase	fixedValue	(current.phase="Problem Detection, Logging and Categorization" or current.phase="Problem Prioritization and Planning") and rcStatus~="Closed"	Categorization
current.phase	fixedValue	current.phase="Problem Investigation and Diagnosis" and rcStatus~="Closed"	Investigation
current.phase	fixedValue	current.phase="Problem Resolution" and rcStatus~="Closed"	Resolution
current.phase	fixedValue	current.phase="Problem Closure and Review" and rcStatus~="Closed"	Review

Target Value Mapping Field	Mapping Type	Condition	Target Value
current.phase	fixedValue	rcStatus="Closed"	Closure
rcStatus	fixedValue	(current.phase="Problem Detection, Logging and Categorization" or current.phase="Problem Prioritization and Planning") and rcStatus~="Closed" and rcStatus~="Deferred"	Categorize
rcStatus	fixedValue	current.phase="Problem Resolution" and (rcStatus="Pending Vendor" or rcStatus="Pending User")	Pending
rcStatus	fixedValue	(current.phase="Problem Investigation and Diagnosis" or current.phase="Problem Resolution") and rcStatus~="Closed" and rcStatus~="Deferred"	Work In Progress

Target Value Mapping Field	Mapping Type	Condition	Target Value
rcStatus	fixedValue	current.phase~="Problem Closure and Review" and rcStatus="Deferred"	Deferred
rcStatus	fixedValue	current.phase="Problem Closure and Review" and rcStatus~="Closed"	Resolved
rcStatus	fixedValue	rcStatus="Closed"	Closed

Problem Task

Default Settings:

Source Table	rootcausetask
Target Table	rootcausetask
Filter Condition	task.category="Default"

Field Mapping:

Target Field	Source Field
task.category	task.category
current.phase	current.phase
rcStatus	rcStatus

Value Mapping:

Target Value Mapping Field	Mapping Type	Condition	Target Value
task.category	fixedValue		Investigation
current.phase	fixedValue	rcStatus~="Closed"	Active
current.phase	fixedValue	rcStatus="Closed"	Closure
rcStatus	fixedValue	rcStatus="Work In Progress"	Work In Progress
rcStatus	fixedValue	rcStatus="Pending Vendor" or rcStatus="Pending User"	Pending
rcStatus	fixedValue	rcStatus="Closed"	Closed
rcStatus	fixedValue	rcStatus~="Work In Progress" and rcStatus~="Pending Vendor" and rcStatus~="Pending User" and rcStatus~="Closed"	Assigned

Task Planner data migration

In versions of Service Manager earlier than 9.40, Task Planner data is stored in the changeModel and changePlan tables. However, in Service Manager 9.40, Task Planner data is stored in the changePlan table only. Therefore, two data migration settings are provided to migrate Task Planner data from Service Manager 9.40 and earlier versions.

Migrate legacy change model Task Planner data to new Task Planner

Default settings:

Source table	changeModel
Target Table	changePlan
Query	not null(tasks)
Batch Update	false

Field mapping:

Target field	**Source field**
fileName	
number	id

Value mapping:

Target value mapping field	**Mapping type**	**Condition**	**Target value**
fileName	fixedValue		changeModel

Post script:

```
for(var i=0;i<$sourceTable.tasks.length();i++){

$targetTable.tasks[i].taskId=$sourceTable.tasks[i].taskId;

$targetTable.tasks[i].taskCoords=$sourceTable.tasks[i].taskCoord
s;

$targetTable.tasks[i].dependentIds=$sourceTable.tasks[i].depende
ntIds;

$targetTable.tasks[i].dependentCoords=$sourceTable.tasks[i].depe
ndentCoords;

$targetTable.tasks[i].taskCategory=$sourceTable.tasks[i].taskCat
egory;

$targetTable.tasks[i].taskTemplate=$sourceTable.tasks[i].taskTem
plate;

$targetTable.tasks[i].taskDescription=$sourceTable.tasks[i].task
Description;

$targetTable.tasks[i].openInPhase=$sourceTable.tasks[i].openInPh
ase;

$targetTable.tasks[i].closeByPhase=$sourceTable.tasks[i].closeBy
Phase;

$targetTable.tasks[i].activeCond=$sourceTable.tasks[i].activeCon
d;

$targetTable.tasks[i].activeCondXML=$sourceTable.tasks[i].active
CondXML;

$targetTable.tasks[i].activeCondDesc=$sourceTable.tasks[i].activ
eCondDesc;

$targetTable.tasks[i].mandatory=$sourceTable.tasks[i].mandatory;
```

```
}
$targetTable.doUpdate();
```

Migrate legacy change instance Task Planner data to new Task Planner

Default settings:

Source table	changePlan
Target Table	changePlan
Query	null(fileName)
Batch Update	true

Field mapping:

Target field	**Source field**
fileName	fileName

Value mapping:

Target value mapping field	**Mapping type**	**Condition**	**Target value**
fileName	fixedValue		cm3r

Migrate legacy change/task data to new Task Planner

In Service Manager Hybrid, task information is stored in the change's task plan. Change task information must be migrated to the new Task Planner. To do this, use the "Legacy change/task data to new taskplanner" data migration setting in the data migration tool.

Default settings:

Source table	cm3r
Target Table	changePlan
Query	open=true
Batch Update	

Field mapping:

Target field	Source field
fileName	
Number	header,number

Value mapping:

Target value mapping field	Mapping type	Condition	Target value
fileName			

Known Error data migration

Known Error data migration differs from Interaction, Incident, and Problem because Known Error data must be copied from the knownerror file to the rootcause file, and the field mapping is also more extensive than in other modules.

Because it is a data copy, you can't use the batch update mode. However, knownerror data volume is usually much less than that of Incident or Interaction. Therefore, the normal copy mode should be sufficient for most environments.



Note

After you migrate the knownerror data, remove the “KnownError_Library_disabled_by_PDHD” knowledge base as this legacy library is no longer used.

The Data Migration Tool provides four migration settings, which you should run in their corresponding scenario as described in the following table.

Scenario	Migration setting(s)
Migrating from legacy Known Error	Legacy known error to new known error Legacy known error related record migration Legacy known error attachment to new known error attachment
Migrating from legacy Process Designer Known Error	Legacy Process Designer known error to new Process Designer known error For more information about how to use this migration setting, see the *Migrating from legacy Process Designer known error records* section.

Additionally, because knownerror data is copied and the records are re-created in the rootcause file, you must also update the screlation file to correct the relations of known errors. To do this, the out-of-box Process Designer provides another migration script called “Legacy known error related record migration”.



Note

Attachments for legacy knownerror records are migrated as attachments for the new known error records.

Default Settings:

Source Table	knownerror
Target Table	rootcause
Filter Condition	true

Field Mappings:

Target Field(Problem)	Source Field(Known Error)
id	id
category	category
assignment	assignment
status	status
logical.name	logical.name
brief.description	brief.description
description	description
root.cause	root.cause
update	update
open	open
open.time	open.time
opened.by	opened.by
update.time	update.time
updated.by	updated.by
close.time	close.time
closed.by	closed.by
reopen.time	reopen.time
reopened.by	reopened.by
priority.code	priority.code
ticket.owner	ticket.owner

Target Field(Problem)	Source Field(Known Error)
severity	severity
sysmodtime	sysmodtime
sysmodcount	sysmodcount
sysmoduser	sysmoduser
assignee.name	assignee.name
subcategory	subcategory
product.type	product.type
problem.type	problem.type
company	company
dump	dump
resolution	resolution
workaround	workaround
incident.category	incident.category
current.phase	current.phase
incident.count	incident.count
initial.impact	initial.impact
future.impact	future.impact
impact	impact
users.affected	users.affected
problem.start.time	problem.start.time

Target Field(Problem)	Source Field(Known Error)
expected.resolution.time	expected.resolution.time
location.type	location.type
frequency	frequency
proposed.solution	proposed.solution
review.notes	review.notes
cause.code	cause.code
affected.ci	matching.ci
ci.device.name	matching.device.name
ci.device.type	matching.device.type
ci.assign.group	matching.assign.group
ci.location	matching.location
affected.companies	affected.companies
affected.ci.count	matching.ci.count
last.task.no	last.task.no
kpf.id	kpf.id
kpf.file	kpf.file
folder	folder
closure.code	closure.code
estimatedCost	estimatedCost
estimatedMandays	estimatedMandays

Target Field(Problem)	Source Field(Known Error)
rootcauseDate	knownerrorDate
solutionDate	solutionDate
affected.item	affected.item
rcStatus	rcStatus
interaction.count	interaction.count
publishWorkaround	publishWorkaround

Value Mappings:

Target Value Mapping Field	Mapping Type	Condition	Target Value
category	fixedValue	true	known error
current.phase	fixedValue	rcStatus~="Closed"	Logging
current.phase	fixedValue	rcStatus="Closed"	Closure
isKnownError	fixedValue	true	true
rcStatus	fixedValue	rcStatus~="Closed"	Open
rcStatus	fixedValue	rcStatus="Closed"	Closed

Migrating from legacy Process Designer known error records

If you upgraded your system from an older Process Designer version, you need to migrate your old Process Designer knownerror records to new Process Designer knownerror records by using the **Legacy PD known error to new PD known error** migration setting. The migration process includes the following procedures:

1. Change the old Known Error record to Problem record, and then create a new Known Error record with a category of "known error."
2. Link the new Known Error record to the Problem record.
3. Copy or don't copy the original relations to the new Known Error record, based on the option you select in the **Post Script** tab of this migration setting, as described in the following table.

Option for relation processing	Description
Skip Copying Relations to New Known Error Records	This option doesn't copy original relations to new known error records.
Copy Relations to New Known Error Records	This option copies original relations to new known error records.

Service Level Management data migration

As of Service Manager 9.40, the Service Level Management (SLM) module is reimplemented on Process Designer (PD). If you upgraded from an earlier version of Service Manager, you need to migrate your legacy SLM data so that your SLM module can work correctly on Process Designer-based workflows.

The purpose of SLM data migration is to set a category and set a phase for agreement records based on the following rules:

- The Category of an agreement record is populated with the category of one of the targets if all the targets of the agreement have the same category (that is, the **Service Level Category** field in the target record).
- The Category of an agreement record is populated with a value of **Service Level Agreement** if the targets of the agreement have different categories or have an empty category.

- The Phase of an agreement record is populated with “agreed” if the expiration date is later than the current date.
- The Phase of an agreement record is populated with “expired” if the expiration date is earlier than the current date.

For SLM data migration, the **Legacy SLA to new Agreement** migration setting is provided. See the following tables for its details.

Default settings

Source Table	
Source Table	sla
Target Table	sla
Filter Condition	true

Field Mappings

Source Field	**Target Field**
agreement.id	agreement.id
agreements	agreements
category	
contacts	contacts
current.phase	
customer	customer
description	description
display.category	display.category
expiration	expiration
external.support.groups	external.support.groups

Value Mappings

Target Value Mapping Field	Mapping Type	Condition	Target Value
category	jsCallback		$value=lib.MigrateSLA.migrateSLA(sourceTable['agreement.id'])
current.phase	fixedValue	expiration>tod()	agreed
current.phase	fixedValue	expiration<tod()	expired
display.category	jsCallback		$value=lib.MigrateSLA.migrateSLA(sourceTable['agreement.id'])

Related Records data migration

Process Designer-based related records retrieves information that is displayed in the related records columns directly from the screlation table. The table contains a large number of screlation records and the field values are generated based on legacy sccrelconfig settings. Therefore, you must perform data migration in order for the related records section to be displayed correctly.

An out-of-box script ("DataMigrationForRelatedRecords") is available to perform this data migration. To run the script, follow these steps:

1. Open the Script Library and search for the "DataMigrationForRelatedRecords" file.
2. Open the file and uncomment the following line:

   `//migrateScrelations`

3. Execute the script. The data migration is performed automatically.



Note

By default, only active screlation records are migrated (for time-saving purposes). Therefore, if you view inactive (closed) records in the related records section, the wrong field value may be displayed.

To migrate all the screlation records, uncomment the following line, and then execute the script:

```
//var REFRESH_All_SCRELATION_RECORDS = "true";
```

If you have a large number (millions) of screlation records, the script may take hours to update all the records. Therefore, you must ensure that the Windows client session timeout setting is large enough for the script to complete.

Service Catalog data migration

Considering that Service Catalog doesn't need to copy data from two different tables, and that it typically has a large volume of data, we recommend that you run the migration by using the batch update mode (especially for migrating a large volume of closed records).

The out-of-box example data migration is based on the following two key fields:

- status
- current.phase



Note

You can add additional fields if needed, but don't remove these two fields.

Default settings:

Source Table	svcCatalog
Target Table	svcCatalog
Filter Condition	null(current.phase)

Field Mapping:

Target Field	**Source Field**
current.phase	
status	

Value Mapping:

Target Value Mapping Field	Mapping Type	Condition	Target Value
current.phase	fixedValue	active="t"	Published
current.phase	fixedValue	null(active) or active="t"	Retired
status	fixedValue	null(active) or active="t"	Retired
status	fixedValue	active="t"	Operational

Configuration Item data migration

Considering that Configuration Item doesn't need to copy data from two different tables, and that it typically has a large volume of data, we recommend that you run the migration by using the batch update mode.

The out-of-box example data migration is based on the following key field:

- current.phase

Note

You can add additional fields if needed, but don't remove these two fields.

Default settings:

Source Table	device
Target Table	device
Filter Condition	null(current.phase)

Field Mapping:

Target Field	Source Field
current.phase	

Value Mapping:

Target Value Mapping Field	Mapping Type	Condition	Target Value
current.phase	fixedValue	istatus=null or (istatus~="Missing" and istatus~="Retired/Consumed" and istatus~="Return for Maintenance" and istatus~="Return to Supplier")	Active
current.phase	fixedValue	istatus="Missing" or istatus="Retired/Consumed" or istatus="Return for Maintenance" or istatus="Return to Supplier"	Inactive

Re-index the knowledge bases

After you remove the `Problem_Library_disabled_by_PDHD` and `KnownError_Library_disabled_by_PDHD` knowledgebases, you must run a full re-index of the knowledgebases if you use Knowledge Management in a production environment.

1.3.10. Appendix C: Run the SQL compare utility

The SQL compare utility is an informational tool that compares your existing table fields and unique key information with those of the Service Manager version you are upgrading to, and then reports the new and modified fields and unique keys that will merge into the existing tables. You can use the list of the fields and unique keys produced by the SQL compare utility to determine whether any fields or unique keys in your current system differ from those in the new version. You can also use the report to determine which new fields and unique keys you must add to RDBMS-mapped files if you choose to make the changes manually during the applications upgrade.

After running the SQL compare utility on the table fields, refer to the following table for the result types and the recommended actions.

Result type	Recommended actions
(Null)	The related field is on the id field in the licenseinfo dbdict. No further action is necessary.
Nonexistent in Service Manager 9.80 fresh installation	This field is added by tailoring. No further action is necessary.
Not to be modified to other field type by the Upgrade Utility	Manually modify this field type according to the matching solution described in Data type mismatches.
Not to be modified to other field database type by the Upgrade Utility	Manually modify this field type according to the matching solution described in Data type mismatches.
Not to be moved to alias table by the Upgrade Utility	Manually modify this field type according to the matching solution described in Data type mismatches.
To be modified by the Upgrade Utility	This field type modification will be done automatically by the Upgrade Utility during the upgrade process. You can also manually modify this field type before the upgrade to improve the upgrade performance.
To be modified by the Upgrade Utility, and the original field name xxxx will be renamed to xxxx.old	This field type modification will be done automatically by the Upgrade Utility. No further action is necessary.
To be added by the Upgrade Utility	This new field will be added automatically by the Upgrade Utility during the upgrade process. You can also manually add this new field manually before the upgrade to improve the upgrade performance.

After running the SQL compare utility on the table unique keys, refer to the following table for the result types and the recommended actions.

Result type	Recommended actions
Nonexistent in Service Manager 9.80 fresh installation	This key normally is added by tailoring. Check this key to see whether it includes any field of added primary key or unique key.
Exist in Old OOB, and will be removed by the Upgrade Utility	This key exists in Old OOB only, but doesn't exist in Service Manager 9.80. The Upgrade Utility will remove this key automatically. No further action is necessary.
To be added by the Upgrade Utility	This key doesn't exist in Old OOB, but exists in Service Manager 9.80. The Upgrade Utility will add this key automatically. No further action is necessary.
To be ignored by the Upgrade Utility	The field is added as a unique key because Service Manager doesn't support operations on tables with primary keys when the applications version is earlier than 9.32. You can manually change the key type to primary key after the system upgrade.



Note

If you are going to accept the new DBDICTS and the changes made to the DBDICTS in the upgrade, you don't need to run this utility.

Running SQL Compare

The following SQL Compare file is included when you install the Service Manager Upgrade Utility:

- upgdbdct.dta

SQL Compare returns messages for dbdict mappings that contain new fields. You can update the dbdicts to contain the fields specified by the SQL Compare applications

before you begin the applications upgrade.

> **Note**
>
> Run SQL Compare on a development system. Do not run SQL Compare on the post-upgrade system.

> **Note**
>
> Before running SQL Compare, load the preupg.bin file using Service Manager Database Manager. See Step 4: Load the applications upgrade file for more information.

To run the SQL Compare utility, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **SQL Compare Utility**.
2. Select **Run SQL Compare Utility**, and then click **Next**.
3. Type the full path to upgdbdct.dta including the final back slash (\) or forward slash (/), depending on your operating system. For example, if you copied the files to a temporary directory, the path might be:

   Windows: `c:\temp\upgrade\`

   Linux: ~/upgrade/

   > **Note**
   >
   > You must have read, write, and execute permission to the folder.

4. Click the **Next** button.

   SQL Compare returns the following message:

   `Process Complete. Please check for any additional messages.`

   The results of the SQL Compare process are stored in the sqlupgrade table. This table resets each time you run SQL Compare.

To view the SQL compare results of the table fields, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **SQL Compare Utility**.

2. Select **View SQL Field Compare Results**, and then click **Next**.
3. Click **Search**. The results are displayed in a record list.

Each table field difference found by the SQL Compare Utility appears as a separate record in the sqlupgradefield file. This record also lists the new fields that you must add to the database dictionary if you are updating your RDBMS mapped system manually.

The sqlupgradefield record provides the following information for each field you must add or modify if you are updating your RDBMS mapped system manually.

List of sqlupgradefield fields

Field	Description
File Name	The exact file name which delegates the database dictionary.
Field Name	The exact field name to add or modify to the associated database dictionary.
Field Structure	The field structure in the database dictionary.
Field Type	The field type in the database dictionary.
Field Level	The field level in the database dictionary.
Result Type	The actions happened or will happen to this field.
Field Alias Of	If this is an alias field, it contains the name of the primary field that it is an alias of. Otherwise, this field is blank.
Field SQL Table Alias	The SQL table alias of this field in the database dictionary.
Field SQL Name	The SQL name of this field in the database dictionary.
Field SQL Type	The SQL type of this field in the database dictionary.
Field SQL RC	The encoding format of this field in the database dictionary.
Exceptions	The exceptions that occurs when adding or modifying this field.

To view the SQL Compare results of the table unique keys, follow these steps:

1. From the System Navigator, click **System Administration** > **Ongoing Maintenance** > **Upgrade Utility** > **SQL Compare Utility**.
2. Select **View SQL Unique Key Compare Results**, and then click **Next**.
3. Click **Search**. The results are displayed in a record list.

Each table unique key difference found by the SQL Compare Utility appears as a separate record in the sqlupgradekey file. This record also lists the new unique keys that you must add to the database dictionary, if you want to check the possible duplicated record manually.

The sqlupgradekey record provides the following information for each field you must add or modify, if you want to check the possible duplicated record manually.

List of sqlupgradekey fields

Field	Description
File Name	The exact file name which delegates the database dictionary.
Result Type	The actions happened or will happen to this field.
Key Type	The unique key types include primary, unique, and noduplicated.
Key Field	character.
Exceptions	The exceptions that occurs when adding or modifying this unique key.

Add new fields

For the new fields to perform correctly, they must exist in both the Service Manager database dictionary and the SQL database. If you are updating your RDBMS mapped system manually, you must add them to the SQL database and update the existing Service Manager SQL mapping in the database dictionary. When you update a table in sqlsystemtables, add fields only through the database dictionary. Modifying the SQL mapping damages the file structure of the table.

Determine the correct structure

In most cases, you should add the new field to the descriptor structure. However, sometimes the Structure field contains something other than the word "descriptor".

When this occurs, add the new field to the appropriate location.

Action to take with non-descriptor fields:

In this instance	Add the field here
The field resides in another structure	Check the cm3r dbdict. There is a "middle" field of structure type.
The field is an array	If the field is an array, the field name appears twice in the new field list. Check any field of array type. The root field has array type, but the same name field normally has the real type (for example, VARCHAR(60)). Use the first entry to determine the structure where you should add the array. The Structure field in the second entry reflects both the structure for the array (unless it uses the descriptor structure) and the name of the array itself.
The field is part of an array of structures	Check the kmcategory dbdict permission field, which is a structured array field.

Note

When adding fields to an array of structures, add them in the same order as they appear in the sqlupgrade record.

Process the NVARCHAR SQL type

Although the upgrade version uses the VARCHAR SQL type for fields, SQL Compare and Upgrade Utility retain the use of the NVARCHAR SQL type after the upgrade. For example, if the SQL type of a field is NVARCHAR(60) in the current version and the SQL type of this field in the upgrade version should be VARCHAR(60), the SQL type of this field will remain NVARCHAR(60) in the upgraded version.

If the current and updated versions of a field have different length SQL types, SQL Compare and Upgrade Utility use the longer length. For example, if the field SQL Type

is NVARCHAR(60) in the current version and if the SQL Type of this field in the upgrade version is VARCHAR(100), the SQL Type of this field will be NVARCHAR(100) in the upgraded version.

If you want to tailor the SQL type of a field from VARCHAR to NVARCHAR and retain the same length, be sure to perform tailoring from the database instead of from the Service Manager database dictionary. Otherwise, the tailoring will be ignored by Service Manager.

1.3.11. Appendix D: Mapping between legacy security profiles and current PD security roles

This appendix describes the mapping relationships between legacy security profiles and current PD security roles in Service Manager modules including Change Management, Problem Management, Incident Management, Configuration Management, Service Level Management, and Service Desk.

Topics in this appendix include:

Change Management security profiles mappings

Service Desk security profiles mappings

Problem Management security profiles mappings

Incident Management security profiles mappings

Service Level Management security profiles mappings

Configuration Management security profiles mappings

1.3.11.1. Change Management security profiles mappings

If you have upgraded from a previous version of Change Management, which is not Process Designer based, see the following tables for Change Management security profiles mappings.

Mapping between previous security profiles and current PD security roles

The following table lists the mapping relationship between previous Change Management security profiles and current Process Designer security roles in the Change Management module.

Change profile	Security role
DEFAULT	default
SD agent/manager	change requestor
change analyst change	change owner
change analyst tasks	change owner
change approver	change advisory board (CAB)
change coordinator change	change coordinator
change coordinator tasks	change coordinator
change manager	change manager
emergency group	emergency CAB
Initiator	change requestor
problem manager	change requestor
Sysadmin	system administrator

In many cases, change profile rights used in versions earlier than 9.40 have been merged into much fewer security rights. For example, the Expert right represents several change profile rights. The Process Designer Change Management workflows reference security roles; after upgrade security roles that were converted from change profiles are used if they exist with the same name. If a workflow uses a security role that was not converted from a change profile, the system assigns default rights to the security role. It is important to inspect the security roles and modify them as appropriate for use in any of the Process Designer workflows.

Field mapping between security profiles settings and PD security rights/settings

The following table outlines the field mapping between legacy change profile settings and the current Process Designer security role rights and settings.

Change profile settings field names	Security roles settings field names
Copy and open	New
New	New
Open	New
Alternate views	View
Can count records	View
Can notify	View
Clear	View
fill	View
find	View
irquery	View
Show parent	View
Tasks	View
View	View
Alerts	Expert
apprdism	Expert
Can mass approve	Expert
Complex Mass Update	Expert
Duplicates	Expert
Expert Search	Expert
List Pages	Expert

Change profile settings field names	Security roles settings field names
Template Mass Update	Expert
Update when closed	Expert
Change category	Admin
Change phase	Admin
Can override approvals	Admin
Allowed categories	Allowed categories
Allowed status	Allowed Status
Append Query	Append Query
Approval Groups	approver.of in $G.myGroups
Area	Area
Can access approvals	Can Approve
Can Delegate Approvals	Can Delegate Approvals
Change Manage Format	Change Manage Format
Close	Delete/Close
Default Change Category	Default Category
Default Task Category	Default Task Category
Inefficient Query	Allow Inefficient Query
Initial Change View	Initial Change View
Initial Format	Initial Format
Initial Task View	Initial Task View

Change profile settings field names	Security roles settings field names
Manager Group	member.of in $G.myGroups
Modify Template	Modify Template
Name	Role
QBE Format	List Format
Reopen	Reopen
Skip Warning	Skip Inefficient Query Warning
Task Manage Format	Task Manage Format
Update	Update

1.3.11.2. Service Desk security profiles mappings

If you have upgraded from a previous version of Service Desk, which is not Process Designer based, see the following tables for Service Desk security profiles mappings.

Mapping between previous security profiles and current PD security roles

The following table lists the mapping relationship between previous Service Desk security profiles and current PD security roles in the Service Desk module.

Security Profile	Security Role/Area
DEFAULT	DEFAULT/Service Desk
approver	approver/Service Desk
helpdesk tech	helpdesk tech/Service Desk
initiator	initiator/Service Desk
self service	self service/Service Desk
service desk agent	service desk agent/Service Desk
service desk manager	service desk manager/Service Desk
sysadmin	sysadmin/Service Desk
N/A	service desk analyst/service desk process owner/Service Desk

Field mapping between security profiles settings and PD security rights/settings

The following table lists the mapping of fields in legacy Service Desk security profiles and Process Designer security roles.

Security profile settings	Process Designer security rights and settings
New	New
Close	Delete/Close
Update	Update
View	View
Can notify	View
Alternate views	View
Advanced Search	View
Template Mass Update	Expert
Complex Mass Update	Expert
Approval groups	Approval Groups
Allowed Statuses	Allowed Statuses
Can Create Personal Views	Can Create Personal Views
Can Create System Views	Can Create System views
Lock on Display	Lock on Display
Modify Templates	Modify Templates
Delegate Approvals	Can Delegate Approvals
Initial view	Initial View
Edit Format	Edit Format
Search Format	Search Format
List Format	List Format

Security profile settings	Process Designer security rights and settings
Manage Format	Manage Format
Print Format	Print Format
ESS initial format	ESS initial format
ESS edit format	ESS edit format
ESS search format	ESS search format
ESS list format	ESS list format
Default Template	Default Template
Append query	Append query
New Thread: View → Search	New Thread: View → Search
New Thread: Search → List	New Thread: Search → List
New Thread: List → Edit	New Thread: List → Edit
New Thread: View → Edit	New Thread: View → Edit

1.3.11.3. Problem Management security profiles mappings

If you have upgraded from a previous version of Problem Management, which isn't Process Designer based, see the following tables for Problem Management security profiles mappings.

Mapping between previous security profiles and current PD security roles

The following table lists the mapping relationship between previous Problem security profiles and current PD security roles in the Problem module.

Security Profile	Security Role/Area
DEFAULT	DEFAULT/Problem
initiator	initiator/Problem
problem analyst	problem analyst/Problem
problem coordinator	problem coordinator/Problem
problem manager	problem manager/Problem
reviewer	reviewer/Problem
sysadmin	systadmin/Problem
N/A	problem task assignee/Problem
N/A	problem process owner/Problem

Field mapping between security profiles and PD security rights/settings

The following table lists the mapping of fields in legacy Problem security profiles and Process Designer security roles.

Security profile settings	Process Designer security rights and settings
Allow Inefficient Query	Allow Inefficient Query
Allowed Statuses	Allowed Statuses
Append Query	Append Query
Assignment Groups	Assignment Groups
Can Create Personal Views	Can Create Personal Views
Can Create System Views	Can Create System views
Create Known Error	Create Known Error
Close	Delete/Close
Edit Format	Edit Format
Templates Mass Update	Expert
Complex Mass Update	Expert
Initial Inbox	Initial View
List Format	List Format
Lock on Display	Lock on Display
Manage Format	Manage Format
Change Phase	N/A
Post to Knowledge	N/A
Initial Format	N/A
Open Script	N/A
Resolution Script	N/A

Security profile settings	Process Designer security rights and settings
New	New
New Thread: Inbox → Edit	New Thread: Inbox → Edit
New Thread: Inbox → Search	New Thread: Inbox → Search
New Thread: List → Edit	New Thread: List → Edit
New Thread: Search → List	New Thread: Search → List
Print Format	Print Format
Reopen	Reopen
Search Format	Search Format
Skip Query Warning	Skip Inefficient Query Warning
Update	Update
Change Category	Update
View	View
Views	View
Count	View
Advanced Search	View

Note

N/A means the previous fields are obsolete and aren't mapped to PD security rights/settings.

1.3.11.4. Incident Management security profiles mappings

If you have upgraded from a previous version of Incident Management, which isn't Process Designer based, see the following tables for Incident Management security profiles mappings.

Mapping between previous security profiles and current PD security roles

The following table lists the mapping relationship between previous Incident security profiles and current PD security roles in the Incident module.

Security Profile	Security Role/Area
DEFAULT	DEFAULT/Incident
configuration auditor	configuration auditor/Incident
incident analyst	incident analyst/Incident
incident coordinator	incident coordinator/Incident
incident manager	incident manager/Incident
initiator	initiator/Incident
operator	operator/Incident
problem manager/coord	problem manager/Incident problem coordinator/Incident
service desk agent	service desk agent/Incident
service desk manager	service desk manager/Incident
service tech	service tech/Incident
sysadmin	sysadmin/Incident
N/A	incident task assignee/Incident
N/A	incident process owner/Incident

Field mapping between security profiles and PD security rights/settings

The following table lists the mapping of fields in legacy Incident security profiles and Process Designer security roles.

Security profile settings	Process Designer security rights and settings
New	New
Close Inactivate Mass inactivate	Delete/Close
Update Change category	Update
View Log Can notify Search for duplicates Alternate views Can use callback list Advanced search	View
Reopen	Reopen
Mark problem candidate Can suspend Can unsuspend Notes Override Template Mass Update Complex Mass Update	Expert
Allow inefficient query	Allow Inefficient Query
Skip query warning	Skip Inefficient Query Warning
Can create personal views	Can Create Personal Views
Can create system views	Can Create System Views
Lock on display	Lock On Display
Modify Template	Modify Template

Security profile settings	Process Designer security rights and settings
Allowed statuses	Allowed Statuses
QBE format	List Format
Search format	Search format
Manage format	Manage Format
Initial view	Initial View
Auto-notify format	Notify Template **Note** Use this setting to configure the template name for the **Notify** action.
Incident macro mail format Default template Default category	N/A
Assignment groups	Assignment groups
Authorized categories	Allowed Categories
New thread: View → Search	New thread: View → Search
New thread: Search → List	New thread: Search → List
New thread: List → Edit	New thread: List → Edit
New thread: View → Edit	New thread: View → Edit



Note

N/A means the previous fields are obsolete and aren't mapped to PD security rights/settings.

1.3.11.5. Service Level Management security profiles mappings

If you have upgraded from a previous version of Service Level Management, which is not Process Designer based, see the following tables for Service Level Management security profiles mappings.

Mapping between previous security profiles and current PD security roles

The following table lists the mapping relationship between previous Service Level Management security profiles and current PD security roles in the Service Level Management module.

Security Profile	Security Role/Area
DEFAULT	DEFAULT/Service Level Management
administrator	administrator/Service Level Management
sysadmin	sysadmin/Service Level Management

Field mapping between security profiles and PD security rights/settings

Note

In the following two tables, AND and OR relate to whether a mapped security role has the corresponding right. For example, if any of the Service Level Agreement, Response Objectives, or Availability Objective has the "New" security profile setting, then the Process Designer security rights and settings will also be "New."

For Service Level Agreements, Response Objectives and Availability Objectives

Security profile settings		Process Designer security rights and settings
New	OR	New
Delete	OR	Delete/Close
Update	OR	Update
View	OR	View
Allowed Statuses	AND	Allowed Statuses

SLT Catalog Records

Security profile settings		Process Designer security rights and settings
New Delete Update View	OR	Manage SLT Catalog Records

Settings

Security profile settings	Process Designer security rights and settings
View Monthly Aggregate Results	View Monthly Aggregate Results
View Monthly Response Time Results	View Monthly Process Time Results
View Response Detail Records	View Process Detail Records
View SLA Active Records	View SLM Active Records
View Monthly Availability Results	View Monthly Service Results
View Outage Detail Records	View Outage Detail Records
View Outage Event Records	View Outage Event Records

1.3.11.6. Configuration Management security profiles mappings

If you have upgraded from a previous version of Configuration Management, which is not Process Designer based, see the following tables for Configuration Management security profiles mappings.

Mapping between previous security profiles and current PD security roles basic rights

The following table lists the mapping relationship between previous Configuration Management security profiles and current Process Designer security roles basic rights in the Configuration Management module.

Configuration profile settings field names	Security roles basic rights settings field names
New	New
Delete	Delete/Close
Update	Update
View	View
Allowed statuses	Allowed statuses
Template Mass Update	Expert
Complex Mass Update	Expert
Mass Delete	Mass Delete
Advanced search	View
Allow inefficient query	Allow Inefficient Query
Skip query warning	Skip Inefficient Query Warning
Alternate views	View
Change CI type	Change CI type
Open incident	Open incident
Request change	Request change
Export	View
Required password	N/A
Confirm on end	N/A
Audit	Audit

Note

N/A means the previous fields are obsolete and aren't mapped to PD security rights/settings.

Field mapping between security profiles settings and PD security roles additional rights/settings

The following table outlines the field mapping between legacy Configuration profile settings and the current Process Designer security roles additional rights and settings.

Configuration profile settings field names	Security roles additional rights settings field names
Search format	Search format
Threading View → Search	New thread: View → Search
Threading Search → List	New thread: Search → List
Threading List → Edit	New thread: List → Edit
Threading View → Edit	New thread: View → Edit

Field mapping between security profiles settings and PD security roles basic rights

The following table lists the mapping relationship between previous Configuration Management security profiles and current Process Designer security roles basic rights in the Configuration Management module.

Configuration profile settings field names	Security roles basic rights settings field names
Restrict to these types	Allowed Category, which includes all other CI Types that aren't in the **Restricted to these types** list.

© Copyright 2026 Open Text

For more info, visit https://docs.microfocus.com